Hochschule Karlsruhe

Fakultät für Maschinenbau und Mechatronik

Studiengang Robotik und Künstliche Intelligenz in der Produktion

Forschungs- und Entwicklungsprojekt

Autonome Fahrzeugsteuerung für Carrera Hybrid mittels Deep Reinforcement Learning

Konzeption, 2D-Prototyping und empirische Evaluation einer sample-effizienten Teacher-Student-Pipeline als Grundlage für den Transfer in die 3D-Simulation

Vorgelegt von Felix Faaß (Matrikelnummer [Matrikelnummer])
Magnus Neumann (Matrikelnummer [Matrikelnummer])

Projektbetreuer Prof. Dr.-Ing. habil. Björn Hein
Hochschule Karlsruhe

Abgabedatum Karlsruhe, den [Abgabedatum]

Selbstständigkeitserklärung

Wir versichern hiermit wahrheitsgemäß, dass wir das vorliegende Forschungs- und Entwicklungsprojekt mit dem Titel

> **Autonome Fahrzeugsteuerung für Carrera Hybrid mittels Deep Reinforcement Learning**
> *Konzeption, 2D-Prototyping und empirische Evaluation einer sample-effizienten Teacher-Student-Pipeline als Grundlage für den Transfer in die 3D-Simulation*

selbstständig und ohne unzulässige fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen sind, haben wir als solche kenntlich gemacht.

Die Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht und auch nicht veröffentlicht worden.

Erklärung zum Einsatz generativer KI-Werkzeuge

Diese Arbeit wurde unter Einsatz generativer Text- und Code-KI-Werkzeuge erstellt. Durch diese Erklärung am Anfang der Arbeit gelten das Dokument und die nachfolgend aufgeführten Punkte als gekennzeichnet; auf eine kleinteilige Markierung einzelner Sätze im Haupttext wurde zugunsten des Leseflusses verzichtet.

Verwendete Werkzeuge: Claude (Anthropic), Gemini (Google) und ChatGPT (OpenAI).

- **Quellcode (Kapitel ??):** Die Simulationsumgebung, die Auswertungsskripte und die Notebook-Zellen wurden unter Einsatz generativer Coding-Werkzeuge erstellt. Sämtliche Bestandteile wurden vom Projektteam geprüft, ausgeführt und fachlich verantwortet; die berichteten Messwerte stammen ausnahmslos aus eigenen Läufen.
- **Formatierung und Struktur (global):** Die Werkzeuge wurden zur Unterstützung der LaTeX-Formatierung, der Erstellung von Tabellenstrukturen sowie zur Aufbereitung von Quellcode-Darstellungen eingesetzt.

- **Sprachliche Überarbeitung (global):** Im gesamten Dokument wurden die Werkzeuge unterstützend für Rechtschreibprüfung, Grammatikkorrektur und Formulierungsvorschläge eingesetzt.
- **Formulierungshilfe:** Die Kurzfassung, der englische Abstract, die Zusammenfassung sowie die Ausformulierung der in den Projektnotizen stichpunktartig festgehaltenen Ergebnisse wurden unter Einbezug KI-basierter Formulierungsvorschläge erarbeitet und anschließend vom Projektteam fachlich geprüft.

Karlsruhe, den [Abgabedatum]

Felix Faaß

Magnus Neumann

Kurzfassung

Das Training bildbasierter Reinforcement-Learning-Agenten für die Fahrzeugregelung am physikalischen Grenzbereich ist in fotorealistischen 3D-Simulatoren rechenintensiv und sample-ineffizient. Ein Trainingslauf bindet dort Tage an Rechenzeit, bevor überhaupt feststeht, ob die gewählte Beobachtung, die Belohnungsfunktion und der Lernalgorithmus zueinander passen. Die vorliegende Arbeit begegnet diesem Problem mit einer vorgeschalteten Prototyping-Stufe: Für die frei fahrenden Miniaturfahrzeuge des Systems *Carrera Hybrid* (Maßstab etwa 1:43) wurde eine leichtgewichtige, physikalisch kalibrierte 2D-Simulationsumgebung auf Basis von Gymnasium aufgebaut, in der Beobachtungsräume, Belohnungsfunktionen und Lernverfahren isoliert und zeiteffizient untersucht werden können.

Untersucht wurden fünf Beobachtungsvarianten – niedrigdimensionale Raycasts, fahrzeugfester Einzelbildausschnitt, globale Einzelbildansicht, hybride Kombination aus Bild und Skalarparametern sowie gestapelte Bildfolgen – jeweils in Verbindung mit den Algorithmen Proximal Policy Optimization (PPO) und Soft Actor-Critic (SAC). Als methodischer Kern wurde eine Pipeline aus privilegierter Expertendestillation erprobt: Ein Lehrer-Agent auf Raycast-Daten konvergiert in etwa zwölf Minuten, erzeugt unter Rausch-Injektion einen Demonstrationsdatensatz mit Rettungsmanövern, auf dem ein visuelles neuronales Faltungsnetz (CNN) überwacht vortrainiert wird, bevor SAC den Agenten online weiter optimiert.

Das zentrale Ergebnis ist ein negatives: Nach der Umstellung des Fahrzeugmodells von geschätzten auf videometrisch gemessene Parameter konvergierten die Kamera-Agenten nicht mehr. Ein Kontrollversuch, der alle Randbedingungen konstant hält und lediglich die Beobachtung gegen exakte Zahlenwerte tauscht, löst dieselbe Aufgabe in 54 min mit elf fehlerfreien Runden. Damit sind Physik, Belohnungsfunktion, Algorithmus, Replay-Buffer und Diskontfaktor als Ursache ausgeschlossen; verantwortlich ist eine Wahrnehmungsschwelle. Unterhalb von 0,64 m/s verschiebt sich das Fahrzeug in der 166×100 Pixel großen Beobachtung um weniger als einen Bildpunkt je Einzelbild, sodass der Bildstapel keine Geschwindigkeitsinformation mehr trägt. Eine Ablationsstudie am Lehrer-Agenten bestätigt, dass dessen Leistung vollständig an der Geschwindigkeitsinformation hängt. Als zweiter, methodisch ebenso wichtiger Befund erwies sich ein fester Startpunkt als Fehlerquelle: Der bis dahin beste Agent fiel aus zufälligen Startlagen von 5080 auf -1829 Belohnungspunkte und von neun auf 0,6 Runden – er hatte keine Fahrpolitik, sondern eine Trajektorie auswendig gelernt.

Die Arbeit dokumentiert darüber hinaus vier reproduzierte Fehlerklassen des Belohnungsentwurfs, die Wechselwirkung zwischen Fahrzeugdynamik und allen zeitbasierten Hyperparametern sowie einen speicheroptimierten Replay-Buffer für Bildstapel. Aus diesen Befunden wird eine konkrete,

nach Reihenfolge und Prüfkriterien geordnete Handlungsanweisung für den Transfer nach NVIDIA Isaac Sim abgeleitet, deren Umsetzung Gegenstand der zweiten Projektphase ist.

Abstract

Training vision-based reinforcement learning agents for vehicle control near the physical limit is computationally expensive and sample-inefficient in photorealistic 3D simulators. A single training run occupies days of compute before it becomes clear whether the chosen observation space, reward function and learning algorithm fit together at all. This work addresses the problem with an upstream prototyping stage: for the free-running miniature vehicles of the *Carrera Hybrid* system (scale approximately 1:43), a lightweight, physically calibrated 2D simulation environment was built on top of Gymnasium, allowing observation spaces, reward functions and learning algorithms to be examined in isolation and at low cost.

Five observation variants were investigated – low-dimensional raycasts, a vehicle-fixed image crop, a global single-frame view, a hybrid combination of image and scalar telemetry, and stacked image sequences – each in combination with the algorithms Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). The methodological core is a privileged expert distillation pipeline: a teacher agent operating on raycast data converges within roughly twelve minutes, generates a demonstration dataset containing recovery maneuvers through action noise injection, and this dataset is used to pretrain a visual convolutional neural network (CNN) in a supervised fashion before SAC optimises the agent online.

The central result is a negative one. After replacing the estimated vehicle model with parameters measured from video footage, the camera-based agents stopped converging. A control experiment that holds every boundary condition constant and merely exchanges the observation for exact numerical values solves the same task in 54 min, completing eleven error-free laps. This rules out physics, reward function, algorithm, replay buffer and discount factor as causes; the responsible factor is a perception threshold. Below 0,64 m/s the vehicle moves by less than one pixel per frame within the 166×100 pixel observation, so the frame stack no longer carries velocity information. An ablation study on the teacher agent confirms that its performance depends entirely on that velocity signal. A second, methodologically equally important finding concerns the fixed starting position: the best agent up to that point dropped from 5080 to -1829 reward points and from nine laps to 0,6 laps when evaluated from randomised starting poses – it had memorised a trajectory rather than learned a driving policy.

The report further documents four reproduced failure classes of reward design, the coupling between vehicle dynamics and every time-based hyperparameter, and a memory-optimised replay buffer for frame stacks. From these findings, a concrete transfer procedure for NVIDIA Isaac Sim is derived, ordered by sequence and acceptance criteria; its implementation is the subject of the second project phase.

Inhaltsverzeichnis

Selbstständigkeitserklärung ii

Kurzfassung iv

Abstract vi

Abkürzungsverzeichnis x

Nomenklatur xi

Abbildungsverzeichnis xiv

Tabellenverzeichnis xvi

1 Einleitung 1
- 1.1 Motivation . . . 1
- 1.2 Projektstruktur . . . 2
- 1.3 Ziele dieser Arbeit . . . 3
  - 1.3.1 Einblick in die Projektstruktur . . . 3
- 1.4 Aufbau der Arbeit . . . 5

2 Grundwissen und Begriffserklärungen 6
- 2.1 Theoretische Grundlagen . . . 6
  - 2.1.1 Markov-Entscheidungsprozess (MDP) . . . 6
  - 2.1.2 Partiell beobachtbarer MDP (POMDP) . . . 7
  - 2.1.3 Policy, On-Policy und Off-Policy, modellfrei und modellbasiert . . . 8
  - 2.1.4 PPO und SAC . . . 9
  - 2.1.5 Imitation Learning, Behavioral Cloning und Covariate Shift . . . 10
  - 2.1.6 Privilegiertes Lernen (Learning by Cheating) . . . 11
  - 2.1.7 Frame-Stacking und hybrider Input . . . 12
  - 2.1.8 Replay-Buffer, Pretraining und Online- gegen Offline-Training . . . 13
- 2.2 State of the Art . . . 14

3 Phase 1: 2D-Prototyping 16
- 3.1 Idee . . . 16
- 3.2 Umsetzung . . . 18
  - 3.2.1 Arbeitsweise: Notebook mit ausgelagerten Modulen . . . 18

3.2.2 Vorbereitung für autonome Agenten . . . 18
3.2.3 Entwurf der Belohnungsfunktion . . . 19
3.2.4 Erstes einfaches Training als Machbarkeitsnachweis . . . 20
3.2.5 Systematische Erprobung der Eingabekonzepte . . . 21
3.2.6 Physikalische Überarbeitung auf gemessene Realdynamik . . . 22
3.2.7 Das Subpixel-Wahrnehmungsproblem . . . 24
3.2.8 Technische Lösungsansätze gegen die Wahrnehmungsschwelle . . . 27
3.2.9 Reward Engineering: Evolution zu `v3_rundenzeit` . . . 29
3.2.10 Der Diskontfaktor als teuerster Einzelfehler . . . 31
3.2.11 Systematischer Algorithmenvergleich: PPO gegen SAC . . . 32
3.2.12 Software- und Buffer-Optimierungen . . . 33
3.3 Erkenntnisse (Gut und Schlecht) . . . 35
3.4 Fallen und Fehler . . . 37

4 Umsetzung in 3D (Isaac Sim / Isaac Lab) 42
4.1 Einstieg in Isaac Sim und Isaac Lab . . . 42
4.2 Modellierung einer realitätsnahen Umgebung . . . 43
4.2.1 Fahrzeug . . . 43
4.2.2 Antrieb und Fahrverhalten . . . 44
4.2.3 Strecke und Streckenbegrenzung . . . 45
4.2.4 Kamera und Beobachtung . . . 46
4.2.5 Belohnung und Episodenende . . . 47
4.3 Vergleich zur 2D-Python-Umgebung . . . 48
4.3.1 Was übernommen und geändert wird . . . 48
4.3.2 Parallele Umgebungen und Erfahrungsspeicher . . . 48

5 Trainingsablauf in Isaac 51
5.1 Die Trainingspipeline in 3D . . . 51
5.2 Training des Kamera-Agenten . . . 53
5.3 Probleme auf dem Weg zum ersten Agenten . . . 54
5.4 Einfrieren des Encoders . . . 54
5.5 Der erste fahrende Kamera-Agent . . . 56
5.5.1 Genutzte Parameter . . . 56
5.5.2 Das Ergebnis . . . 57
5.5.3 Eine Einschränkung . . . 58
5.6 Der zweite Agent . . . 58
5.6.1 Verallgemeinern der Erscheinung . . . 59
5.6.2 Das Training . . . 60
5.6.3 Das Ergebnis . . . 60
5.7 Vergleich der beiden Modelle . . . 63
5.8 Test des zweiten Agenten auf anderen Streckenführungen . . . 63
5.8.1 Test ohne Nachtraining . . . 64
5.8.2 Weitertrainieren von Agent 2 gegen Neustart . . . 64

6 Ergebnisse **67**
6.1 Die aktuelle Trainingspipeline 67
6.2 Quantitative Ergebnisse 69
6.3 Übertragbarkeit in die 3D-Simulation 71

7 Fazit und Ausblick **73**
7.1 Fazit zu Phase 1 73
7.2 Fazit zu Phase 2 74
7.3 Ausblick: Phase 3 – Sim-to-Real-Transfer 75

Quellenverzeichnis **77**

Abkürzungsverzeichnis

BC	Behavioral Cloning
CNN	Convolutional Neural Network
IL	Imitation Learning
MDP	Markov-Entscheidungsprozess
MLP	Multi-Layer Perceptron
POMDP	partiell beobachtbarer Markov-Entscheidungsprozess
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
SAC	Soft Actor-Critic

Nomenklatur

α	Entropiekoeffizient in SAC [–]
Δt	Zeitschritt der Simulation [s]
δ	Lenkwinkel am Rad [rad]
γ	Diskontfaktor [–]
$\mathcal{A}$	Aktionsraum
$\mathcal{D}$	Demonstrationsdatensatz
$\mathcal{H}$	Entropie der Policy
$\mathcal{S}$	Zustandsraum
μ	Reibwert zwischen Reifen und Fahrbahn [–]
Ω	Beobachtungsraum
ω	Gierrate [rad/s]
π	Policy, Entscheidungsregel des Agenten
σ	Standardabweichung des injizierten Rauschens
θ	Parameter des neuronalen Netzes
ξ_t	injizierter Störterm auf die Expertenaktion
a	Längsbeschleunigung [m/s^2]

a_t	Aktion zum Zeitschritt t
a_t^*	Expertenaktion zum Zeitschritt t
B_{lap}	Bonus je Zielüberquerung [–]
C_{crash}	Strafe bei Kollision [–]
C_{wrong}	Strafe bei falscher Fahrtrichtung [–]
G_t	diskontierter Ertrag ab Zeitschritt t
H	wirksamer Planungshorizont [Zeitschritte]
k	Anzahl gestapelter Einzelbilder
L	Radstand des Einspurmodells [m]
o_t	Beobachtung zum Zeitschritt t
$Q^\pi(s, a)$	Aktionswertfunktion unter der Policy π
r_t	Belohnung zum Zeitschritt t
s_t	Zustand zum Zeitschritt t
s_{obs}	Maßstab in der Beobachtung [px/m]
v	Fahrgeschwindigkeit [m/s]
$V^\pi(s)$	Zustandswertfunktion unter der Policy π
$v_{\max}$	Höchstgeschwindigkeit des Fahrzeugmodells [m/s]
v_{slow}	Schwelle der Langsam-Strafe [m/s]
w_v	Gewicht des Tempo-Anteils der Belohnung [–]
w_{imp}	Gewicht des Rundenzeit-Bonus [–]

w_{slow} Gewicht der Langsam-Strafe [–]

w_{smooth} Gewicht der Lenk-Glättungsstrafe [–]

Abbildungsverzeichnis

3.1 Grundlage der Streckengeometrie. Aus dem Bildschirmfoto (a) isolieren Farbmasken im HSV-Raum die farbig markierten Randstreifen; aus der Maske (b) entstehen die Splines der Streckenbegrenzung und daraus die Mittellinie. 19

3.2 Kinematik aus der Videomessung. (A) Weg-Zeit-Messung und Modellfit für Anfahren und zwei Bremsversuche. (B) Daraus abgeleitete Geschwindigkeitsverläufe; die Höchstgeschwindigkeit ist aus diesen Aufnahmen nicht bestimmbar, weil das Fahrzeug im Bildausschnitt nicht sättigt. (C) Vergleich der gemessenen Bremsvorgänge mit dem bis dahin verwendeten Simulationsmodell, das rund vierzehnmal zu stark bremst. 23

3.3 Fahrlinienvergleich. Grün die Linie des Lidar-Experten, orange mehrere Episoden des Kamera-Agenten `2230` bei 250 × 150. Der Kamera-Agent pendelt über die Streckenbreite und verunglückt reproduzierbar außen in der linken Haarnadel bei 1,11 m/s; der Experte durchfährt dieselbe Stelle mit 1,04 m/s. Erzeugt mit `src/eval/fahrlinien.py`. 28

3.4 Streuung der Startpunkte nach der Korrektur. Alle möglichen Startlagen liegen auf der Geraden *vor* der Ziellinie, damit die Rundenzählung ihren Sinn behält; grün markiert sind zwölf tatsächlich gezogene Startpunkte. 39

4.1 Sichtbares Fahrzeugmodell in Isaac Sim (links, Porsche 911 GT2 RS Weissach) und reales Fahrzeug der Carrera-Hybrid-Bahn (rechts, Porsche 991 GT3 R). . . . 44

4.2 Fahrzeugmodell ohne Lackierung, wie es der Agent im Training gesehen hat (links), und mit Lackierung (rechts). 45

4.3 Begrenzungslinien als Kontrollanzeige in Isaac Sim (grün: Fahrbahnkante, magenta: Außenkante der Randsteine, gelb: Start- und Ziellinie). 46

4.4 Trainingsstrecke mit den abgeleiteten Grenzen. Grün: Fahrbahnkante; Magenta: Außenkante der Randsteine; Gelb: Start- und Ziellinie; Orange: Startzone; Blau: Mittellinie. 47

5.1 Trainingspipeline in vier Stufen. Oben der gemeinsame Ablauf, darunter die Unterschiede zwischen Agent 1 (blau) und Agent 2 (orange). 52

5.2 Gegenversuch Einfrieren: identischer Aufbau mit eingefrorenem (blau) und nicht eingefrorenem Encoder (orange) auf `kurs_A`. 55

5.3 Trainingsverlauf von Agent 1 (Werte mit Explorationsrauschen). Blau: erstes Training, orange: Fortsetzung mit doppelt so langen Episoden; gestrichelt: Auftauen der Faltungsschichten. 57

5.4 Die zwölf Darstellungen der Trainingsstrecke: drei Untergründe (Zeilen) und vier Helligkeiten der Fahrbahn (Spalten). 59
5.5 Trainingsverlauf von Agent 2 im Vergleich zu einem früheren Versuch ohne Einfrieren (Werte mit Explorationsrauschen). Gestrichelt: Auftauen; gepunktet: erste vollständige Runde. 61
5.6 Agent 2 auf den zwölf Darstellungen (je 20 Episoden à 66,67 s, deterministisch). Bei 6 Runden liegt das Zeitlimit. Die Werte der unteren Darstellungen sind Grenzfälle und schwanken zwischen Messungen; für `fb105_holz` lagen sie in verschiedenen Messungen zwischen 0 und 4 Runden. 62
5.7 Weitertrainieren (durchgezogen) gegen Neustart (gestrichelt) auf `kurs_A` (blau) und `kurs_D` (orange). Bei 2 Mio. Übergängen wird der Encoder der Neustarts aufgetaut. 65

Tabellenverzeichnis

3.1 Spezifikationen der 2D-Simulationsumgebung und ihre Herkunft. Werte ohne Quellenangabe stammen aus der Implementierung in `src/envs/carrera_2d_env.py`. *Anmerkung zum Radstand:* Aus der Differenz der gemessenen Achsabstände ergäben sich 48 mm; die Abweichung zum implementierten Wert von 58 mm ist nicht dokumentiert und in Abschnitt **??** als offener Punkt vermerkt. 17
3.2 Gewichte der Belohnungsfunktion `v3_rundenzeit`. 20
3.3 Reihenfolge der Vergleichsläufe zu den Eingabekonzepten mit den zugehörigen Notebook-Zellen. 21
3.4 Gegenüberstellung der beiden Fahrzeugmodelle. Die Werte für `measured_v2` sind Vollgas- beziehungsweise Vollbremswerte aus dem Stand. 24
3.5 Bildverschiebung je Einzelbild in der Beobachtung des Agenten, für beide erprobten Auflösungen. 25
3.6 Kontrollversuch, Auswertungsstand 14.08.2026. Alle Modelle wurden zur Vergleichbarkeit in derselben Umgebung und mit demselben Belohnungsstand neu gefahren (`python -m src.eval.vergleich`). Die Zahlen sind untereinander, aber nicht mit der späteren Nachauswertung in Tabelle 6.2 vergleichbar – siehe die Anmerkung zur Vergleichbarkeit in Abschnitt 6.2. 26
3.7 Ablation am trainierten Lidar-Agenten: gezielte Verfälschung einzelner Beobachtungsanteile. 27
3.8 Gespreizter Framestapel gegen aufeinanderfolgende Bilder, beide bei 250 × 150 und 1,5 Mio. Schritten. 29
3.9 Zerlegung einer Episode des Kamera-Agenten nach Belohnungsposten (Stand 850 000 Schritte, 250 × 150, 193 Frames). 31
3.10 PPO-Läufe vom 26.07.2026, alle unter dem alten Fahrzeugmodell `force_drag_v1`. `BSP` bezeichnet die Backbone-Transplantation. 32
3.11 Kreuzvergleich von festem und streuendem Startpunkt. Lauf `2254` wurde mit festem, Lauf `2215` mit streuendem Startpunkt trainiert; beide wurden unter beiden Bedingungen ausgewertet. 38

4.1 Verwendete Hardware und jeweilige Aufgaben während des Trainings. 43
4.2 Gemessene Fahrzeugdaten im Kollisionsmodell. 44
4.3 Terme der Belohnung je Schritt. 48
4.4 Bestandteile der 2D-Umgebung aus Phase 1 und der 3D-Umgebung im Vergleich. 49

5.1 Gegenversuch Einfrieren, Werte im Trainingsverlauf (Episodenlänge in Schritten / Anteil inaktiver Neuronen der 512er-Schicht / Streuung des Lenkbefehls). 55

5.2 Einstellungen von Agent 1. 56
5.3 Die beiden fahrenden Kamera-Agenten. 63
5.4 Agent 2 ohne Nachtraining (deterministisch). 64
5.5 Übergänge bis zur ersten vollständigen Runde. 65

6.1 Konfiguration der aktuellen Pipeline-Stufe 4, entnommen aus `train_config.json` des Laufs `09_Vision_SAC_Finetuned_20260820_2215`. 68
6.2 Nachauswertung aller Modelle unter `measured_v2` im aktuellen Codestand, je drei deterministische Episoden vom festen Startpunkt (`models/vergleich.json`). Die Rundenzahl zählt die erste Zielüberquerung als Start und nicht als Runde. 70
6.3 Abnahmekriterien für die 3D-Umsetzung. Sie sind aus den Ergebnissen von Phase 1 abgeleitet und in dieser Reihenfolge zu prüfen; die Messwerte trägt Phase 2 nach. 72

1 Einleitung

1.1 Motivation

Unter dem Begriff *Physical AI* wird die Übertragung datengetriebener Entscheidungsverfahren auf physische Systeme zusammengefasst – also der Schritt von einem Modell, das Text oder Bilder verarbeitet, zu einem Modell, das Kräfte, Trägheiten und Sensorlatenzen beherrschen muss. Der naheliegende Träger für solche Experimente ist ein Roboterarm oder eine mobile Plattform. Beides ist teuer, langsam in der Iteration und im Fehlerfall reparaturbedürftig.

Dieses Projekt wählt daher einen anderen Träger: das System *Carrera Hybrid*. Dabei handelt es sich um Miniaturfahrzeuge im Maßstab von etwa 1:43, die sich – anders als klassische Slotcars – *ohne* mechanische Führungsschiene frei auf einer Fahrbahn bewegen und über zwei kontinuierliche Stellgrößen, Lenkung und Antrieb, ferngesteuert werden. Damit liegt ein vollwertiges nichtlineares Regelungsproblem vor: Haftungsgrenzen, Untersteuern, Trägheit und ein begrenzter Bremsweg sind real vorhanden, während ein Fehlversuch lediglich bedeutet, dass ein Fahrzeug von der Bahn rutscht.

Die angestrebte Gesamtvision ersetzt den Roboter durch eine stationäre Deckenkamera: Eine einzelne Kamera über dem Aufbau erfasst die gesamte Strecke aus der Vogelperspektive, ein neuronaler Agent verarbeitet diesen Bildstrom und sendet Lenk-, Gas- und Bremsbefehle an das Fahrzeug. Der Agent erhält damit ungefähr dieselbe Informationslage wie ein menschlicher Fahrer, der neben der Bahn steht und sein Auto beobachtet – er sieht die ganze Strecke, aber keine Messwerte. Genau diese Gleichstellung ist gewollt, denn das Vergleichsmaß des Projekts ist die menschliche Rundenzeit.

Zwei weitere Beweggründe prägen den Zuschnitt der Arbeit. Erstens besteht ein ausdrückliches fachliches Interesse an Reinforcement Learning (RL) als Methode, also am Lernen durch Versuch und Rückmeldung anstelle eines analytisch entworfenen Reglers. Als Motivation im engeren Sinne dienten dabei die bildbasierten Trackmania-Agenten des YouTube-Kanals *yoshtm* [1], die

zeigen, dass ein rein lernend erzeugter Fahrer in einem Rennspiel bis an und über menschliche Bestzeiten heranreicht. Zweitens, und für den methodischen Aufbau entscheidend, standen für dieses Projekt nur begrenzte Rechenressourcen zur Verfügung. Ein vollständiger bildbasierter Trainingslauf in der 3D-Simulation bindet einen Arbeitsplatzrechner über Tage; die Frage, ob eine bestimmte Beobachtungsform überhaupt tragfähig ist, lässt sich unter dieser Bedingung nicht durch Ausprobieren beantworten. Daraus folgt der zentrale Entwurfsentscheid dieser Arbeit: Die konzeptionellen Fragen werden zuvor in einer eigens gebauten, sehr schnellen 2D-Umgebung geklärt.

Dass dieser Aufgabentyp tragfähig ist, zeigt bereits eine frühe Arbeit von Lange, Riedmiller und Voigtländer [2], in der ein realer Slotcar allein aus unverarbeiteten Kamerabildern gesteuert wird und die gelernte Politik von einem geübten menschlichen Spieler kaum geschlagen wird. Die dort gestellte Frage – wie ein System aus rohen Pixeln selbst herausfindet, welche Bildinformation regelungsrelevant ist – ist auch die Leitfrage dieses Projekts.

1.2 Projektstruktur

Das Gesamtvorhaben ist in drei aufeinander aufbauende Phasen gegliedert. Die vorliegende Arbeit deckt Phase 1 vollständig und Phase 2 in ihrer konzeptionellen Vorbereitung ab.

Phase 1 – 2D-Python-Prototyping und Methodenevaluation: Aufbau einer leichtgewichtigen, hochperformanten 2D-Simulationsumgebung (`Carrera2DEnv`), um fundamentale Fragen zu Beobachtungsräumen, Netzwerkarchitekturen, Lernalgorithmen (Proximal Policy Optimization (PPO) gegen Soft Actor-Critic (SAC)), Belohnungsfunktionen und Datenvorverarbeitungsmethoden isoliert und zeiteffizient zu untersuchen. Eine Trainingsepisode kostet hier Sekunden statt Minuten, ein vollständiger Lauf Stunden statt Tage.

Phase 2 – Transfer in die fotorealistische 3D-Simulation: Übertragung der im 2D-Prototyping validierten Vorgehensweisen in die fotorealistische 3D-Simulation (NVIDIA Isaac Sim beziehungsweise Isaac Lab). Hier ersetzt eine PhysX-basierte Starrkörpersimulation das vereinfachte Fahrzeugmodell, und ein gerendertes Kamerabild tritt an die Stelle der synthetischen Vogelperspektive.

Phase 3 – Sim-to-Real-Transfer (Ausblick): Erforschung des Sim-to-Real-Gaps auf der realen Carrera-Hybrid-Hardware, also die Überführung eines in Simulation trainierten Agenten auf die echte Bahn. Diese Phase ist als Anschlussprojekt vorgesehen und wird in Kapitel 7 nur als Ausblick behandelt.

Die Aufteilung folgt einem einfachen Grundsatz: Jede Frage wird auf der billigsten Stufe beantwortet, auf der sie sich beantworten lässt. Ob eine Belohnungsfunktion überhaupt ein sinnvolles Verhalten prämiert, ob ein Bildstapel Geschwindigkeitsinformation trägt oder ob ein Algorithmus mit dem verfügbaren Datenaufkommen konvergiert, sind Fragen, die keine Fotorealistik benötigen. Erst die Fragen nach Reibwerten, Beleuchtung und Kameraabbildung erfordern sie zwingend.

1.3 Ziele dieser Arbeit

Aus der Motivation und der Phasenstruktur leiten sich drei inhaltliche Ziele ab:

- **Schnelles 2D-Prototyping:** Konzepte und Probleme sollen bei geringer Rechenlast so früh wie möglich gegeneinander abgewogen werden. Gesucht ist nicht der beste Agent, sondern die belastbare Aussage, welche Beobachtungsform, welcher Algorithmus und welche Vorverarbeitung den Aufwand in Phase 2 rechtfertigen.
- **Reward Engineering für stabiles Fahren:** Die Belohnungsfunktion ist der einzige Kanal, über den das gewünschte Verhalten spezifiziert wird. Ziel ist eine Funktion, die zügiges, spurstabiles und – im Hinblick auf den späteren Realtransfer – aktorschonendes Fahren belohnt, ohne dabei unbeabsichtigte Nebenoptima zu erzeugen.
- **Physikalisch präzises Fundament für das 3D-Training:** Die 2D-Umgebung soll das reale Fahrzeug so genau abbilden, dass die gewonnenen Erkenntnisse übertragbar bleiben. Die entwickelte Pipeline dient als Grundlage für ein Sim-to-Real-Vorhaben in einem möglichen Folgeprojekt.

1.3.1 Einblick in die Projektstruktur

Da ein wesentlicher Teil des Arbeitsergebnisses im Repository selbst liegt, sei dessen Aufbau hier kompakt erläutert. Die vollständige Verzeichnisstruktur, ein Leitfaden durch das Notebook und die wiederkehrenden Aufrufe stehen in Anhang **??**; die `README.md` im Repository fasst denselben Einstieg für die tägliche Arbeit zusammen.

Warum Gymnasium? Die Simulationsumgebung implementiert die Schnittstelle von *Gymnasium*, dem Nachfolger von OpenAI Gym. Der Grund ist Austauschbarkeit: Gymnasium legt mit `reset()`, `step()`, `observation_space` und `action_space` genau die fünf Berührungspunkte

fest, die ein Lernalgorithmus zu einer Umgebung benötigt. Dadurch lassen sich fertige, geprüfte Implementierungen aus *Stable-Baselines3* [3] ohne Anpassung verwenden, und ein Wechsel von PPO zu SAC ist eine Zeile Code statt einer Neuimplementierung. Ebenso wichtig ist die Gegenrichtung: Weil die Umgebung selbst nichts über den Algorithmus weiß, lassen sich Beobachtungsräume austauschen, ohne den Trainingscode anzufassen – genau diese Eigenschaft trägt den in Abschnitt 3.2.7 beschriebenen Kontrollversuch.

Wo liegen Modelle und Agenten? Jeder Trainingslauf legt unter `models/` ein eigenes, mit Zeitstempel benanntes Verzeichnis an. Darin stehen neben den Modellarchiven (`best_model.zip`, `final_model.zip`) zwei Dateien, die für die Nachvollziehbarkeit entscheidend sind: `train_config.json` protokolliert den vollständigen Zustand von Umgebung, Belohnungsfunktion und Hyperparametern, und `evaluations.npz` enthält die Zwischenauswertungen über den Lauf. Die Diagnosegrößen des Trainings selbst – Critic-Verlust, Entropiekoeffizient, mittlere Episodenlänge und -belohnung – liegen getrennt davon unter `tensorboard_logs/`.

Wie wird trainiert? Der Einstiegspunkt ist in allen Fällen eine Notebook-Zelle, die eine Umgebung erzeugt, sie gegebenenfalls mit einem Framestapel-Wrapper umhüllt, ein Modell konfiguriert und `model.learn()` aufruft. Wiederkehrende Bausteine sind in Skripte ausgelagert: `src/envs/` enthält die Umgebung, `src/utils/` die Belohnungsfunktion, die virtuelle Kamera, den Replay-Buffer und die Streckenextraktion, `src/train/` die eigenständig lauffähigen Trainingsskripte und `src/eval/` die Auswertungswerkzeuge. Letztere sind bewusst als Module gebaut und von der Kommandozeile aufrufbar, etwa `python -m src.eval.vergleich`, damit eine Auswertung nicht davon abhängt, welche Notebook-Zellen zuvor ausgeführt wurden.

Das interaktive Notebook. Das `Notebook_prototyping.ipynb` ist der eigentliche Arbeits- und Denkverlauf des Projekts und zugleich die interaktive Dokumentation. Es führt in der Reihenfolge der tatsächlichen Entstehung durch die Umgebung: von der Streckenextraktion mit OpenCV über die manuelle Steuerung des Fahrzeugs per Tastatur, die Definition des Belohnungsfensters und die virtuelle Kamera bis zu den Trainings- und Beobachtungszellen. Wer die Entscheidungen dieser Arbeit nachvollziehen will, kann dort das Fahrzeug selbst fahren, die Belohnung dabei live ablesen, die Streckenbegrenzungen einblenden und einen beliebigen gespeicherten Agenten unter seiner Originalphysik beim Fahren beobachten. Der Bericht referenziert die zugehörigen Zellen an den entsprechenden Stellen; eine vollständige Zuordnung enthält Anhang **??**.

1.4 Aufbau der Arbeit

Kapitel 2 erklärt die in dieser Arbeit verwendeten Begriffe und ordnet das Vorhaben in den Stand der Technik ein. Kapitel **??** bildet den Hauptteil: Es beschreibt zunächst Idee, Umsetzung, Erkenntnisse und Fehlerquellen des 2D-Prototypings (Phase 1) und anschließend die daraus abgeleitete Übertragung in die 3D-Simulation (Phase 2). Kapitel 6 stellt die Ergebnisse in Form der entstandenen Trainingspipeline und der zugehörigen Messwerte dar. Kapitel 7 zieht das Fazit über beide bearbeiteten Phasen und gibt einen Ausblick auf den Sim-to-Real-Transfer in Phase 3.

2 Grundwissen und Begriffserklärungen

2.1 Theoretische Grundlagen

Ein einzelnes Grundlagenkapitel kann das Feld des Reinforcement Learning (RL) nicht lückenlos abbilden – es umfasst Jahrzehnte an Theorie zwischen dynamischer Programmierung, Stochastik und Optimierung. Für Leserinnen und Leser, die den Einstieg suchen, sei daher ausdrücklich auf das Video *The FASTEST introduction to Reinforcement Learning on the internet* [4] verwiesen, das die Begriffskette von Markov-Entscheidungsprozessen über Monte-Carlo- und Temporal-Difference-Verfahren bis zu Deep-Q-Netzen und Policy-Gradienten in kompakter, anschaulicher Form entwickelt. Es ist nach Einschätzung der Verfasser der schnellste verfügbare Weg, ein tragfähiges Grundverständnis aufzubauen.

Dieses Kapitel beschränkt sich entsprechend darauf, genau die Begriffe präzise zu definieren, die im weiteren Verlauf dieser Arbeit verwendet werden. Die Definitionen folgen, sofern nicht anders angegeben, dem Standardwerk *Reinforcement Learning: An Introduction* von Sutton und Barto [5].

2.1.1 Markov-Entscheidungsprozess (MDP)

Die formale Grundlage des RL ist der Markov-Entscheidungsprozess (MDP). Er beschreibt die Wechselwirkung zwischen einem *Agenten* und einer *Umgebung* als Fünftupel

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma), \tag{2.1}$$

mit dem Zustandsraum $\mathcal{S}$, dem Aktionsraum $\mathcal{A}$, der Übergangsverteilung $\mathcal{P}(s' \mid s, a)$, der Belohnungsfunktion $\mathcal{R}(s, a, s')$ und dem Diskontfaktor $\gamma \in [0, 1)$.

Der Ablauf ist zyklisch: Der Agent beobachtet den Zustand s_t, wählt eine Aktion a_t, die Umgebung

geht in den Zustand s_{t+1} über und zahlt die Belohnung r_{t+1} aus. Ziel des Agenten ist es, den *diskontierten Ertrag*

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1} \tag{2.2}$$

in Erwartung zu maximieren.

Der namensgebende Kern ist die *Markov-Eigenschaft*: Der Zustand s_t muss alles enthalten, was für die Zukunft relevant ist. Formal gilt

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t). \tag{2.3}$$

Die Vergangenheit darf also keine zusätzliche Information über den Folgezustand tragen. Für das vorliegende Projekt ist das eine scharfe Anforderung: Ein Zustand, der die Fahrzeugposition enthält, aber nicht die Geschwindigkeit, verletzt sie, denn aus derselben Position folgen je nach Tempo völlig verschiedene Folgepositionen.

Der Diskontfaktor γ steuert, wie weit der Agent in die Zukunft plant. Der wirksame Planungshorizont beträgt näherungsweise

$$H \approx \frac{1}{1-\gamma} \text{ Zeitschritte}, \tag{2.4}$$

also bei $\gamma = 0{,}99$ und 30 Hz rund 100 Einzelbilder oder 3,3 s. Diese scheinbar harmlose Größe erwies sich in diesem Projekt als eine der folgenreichsten Einstellungen überhaupt (Abschnitt 3.2.10).

2.1.2 Partiell beobachtbarer MDP (POMDP)

In realen Systemen ist der wahre Zustand s_t selten messbar. Reibwerte, Radschlupf, exakte Gierraten und die Fahrzeuggeschwindigkeit liegen nicht als Zahl vor, sondern müssen aus dem geschätzt werden, was ein Sensor liefert. Dieser Fall wird als partiell beobachtbarer Markov-Entscheidungsprozess (POMDP) bezeichnet und um zwei Komponenten erweitert:

$$\mathcal{M}_{\mathrm{PO}} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \Omega, \mathcal{O}), \tag{2.5}$$

mit dem Beobachtungsraum Ω und der Beobachtungsfunktion $\mathcal{O}(o \mid s)$. Der Agent sieht nur $o_t \in \Omega$, nicht s_t.

Ein einzelnes Kamerabild ist der Musterfall einer solchen unvollständigen Beobachtung. Es zeigt, *wo* das Fahrzeug ist, nicht aber, *wie schnell* und in welche Richtung es sich bewegt. Ein optimaler

Regler auf dieser Beobachtung existiert im Allgemeinen nicht als einfache Abbildung $o_t \mapsto a_t$, weil derselben Beobachtung verschiedene wahre Zustände mit verschiedenen richtigen Aktionen zugrunde liegen können. Diese Mehrdeutigkeit ist kein technisches Detail, sondern die theoretische Wurzel des zentralen Befunds dieser Arbeit (Abschnitt 3.2.7).

Die gängigen Gegenmaßnahmen sind:

1. die Beobachtung um zeitliche Information zu erweitern (*Frame-Stacking*, Abschnitt 2.1.7),
2. die fehlenden Größen direkt als Zahlenwerte mitzugeben (*hybrider Input*, ebenfalls Abschnitt 2.1.7),
3. ein rekurrentes Netz einzusetzen, das einen internen Zustand mitführt.

In diesem Projekt wurden die ersten beiden Wege verfolgt.

2.1.3 Policy, On-Policy und Off-Policy, modellfrei und modellbasiert

Die *Policy* π ist die Entscheidungsregel des Agenten, also die Abbildung von Beobachtungen auf Aktionen. Sie kann deterministisch sein, $a = \pi(s)$, oder stochastisch, $a \sim \pi(a \mid s)$. Im tiefen RL ist π_θ ein neuronales Netz mit den Parametern θ; „Lernen“ bedeutet, diese Parameter per Gradientenabstieg zu verschieben.

Ein zweiter Baustein ist die *Wertfunktion*. Die Zustandswertfunktion $V^\pi(s)$ gibt den erwarteten Ertrag an, wenn ab s nach π gehandelt wird; die Aktionswertfunktion $Q^\pi(s, a)$ tut dasselbe für ein festes erstes Aktionspaar. Verfahren, die beides kombinieren – ein Netz, das handelt (*Actor*), und eines, das bewertet (*Critic*) –, heißen *Actor-Critic*-Verfahren. Beide in dieser Arbeit verwendeten Algorithmen gehören dazu.

Zwei Unterscheidungen sind für die Algorithmenwahl entscheidend:

Modellfrei gegen modellbasiert. *Modellbasierte* Verfahren lernen ein explizites Modell der Umgebungsdynamik $\hat{\mathcal{P}}$ und planen darin, etwa indem sie Trajektorien vorwärts simulieren. Sie sind dadurch oft sample-effizient, hängen aber von der Güte des gelernten Modells ab. *Modellfreie* Verfahren verzichten darauf und optimieren die Policy ausschließlich aus tatsächlich erlebten Übergängen. In diesem Projekt wurden ausschließlich modellfreie Verfahren eingesetzt, weil die Umgebung selbst bereits sehr schnell ist – ein gelerntes Weltmodell würde hier die teure Komponente durch eine ungenauere ersetzen, ohne Zeit zu sparen.

On-Policy gegen Off-Policy. Der Unterschied betrifft die Frage, *welche* Erfahrung zum Lernen verwendet werden darf.

On-Policy Es darf nur mit Erfahrung gelernt werden, die die *aktuelle* Policy erzeugt hat. Sobald die Gewichte verändert wurden, sind die alten Daten ungültig und werden verworfen.

Off-Policy Es darf auch mit Erfahrung gelernt werden, die eine andere – etwa eine ältere oder eine fremde – Policy erzeugt hat. Damit wird ein Gedächtnis in Form eines Replay-Buffers möglich (Abschnitt 2.1.8).

Diese Unterscheidung schlägt unmittelbar auf die Sample-Effizienz durch und ist der Grund für den in Abschnitt 3.2.11 beschriebenen Leistungsunterschied.

2.1.4 PPO und SAC

Proximal Policy Optimization (PPO). PPO [6] ist ein modellfreies On-Policy-Verfahren. Es sammelt einen Block an Erfahrung, schätzt daraus Vorteilswerte und führt mehrere Gradientenschritte auf diesem Block aus. Die namensgebende Idee ist eine Begrenzung der Schrittweite im Verteilungsraum: Über das Verhältnis $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{alt}}}(a_t \mid s_t)$ wird die Zielfunktion

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\,\hat{A}_t\Big)\Big] \tag{2.6}$$

maximiert. Der `clip`-Term verhindert, dass ein einzelnes Update die Policy zu weit von ihrem Ausgangspunkt entfernt – die Grundlage für PPOs Ruf als robuster, wenig empfindlicher Algorithmus. Der Preis ist, dass nach dem letzten Gradientenschritt der gesamte gesammelte Datenblock verworfen wird.

Soft Actor-Critic (SAC). SAC [7] ist ein modellfreies Off-Policy-Actor-Critic-Verfahren für kontinuierliche Aktionsräume. Es verfolgt ein erweitertes Ziel, das neben dem Ertrag auch die Entropie der Policy maximiert:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha\,\mathcal{H}(\pi(\cdot \mid s_t))\right]. \tag{2.7}$$

Der Entropieterm $\mathcal{H}$ hält die Policy so zufällig wie mit gutem Ertrag vereinbar und wirkt damit einem vorzeitigen Festfahren in lokalen Optima entgegen; der Faktor α wird automatisch nachgeregelt. SAC nutzt zwei unabhängige Critic-Netze und bildet beim Bootstrapping das Minimum beider Schätzungen – ein Mittel gegen die systematische Überschätzung von Q-Werten,

wie sie aus dem Umfeld von TD3 [8] bekannt ist. Da SAC off-policy arbeitet, kann jede gespeicherte Erfahrung vielfach zum Lernen verwendet werden.

2.1.5 Imitation Learning, Behavioral Cloning und Covariate Shift

Imitation Learning und Learning from Demonstration. Beim Imitation Learning (IL) lernt ein Agent nicht aus Belohnung, sondern aus dem Vorbild eines Experten. *Learning from Demonstration* bezeichnet dieselbe Idee mit Betonung auf der Datenquelle: Ein Experte – ein Mensch, ein klassischer Regler oder, wie hier, ein bereits trainierter Agent – führt die Aufgabe vor, und die dabei entstehenden Zustands-Aktions-Paare bilden den Trainingsdatensatz.

Behavioral Cloning. Behavioral Cloning (BC) ist die einfachste Umsetzung dieser Idee: Das Lernproblem wird zu überwachtem Lernen zurückgeführt. Auf einem Datensatz $\mathcal{D} = \{(s_i, a_i^*)\}$ von Expertenpaaren wird

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(s,a^*)\in\mathcal{D}} \left\| \pi_\theta(s) - a^* \right\|^2 \tag{2.8}$$

minimiert. Kein Reward, keine Exploration, keine Wertfunktion – eine gewöhnliche Regression.

Covariate Shift. Genau darin liegt die grundlegende Schwäche des Verfahrens. Der Datensatz enthält ausschließlich Zustände, in die der *Experte* gerät, also im Wesentlichen die Ideallinie. Weicht der Schüler durch einen kleinen Approximationsfehler davon ab, gelangt er in Zustände, die im Datensatz nicht vorkommen (*Out-of-Distribution*). Dort hat er keine Antwort gelernt, weicht weiter ab, und der Fehler schaukelt sich auf. Ross und Bagnell zeigen, dass der erwartete Verlust dabei nicht linear, sondern quadratisch mit dem Zeithorizont T wächst [9]. Die Verteilung der Zustände unter der Schülerpolicy unterscheidet sich also systematisch von der Trainingsverteilung – daher der Name *Covariate Shift* oder *Distribution Shift*.

Zwei Gegenmittel sind gebräuchlich. *DAgger* [9] lässt den Schüler fahren, fragt in *seinen* Zuständen den Experten nach der richtigen Aktion und erweitert den Datensatz iterativ. Die in diesem Projekt gewählte, billigere Variante ist die *Rausch-Injektion*: Auf die Expertenaktion wird während der Aufzeichnung ein Störterm addiert,

$$\tilde{a}_t = \mathrm{clip}\left(a_t^* + \xi_t,\ -1,\ 1\right), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2), \tag{2.9}$$

sodass das Fahrzeug von der Ideallinie abdriftet und der Experte im Folgeschritt aktiv gegen-

steuern muss. Gespeichert wird dabei die *unverrauschte* Korrekturaktion a_t^*. Der Datensatz enthält dadurch Rettungsmanöver, die eine perfekte Vorführung nie erzeugen würde. Amini et al. begründen mit dem Framework VISTA, warum das gezielte Erleben solcher Zustände für einen robusten Realtransfer unverzichtbar ist [10].

2.1.6 Privilegiertes Lernen (Learning by Cheating)

Beim *privilegierten Lernen* wird das Gesamtproblem in zwei orthogonale Teilprobleme zerlegt, die getrennt und dadurch jeweils erheblich leichter lösbar sind. Chen et al. haben dieses Vorgehen unter dem Namen *Learning by Cheating* [11] systematisch untersucht:

1. **Das Regelungsproblem.** Ein *privilegierter* Lehrer-Agent erhält Zugriff auf Informationen, die zur Laufzeit nicht verfügbar wären – exakte Abstände, die eigene Geschwindigkeit, die Streckengeometrie. Er muss nur noch lernen, *wie man fährt*, und konvergiert in einem niedrigdimensionalen Zustandsraum sehr schnell.
2. **Das Wahrnehmungsproblem.** Ein Schüler-Agent erhält ausschließlich die realistisch verfügbare Beobachtung – hier Kamerabilder – und lernt überwacht, die Ausgaben des Lehrers zu reproduzieren. Er muss nur noch lernen, *was er sieht*.

Der Vorteil gegenüber direktem visuellem RL liegt im Fehlersignal. Ein bildbasierter RL-Agent erhält als einzige Rückmeldung eine verrauschte, verzögerte, skalare Belohnung und muss daraus gleichzeitig ableiten, welche Pixel überhaupt bedeutsam sind und welche Aktion richtig war. Der Schüler im privilegierten Aufbau erhält dagegen für jedes Bild einen vollständigen, dichten Zielvektor.

Wichtig für die Einordnung der Ergebnisse in Kapitel **??** ist, was dabei tatsächlich übertragen wird. Wird – wie in diesem Projekt – nur der Faltungsteil des vortrainierten Netzes in den RL-Agenten kopiert, so überträgt sich *Sehfähigkeit*, nicht *Fahrwissen*: Der Agent erbt die Merkmalsextraktion, nicht die Entscheidungsregel. Diese Unterscheidung wird in Abschnitt 3.3 quantitativ belegt.

2.1.7 Frame-Stacking und hybrider Input

Frame-Stacking. Die verbreitetste Antwort auf die fehlende Zeitinformation eines Einzelbildes ist das Stapeln der letzten k Aufnahmen entlang der Kanaldimension:

$$s_t = [o_t,\ o_{t-1},\ \ldots,\ o_{t-k+1}] \in \mathbb{R}^{k\times H\times W}. \tag{2.10}$$

Das Verfahren geht auf die Atari-Arbeiten von Mnih et al. [12] zurück und ist heute Standard. Die ersten Faltungsschichten können aus der Pixelverschiebung zwischen den Kanälen zeitliche Ableitungen bilden und damit Geschwindigkeit und Drehrate implizit rekonstruieren. Der POMDP wird so näherungsweise wieder zu einem MDP.

Die stillschweigende Voraussetzung lautet, dass sich zwischen zwei gestapelten Bildern überhaupt etwas *messbar* verschiebt. Genau diese Voraussetzung ist in der vorliegenden Anwendung verletzt; die Herleitung folgt in Abschnitt 3.2.7.

Eine Variante ist das *zeitliche Spreizen* (*temporal striding* oder *dilated frame stacking*): Statt t, $t-1$, $t-2$ werden Bilder mit größerem Abstand gestapelt, etwa t, $t-4$, $t-8$. Der zeitliche Hebel vervierfacht sich damit, ohne dass Rechenzeit oder Speicher zunehmen.

Hybrider Input. Als *hybrider Input* wird die Fusion verschiedener Datenformate in einer Beobachtung bezeichnet – hier ein Bild zusammen mit einem Vektor von Skalarparametern wie der aktuellen Geschwindigkeit und dem aktuellen Lenkwinkel. Technisch wird das über einen zusammengesetzten Beobachtungsraum umgesetzt: Ein Convolutional Neural Network (CNN) verarbeitet den Bildanteil, ein kleines Multi-Layer Perceptron (MLP) den Vektoranteil, und beide Merkmalsvektoren werden vor dem Entscheidungskopf verkettet.

Der Nutzen ist bei Einzelbildeingaben unmittelbar einsichtig, weil dort Geschwindigkeit und Lenkwinkel prinzipiell nicht aus der Beobachtung hervorgehen. Bei gestapelten Bildern ist er weniger offensichtlich, in dieser Anwendung aber nicht geringer: Im Einspurmodell gilt für die Gierrate

$$\omega = \frac{v \cdot \tan(\delta)}{L}, \tag{2.11}$$

mit dem Lenkwinkel δ und dem Radstand L. Wer v und δ exakt kennt, besitzt bereits die vollständige Bewegungsinformation, die ein Bildstapel bestenfalls näherungsweise liefert – für acht Byte statt für mehrere zehn Kilobyte je Übergang.

2.1.8 Replay-Buffer, Pretraining und Online- gegen Offline-Training

Replay-Buffer. Ein *Replay-Buffer* ist ein Ringspeicher vergangener Übergänge der Form (s, a, r, s', fertig). Er ist die technische Voraussetzung dafür, dass Off-Policy-Verfahren ihren Vorteil ausspielen, und erfüllt drei Aufgaben:

1. **Wiederverwendung.** Jeder Übergang kann hunderte Male in ein Gradientenupdate eingehen, statt nach einem Update verworfen zu werden.
2. **Entkopplung.** Aufeinanderfolgende Einzelbilder sind nahezu identisch. Zufälliges Ziehen aus dem Puffer bricht diese zeitliche Korrelation auf; andernfalls bestünde ein Minibatch aus nahezu derselben Situation.
3. **Schutz vor Vergessen.** Der Critic soll auch Zustände bewerten können, die die aktuelle Policy nicht mehr ansteuert.

Der Preis ist Arbeitsspeicher. Bei Bildbeobachtungen wird der Puffer zur bindenden Ressource – eine Rechnung dazu findet sich in Abschnitt 3.2.12.

Pretraining. Als *Pretraining* wird jede Initialisierung von Netzgewichten bezeichnet, die nicht zufällig erfolgt, sondern aus einer vorgelagerten Lernaufgabe stammt. In dieser Arbeit ist das ausschließlich das überwachte Training eines CNN auf dem Demonstrationsdatensatz des Lehrer-Agenten; anschließend werden dessen Faltungsschichten in den RL-Agenten übertragen (*Backbone-Transplantation*).

Online gegen Offline. *Online-Training* bedeutet, dass der Agent während des Lernens selbst mit der Umgebung interagiert und die dabei entstehenden Daten unmittelbar verwendet – der Normalfall im RL. *Offline-Training* (auch *Batch-RL*) lernt ausschließlich aus einem zuvor aufgezeichneten, festen Datensatz ohne jede weitere Interaktion; der Reiz liegt darin, dass keine riskanten oder teuren Versuchsfahrten nötig sind, die Schwierigkeit in der fehlenden Möglichkeit, eine eigene Hypothese zu überprüfen [13]. Die in dieser Arbeit verwendete Pipeline ist in diesem Sinne gemischt: Das Vortraining ist offline und überwacht, die anschließende Optimierung online und bestärkend.

2.2 State of the Art

Der für dieses Projekt relevante Ausschnitt des Forschungsstands betrifft RL für RC-Fahrzeuge, Slotcars und Rennagenten. Vier Arbeiten prägen die Konzeption dieser Arbeit unmittelbar.

DIRL – Vision-Based Autonomous Car Racing (Cai et al.). Cai et al. [14] stellen mit *Deep Imitative Reinforcement Learning* einen hybriden Ansatz vor, der Imitationslernen zur Initialisierung mit modellbasiertem RL zur Selbstverbesserung verbindet, und validieren ihn neben der Simulation auf einem realen RC-Fahrzeug im Maßstab 1:20 mit begrenzter Rechenleistung an Bord. Die Arbeit belegt die Überlegenheit des hybriden Vorgehens gegenüber reinem IL wie reinem RL in Sample-Effizienz und Fahrleistung und ist damit der unmittelbare Bauplan für die hier verfolgte Teacher-Student-Pipeline.

Gran Turismo Sophy (Wurman et al.). Wurman et al. [15] erreichen mit dem Agenten *GT Sophy* übermenschliche Rundenzeiten und gewinnen ein Kopf-an-Kopf-Rennen gegen vier der weltbesten Gran-Turismo-Fahrer. Methodisch relevant sind drei Bausteine: der Einsatz modellfreier SAC-Varianten für die Fahrzeugregelung am Haftungslimit, ein sorgfältig entworfener Streckenfortschritts-Reward entlang der Mittellinie und eine asymmetrische Actor-Critic-Architektur, bei der der Critic während des Trainings über privilegierte Zustandsinformationen verfügt, die dem Actor nicht zur Verfügung stehen. Der letzte Punkt ist für Phase 2 dieses Projekts von besonderem Interesse (Abschnitt **??**).

VISTA (Amini et al.). Amini et al. [10] zeigen mit dem datengetriebenen Simulator *VISTA*, dass sich robuste End-to-End-Regelungspolitiken aus wenigen realen Fahrten lernen lassen, wenn die Simulation gezielt Trajektorien synthetisiert, die von der aufgezeichneten Ideallinie abweichen. Der Kern der Aussage lautet: Das gezielte Erleben und Trainieren von Out-of-Distribution-Rettungsmanövern ist für einen robusten Sim-to-Real-Transfer nicht optional, sondern essenziell. Dieses Ergebnis begründet die in Abschnitt 2.1.5 beschriebene Rausch-Injektion.

Trackmania-Agenten. Außerhalb des begutachteten Schrifttums, aber als Machbarkeitsnachweis und Motivationsquelle einschlägig, sind die bildbasierten Trackmania-Agenten des Kanals *yoshtm* [1]. Sie belegen, dass ein rein lernend erzeugter Fahrer in einem vollwertigen Rennspiel an menschliche Rekordzeiten heranreicht, und liefern zugleich eine anschauliche Vorstellung davon, welchen Trainingsaufwand das erfordert.

Weitere Einordnung. Ergänzend ist der Stand der Technik durch mehrere Übersichtsarbeiten gut erschlossen. Chib und Singh [16] systematisieren End-to-End-Fahrverfahren nach Eingabemodalität, Lernparadigma und Bewertungsmethodik; Delavari et al. [17] werten rund einhundert RL-Arbeiten im CARLA-Simulator aus und stellen fest, dass über 80 % weiterhin auf modellfreie Verfahren wie DQN, PPO und SAC setzen. Ly und Akhloufi [18] geben einen Überblick über tiefe BC-Verfahren einschließlich ihrer Verteilungsprobleme. Wertbasierte Verfahren sind im Rennkontext ebenfalls erprobt – Peng et al. [19] übertreffen mit einem Dueling-Double-DQN im Simulator TORCS menschliche Fahrer bei der Spurhaltung, und Khlifi et al. [20] vergleichen DQN und DDQN in CARLA –, sie sind für die hier benötigte feinstufige, kontinuierliche Stellgrößenvorgabe jedoch weniger geeignet als Actor-Critic-Verfahren.

Methodisch besonders nahe liegt *SLAV-Sim* von Crewe et al. [21]. Die Autoren begründen dort denselben Ansatz, der auch dieser Arbeit zugrunde liegt: Anstelle eines schwergewichtigen Simulators wird eine leichtgewichtige Umgebung gebaut, die genau die benötigten Sensormodalitäten – Kamera und Raycasts – bereitstellt und dadurch schnelle Iteration über RL-Algorithmen erlaubt. Für die konkrete Kombination aus Demonstration und Bestärkung sind ferner Huang et al. [22], die Expertenwissen als Regularisierung über eine KL-Divergenz einbringen, und Chekroun et al. [23] relevant, die Demonstrationsdaten mit konstant hoher Belohnung direkt in den Replay-Buffer eines Off-Policy-Verfahrens einspeisen. Mohammed et al. [24] untersuchen speziell die Frage, welchen Beitrag offline vortrainierte Bildencoder für nachgelagertes RL leisten – eine Frage, die sich in diesem Projekt für den transplantierten Backbone in gleicher Form stellt (Abschnitt 3.3).

Historisch bemerkenswert bleibt schließlich die bereits erwähnte Arbeit von Lange et al. [2]: Sie löst dieselbe Aufgabenklasse – kamerabasierte Regelung eines realen Slotcars – bereits vor über einem Jahrzehnt, allerdings mit einem Autoencoder zur Zustandsreduktion und batchbasiertem Q-Learning statt mit heutigen Actor-Critic-Verfahren.

3 Phase 1: 2D-Prototyping

3.1 Idee

Dem Aufbau der Simulationsumgebung lag ein pragmatischer, industriell geprägter Entwicklungsansatz zugrunde. Die Leitfrage war nicht „Was wäre die genaueste Modellierung?“, sondern „Was ist bereits vorhanden, und was lässt sich sofort beschaffen oder erstellen?“. Ziel war ein lauffähiger Prototyp ohne Vorlaufzeit, an dem sich die eigentlichen Forschungsfragen anschließend beantworten lassen.

Das Zielbild dieser ersten Stufe war bewusst einfach: Ein Fahrzeug – als Grafik – soll sich auf einer Strecke – ebenfalls als Grafik – per WASD-Tastatur steuern lassen. Die Schnittstelle, über die später der Agent fährt, ist dieselbe, über die zunächst ein Mensch fährt. Damit ist von Anfang an sichergestellt, dass sich jede Aussage über die Umgebung von Hand nachprüfen lässt.

Die benötigten Größen wurden aus vier unterschiedlichen Quellen zusammengetragen:

Strecke. Die offizielle Carrera-Hybrid-App stellt das Streckenlayout als Grafik dar. Ein Bildschirmfoto dieses Layouts dient als Hintergrundbild der Simulation. Die Maßstabsübertragung erfolgt über die an der realen Bahn abgenommenen Abmessungen – insbesondere die Fahrbahnbreite von 250 mm und die Randstreifen von je 21 mm, die in der Grafik rot (innen) und blau (außen) markiert sind.

Fahrzeug und Maße. Aus Produktbildern eines Onlineshops wurde eine freigestellte Vogelperspektive des Fahrzeugs gewonnen. Die geometrischen Maße und die Masse wurden direkt am realen Fahrzeug gemessen.

Physikalische Randbedingungen. Die Höchstgeschwindigkeit wurde aus Messungen an YouTube-Videos entnommen und über den Maßstab auf die Simulation übertragen.

Beschleunigungsvermögen. Für den ersten Wurf genügte eine Schätzung nach Augenmaß, eingestellt im interaktiven WASD-Demonstrationsmodell des Notebooks. Diese Schätzung wurde später durch eine systematische Videomessung ersetzt – mit weitreichenden Folgen,

die Abschnitt 3.2.6 beschreibt.

Als Fahrphysik dient ein *Einspurmodell* (Bicycle Model). Es nimmt Befehle für Gas, Bremse und Lenkung entgegen und berechnet daraus Längsbeschleunigung, Geschwindigkeit und Gierrate. Die Gierrate folgt Gleichung (2.11); sie wird zusätzlich durch ein Haftungslimit $\omega_{\max} = \mu g/|v|$ begrenzt, sodass das Fahrzeug bei zu hoher Kurvengeschwindigkeit untersteuert, statt der Lenkvorgabe exakt zu folgen. Die Lenkvorgabe selbst ist tiefpassgefiltert: Der Lenkwert darf sich je Zeitschritt um höchstens einen festen Betrag ändern, was die Stellzeit eines realen Servos nachbildet.

Mit diesen Bausteinen konnte das Prototyping ohne lange Vorlaufzeit starten. Tabelle 3.1 fasst die resultierenden Spezifikationen zusammen.

Tabelle 3.1: Spezifikationen der 2D-Simulationsumgebung und ihre Herkunft. Werte ohne Quellenangabe stammen aus der Implementierung in `src/envs/carrera_2d_env.py`. *Anmerkung zum Radstand:* Aus der Differenz der gemessenen Achsabstände ergäben sich 48 mm; die Abweichung zum implementierten Wert von 58 mm ist nicht dokumentiert und in Abschnitt **??** als offener Punkt vermerkt.

Größe	**Wert**	**Herkunft**
Strecke		
Fahrbahnbreite	250 mm	an der realen Bahn gemessen
Randstreifen je Seite	21 mm	an der realen Bahn gemessen
Streckenbild	830 × 500 px	Bildschirmfoto der Carrera-Hybrid-App
Maßstab	236 px/m	aus Bildbreite und Realmaß
Rundenlänge (Mittellinie)	7,66 m	1807 px der extrahierten Mittellinie
Außen- / Innenrand	8,55 m / 6,76 m	`outer_raw_spline.npy`, `inner_raw_spline.npy`
Start-/Ziellinie	$x = 462 \ldots 468$ px	manuell am Cursor abgelesen, fest hinterlegt
Fahrzeug		
Länge × Breite	98 mm × 39 mm	am realen Fahrzeug gemessen
Heck bis Hinterachse	23 mm	am realen Fahrzeug gemessen
Heck bis Vorderachse	71 mm	am realen Fahrzeug gemessen
Radstand L im Modell	58 mm	Modellparameter, siehe Anmerkung unter der Tabelle
Masse	80 g	am realen Fahrzeug gewogen
Max. Lenkwinkel	30°	Modellannahme
Reibwert μ	0,3	Modellannahme (Haftungslimit)
Kollisionskörper	20 × 7 px	Modellannahme
Simulation		
Zeitschritt Δt	1/30 s	entspricht 30 Hz Bildrate
Episodenlimit	2000 Frames $\hat{=}$ 66,7 s	Modellannahme
Aktionsraum	[Gas, Bremse, Lenken]	$[0,1] \times [0,1] \times [-1,1]$
Max. Lenkänderung / Frame	0,5	entspricht 133 ms von Anschlag zu Anschlag

3.2 Umsetzung

3.2.1 Arbeitsweise: Notebook mit ausgelagerten Modulen

Gearbeitet wurde in einem Jupyter-Notebook, dessen wiederverwendbare Bestandteile konsequent in Skripte ausgelagert sind: die Umgebung nach `src/envs/`, die Belohnungsfunktion, die virtuelle Kamera, der Replay-Buffer und die Streckenextraktion nach `src/utils/`, die Trainingsläufe nach `src/train/` und die Auswertungen nach `src/eval/`. Das Notebook behält dadurch seine Rolle als Arbeits- und Denkverlauf, ohne zum eigentlichen Programm zu werden.

Da die Streckenabmessungen sowohl in Pixeln als auch in Metern bekannt sind, lässt sich das Fahrzeugbild über den Maßstab exakt skalieren: Aus 98 mm Fahrzeuglänge bei 236 px/m folgt ein Sprite von 23×9 Pixeln. Mit Unterstützung generativer Coding-Werkzeuge entstand daraus eine Notebook-Zelle, die ein Fenster öffnet, das Streckenbild als Hintergrund zeichnet und das Fahrzeug per WASD steuerbar macht.

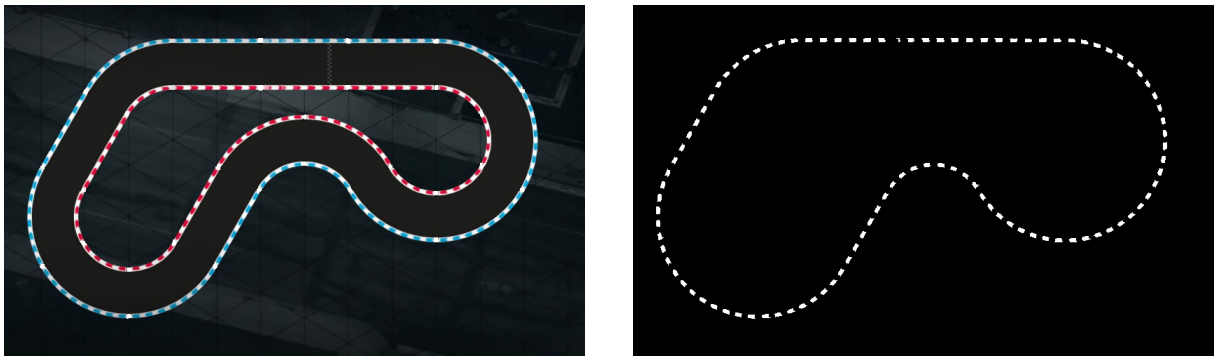
3.2.2 Vorbereitung für autonome Agenten

Aus der Zielvorstellung eines autonomen Agenten ergeben sich drei Anforderungen an die Umgebung, die die manuelle Steuerung noch nicht benötigt:

- Der Agent benötigt Informationen über Fahrtrichtung und Streckenbegrenzungen.
- Der Agent benötigt eine virtuelle Kamera – sowohl als globale Deckenansicht als auch als fahrzeugfester Ego-Ausschnitt.
- Der Agent benötigt Informationen über Streckenfortschritt und Ziellinie.

Die ersten beiden Punkte löst eine Computer-Vision-Vorverarbeitung des Streckenbildes. In `src/utils/tracklimits_cv.py` isolieren Farbmasken im HSV-Raum die rote Innen- und die blaue Außenbegrenzung (Abbildung 3.1). Die resultierenden Konturen werden als geglättete Splines abgelegt und sind im weiteren Verlauf die maßgebliche Streckengeometrie – als Kollisionskörper, als Schnittobjekt für die Raycasts und als Bezug für die Fahrtrichtungsprüfung. Die Mittellinie entsteht im Notebook: Zu jedem Punkt der einen Begrenzung wird der nächstgelegene Punkt der anderen gesucht, auf halber Strecke ein Punkt gesetzt, und alle so entstandenen Punkte werden verbunden und geglättet. Alle Ergebnisse werden als `.npy`-Arrays gespeichert, damit sie nicht bei jedem Programmstart neu berechnet werden müssen.

Die Start-/Ziellinie wurde manuell per Cursor über ihre Start- und Endpixel abgelesen und als feste Strecke von $(462, 55)$ bis $(468, 114)$ im Code hinterlegt. Eine Überquerung wird als geometrischer Schnitt zwischen dem Kollisionskörper des Fahrzeugs und dieser Linie erkannt.



(a) Streckenbild aus der Carrera-Hybrid-App **(b)** Extrahierte Außenbegrenzung (HSV-Maske)

Abbildung 3.1: Grundlage der Streckengeometrie. Aus dem Bildschirmfoto (a) isolieren Farbmasken im HSV-Raum die farbig markierten Randstreifen; aus der Maske (b) entstehen die Splines der Streckenbegrenzung und daraus die Mittellinie.

Die virtuelle Kamera (`src/utils/virtual_camera.py`) stellt zwei Ansichten bereit. Die *globale* Ansicht skaliert das gesamte Streckenbild auf die Beobachtungsgröße herunter und wandelt es in Graustufen. Die *fahrzeugfeste* Ansicht rotiert und schneidet einen Bereich von 150×150 Pixeln um das Fahrzeug so zu, dass dieses stets zentriert und nach oben ausgerichtet erscheint, und skaliert das Ergebnis auf 84×84 Pixel. In beiden Fällen wird die Helligkeit abschließend auf vier Stufen quantisiert.

3.2.3 Entwurf der Belohnungsfunktion

Die erste Fassung der Belohnungsfunktion folgte einer einfachen Intuition: Wandkollisionen und Rückwärtsfahren werden hart bestraft, schnelleres Fahren steigert die Belohnung quadratisch, das korrekte Überfahren der Ziellinie bringt einen deutlichen Bonus, und Stillstand kostet in jedem Zeitschritt Punkte.

Entscheidend für die Qualität dieser Funktion war die Möglichkeit, sie *von Hand* zu überprüfen. Im WASD-Fenster wird die laufende Belohnung eingeblendet, sodass unmittelbar sichtbar ist, welches Verhalten sie tatsächlich prämiert. Mehrere der in Abschnitt 3.2.9 beschriebenen Fehler wurden auf diesem Weg gefunden.

Die im weiteren Verlauf der Arbeit maßgebliche Fassung trägt die Versionskennung

`v3_rundenzeit` und ist in `src/utils/reward_func.py` implementiert. Sie setzt sich je Zeitschritt aus folgenden Posten zusammen:

$$r_t = \underbrace{2\,w_v\,v^2}_{\text{Tempo}} \; \underbrace{- \; w_{\text{slow}} \max\!\left(0,\; \frac{v_{\text{slow}} - v}{v_{\text{slow}}}\right)}_{\text{Langsam-Strafe}} \; \underbrace{- \; w_{\text{smooth}}\,|\Delta\delta_t|}_{\text{Lenk-Glättung}} \; + \; \underbrace{r_t^{\text{Runde}}}_{\text{Ereignisse}} \tag{3.1}$$

mit den Ereignisanteilen

$$r_t^{\text{Runde}} = \begin{cases} C_{\text{crash}} & \text{bei Kollision (Episode endet)} \\ B_{\text{lap}} & \text{bei korrekter Zielüberquerung} \\ w_{\text{imp}}\,(T_{\text{best}} - T_{\text{lap}}) & \text{bei Unterbieten der eigenen Bestzeit} \\ C_{\text{wrong}} - 5\,v & \text{bei Fahrt entgegen der Streckenrichtung.} \end{cases} \tag{3.2}$$

Tabelle 3.2 listet die zugehörigen Gewichte. Ergänzend endet eine Episode, sobald die kumulierte Belohnung -3000 unterschreitet – ein Auffangmechanismus gegen Agenten, die dauerhaft stehen bleiben.

Tabelle 3.2: Gewichte der Belohnungsfunktion `v3_rundenzeit`.

Parameter	Bedeutung	Wert
w_v	Tempo-Faktor (wirkt als $2w_v v^2$)	1,0
v_{slow}	Schwelle des Langsam-Gefälles	0,6 m/s
w_{slow}	Strafe je Frame bei $v = 0$	4,0
w_{smooth}	Strafe je Einheit Lenkänderung	0,5
w_{integral}	Oszillationsschutz (abgeschaltet)	0,0
B_{lap}	Bonus je Zielüberquerung	50
w_{imp}	Bonus je Frame Rundenzeitverbesserung	5,0
C_{crash}	Strafe bei Kollision	-300
C_{wrong}	Strafe für falsche Fahrtrichtung	-30

3.2.4 Erstes einfaches Training als Machbarkeitsnachweis

Für den ersten Trainingslauf wurde bewusst die am schnellsten umsetzbare Beobachtungsform gewählt: *Raycasts*. Virtuelle Laserstrahlen gehen vom Fahrzeug aus, werden mit den Streckenbegrenzungen geschnitten und liefern normierte Distanzwerte. Zu Beginn kamen zwei Strahlen im Winkel von $\pm 30°$ zum Einsatz; die spätere Fassung verwendet drei Strahlen bei $-25°$, $0°$

und +25°. Zusammen mit der normierten Eigengeschwindigkeit ergibt das eine vierdimensionale Beobachtung

$$o_t = \left[\frac{v}{v_{\max}},\ d_{\text{links}},\ d_{\text{mitte}},\ d_{\text{rechts}} \right] \in \mathbb{R}^4. \tag{3.3}$$

Ein erster Agent mit einem zweischichtigen MLP und PPO lernte darauf bereits nach rund 50 000 Frames einfaches Fahren auf der Strecke. Das belegt, dass Setup, Belohnungsfunktion, Streckenbegrenzungen, Basisphysik, Steuerung und Trainingspipeline grundsätzlich funktionieren. Streckenbegrenzungen, Basisphysik, Steuerung und Trainingspipeline grundsätzlich funktionieren. Von dieser gesicherten Basis aus lassen sich Hypothesen verlässlich testen, weil ein Fehlschlag nicht mehr auf einen Grundlagenfehler zurückgeführt werden muss. Diese Vorgehensweise – erst eine billige Referenz herstellen, dann die eigentliche Frage stellen – erwies sich im späteren Verlauf als die wertvollste methodische Einzelentscheidung des Projekts (Abschnitt 3.2.7).

3.2.5 Systematische Erprobung der Eingabekonzepte

Der nächste Schritt war der systematische Vergleich der Beobachtungsformen. Die Varianten gliedern sich entlang zweier Achsen. Dabei werden unterschliedliche Bebobachtungsräume erzeugt mit steigender komplexität um zu testen, mit welchem visuellen Input ein Agent überhaupt lernen kann. **Zeitliche Tiefe** wird durch das aneinanderreihen mehrerer Frames dargestellt und erfordert die größten Netze im Test. *Single-Frame* bezeichnet die Bereitstellung eines einzelnen Frames um daraus einen Steuerungsentscheidung zu treffen. Zusätzlich wird unterschiedlicher Informationsgehalt des visuellen Input betrachtet: Die *globale Draufsicht* umfasst die gesamte Strecke. Der *fahrzeugfeste Ego-Crop* hält das Fahrzeug stets in der Bildmitte und nach oben ausgerichtet, während sich die Strecke relativ dazu bewegt. Um den Datenfluss zu reduzieren und die Netzgröße zu verringern wird ein **hybrider Input** getestet. Dabei wird ein Bild zusammen mit dem Lenkwinkel als normierter Zahlenwert übergeben.

Getestet wurde in der in Tabelle 3.3 angegebenen Reihenfolge.

Tabelle 3.3: Reihenfolge der Vergleichsläufe zu den Eingabekonzepten mit den zugehörigen Notebook-Zellen.

Nr.	Variante	Laufname	Zellen
1	Single-Frame, Ego-Crop	`02_Vision_SingleFrame`	29–30
2	Single-Frame, global	`02_Vision_SingleFrameGlobal`	31–32
3	Hybrid / Multi-Input	`03_Hybrid_MultiInput`	33–36
4	Frame-Stacking (Crop, global)	`04_Vision_FrameStack_3`, `07_Vision_Global_3Stack`	37–40

3.2.6 Physikalische Überarbeitung auf gemessene Realdynamik

Die zunächst nach Augenmaß eingestellte Fahrzeugdynamik wurde nachträglich durch eine Videoanalyse realer Carrera-Hybrid-Fahrzeuge ersetzt. Das Ziel war eine realitätsnahe Annäherung an das spätere 3D-Verhalten – konkret die Absicherung, dass der Machbarkeitsnachweis des Projekts nicht durch eine unphysikalisch übermotorisierte Modellierung entwertet wird. Die beiden Modellstände tragen die Kennungen `force_drag_v1` (geschätzt) und `measured_v2` (gemessen).

Messverfahren. Aus Videoaufnahmen mit 30 fps und einer Kalibrierung von 14 px $\hat{=}$ 35 mm wurden Weg-Zeit-Verläufe für Anfahr- und Bremsvorgänge extrahiert und daraus die Kennlinien der Längsdynamik gefittet (Abbildung 3.2). Das resultierende Modell lautet

$$\frac{\mathrm{d}v}{\mathrm{d}t} = \mathrm{gas} \cdot A(v) - \mathrm{brake} \cdot a_{\mathrm{brake}} - (r_0 + r_1 v)\,, \tag{3.4}$$

mit der haftungs- und motorbegrenzten Antriebskennlinie

$$A(v) = \min\left(0{,}30 + 0{,}506\,v,\quad 2{,}05 - 0{,}717\,v\right) \tag{3.5}$$

sowie $a_{\mathrm{brake}} = 0{,}167\,\mathrm{m/s^2}$, $r_0 = 0{,}05\,\mathrm{m/s^2}$ und $r_1 = 0{,}336/\mathrm{s}$.

Belastbarkeit der Messung. Nicht alle Parameter sind gleich gut belegt, und diese Unterscheidung ist für die Bewertung der Ergebnisse wesentlich:

- **Gut belegt** ist die Bremsverzögerung. Der gefittete Verlauf $a(v) = 0{,}217 + 0{,}336\,v$ trifft beide Bremsversuche mit einem RMSE von 0,4 cm bis 0,6 cm über 2,5 m Fahrstrecke; als konstanter Ersatzwert ergibt sich $0{,}68\,\mathrm{m/s^2}$.
- **Gut belegt** ist ferner der haftungsbegrenzte Ast der Beschleunigung.
- **Nicht bestimmbar** ist dagegen die Höchstgeschwindigkeit. Ein Profil-Test über v_∞ zeigt einen monoton fallenden Fehler ohne Minimum – die Daten bevorzugen den Grenzfall konstanter Beschleunigung, weil das Fahrzeug im Bildausschnitt nie sättigt. Der Modellwert $v_{\max} = 1{,}90\,\mathrm{m/s}$ ist damit eine *Annahme*; belegt ist lediglich, dass $1{,}844\,\mathrm{m/s}$ erreicht wurden.
- **Geschätzt** ist der Rollwiderstand r_0, da kein Ausrollversuch vorliegt.

Eine Plausibilitätsprüfung schloss zudem einen systematischen Fehler aus: Wäre die Kamera schräg auf die Bahn gerichtet, würde sich der Maßstab über die Bildhöhe ändern und eine

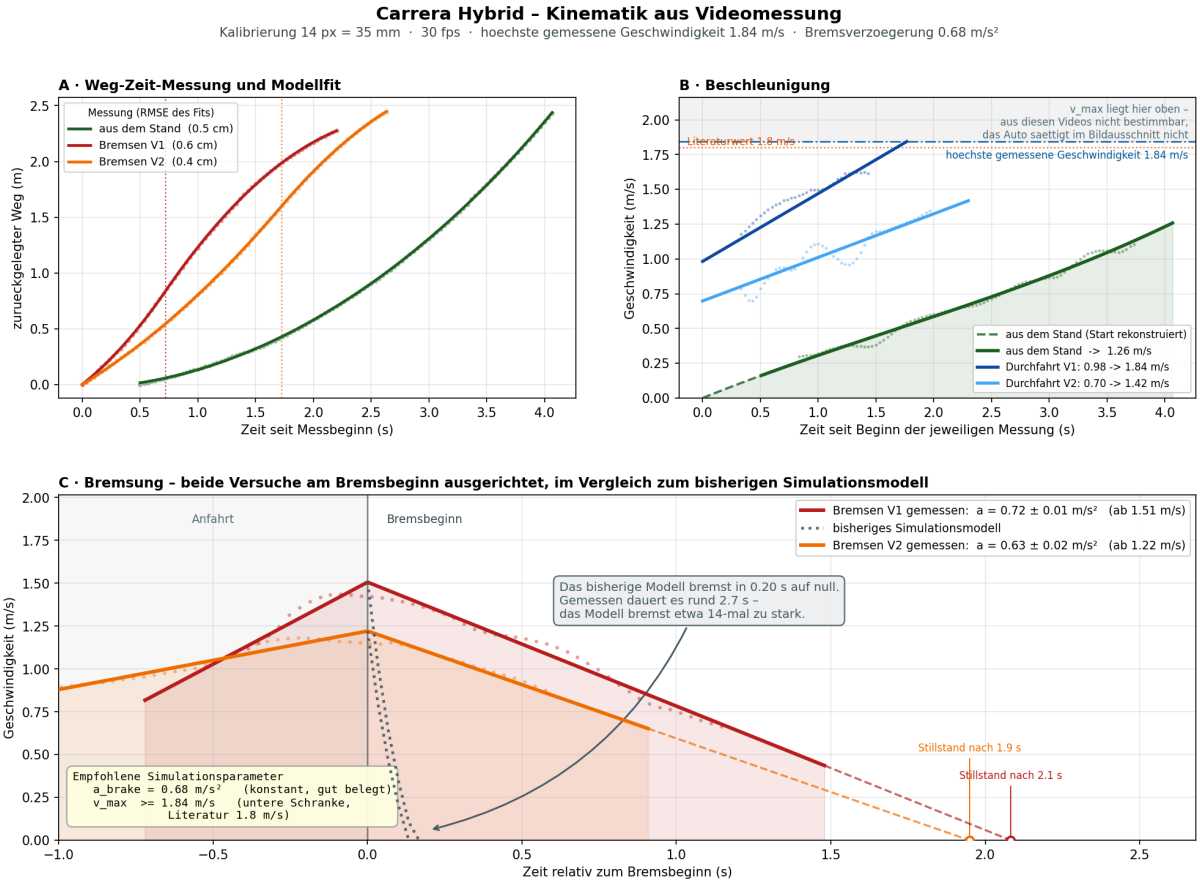


Abbildung 3.2: Kinematik aus der Videomessung. (A) Weg-Zeit-Messung und Modellfit für Anfahren und zwei Bremsversuche. (B) Daraus abgeleitete Geschwindigkeitsverläufe; die Höchstgeschwindigkeit ist aus diesen Aufnahmen nicht bestimmbar, weil das Fahrzeug im Bildausschnitt nicht sättigt. (C) Vergleich der gemessenen Bremsvorgänge mit dem bis dahin verwendeten Simulationsmodell, das rund vierzehnmal zu stark bremst.

Beschleunigung vortäuschen, die es nicht gibt. Da das Fahrzeug eine feste Länge hat, ließ sich das prüfen – die gemessene Fahrzeuglänge bleibt über die Bildhöhe mit 49,2 bis 49,4 px konstant. Dieselbe Prüfung deckte auf, dass eines der Messvideos unbrauchbar ist, weil dort ein Fremdobjekt im ausgewerteten Bereich liegt.

Auswirkung. Tabelle 3.4 stellt beide Modelle gegenüber. Die Endgeschwindigkeit war vorher ungefähr richtig, sämtliche Zeitkonstanten dagegen rund zwanzigfach zu kurz. Ursache war eine im Verhältnis zu den angesetzten Kräften viel zu kleine Masse.

Tabelle 3.4: Gegenüberstellung der beiden Fahrzeugmodelle. Die Werte für `measured_v2` sind Vollgas- beziehungsweise Vollbremswerte aus dem Stand.

Eigenschaft	`force_drag_v1`	`measured_v2`
Beschleunigung aus dem Stand	12,5 m/s²	0,25 m/s²
Bremsverzögerung	14,4 m/s²	0,68 m/s²
Höchstgeschwindigkeit	5 m/s bis 12 m/s	1,90 m/s (Annahme)
Rückwärtsfahren	erlaubt	gesperrt
Max. Lenkänderung je Frame	1,0 ($\hat{=}$ 67 ms)	0,5 ($\hat{=}$ 133 ms)
$0 \rightarrow 1{,}40$ m/s	$< 0{,}2$ s	119 Frames $\hat{=}$ 3,97 s
Pixelversatz je Frame (166×100)	8 bis 19 px	höchstens 3 px

3.2.7 Das Subpixel-Wahrnehmungsproblem

Unter der realistischen Physik konvergierten die visuellen Agenten zunächst überhaupt nicht mehr. Die Ursache ist eine optische Wahrnehmungsschwelle.

Herleitung. Die Beobachtung entsteht durch Herunterskalieren des 830×500 Pixel großen Streckenbildes auf 166×100 Pixel, also um den Faktor fünf. Der Maßstab in der Beobachtung beträgt damit

$$s_{\text{obs}} = 236\,\text{px/m} \cdot \frac{166}{830} = 47{,}2\,\text{px/m}, \tag{3.6}$$

ein Bildpunkt entspricht also 21,2 mm. Damit sich das Fahrzeug zwischen zwei aufeinanderfolgenden Einzelbildern um mindestens einen Bildpunkt verschiebt, ist eine Mindestgeschwindigkeit von

$$v_{\min} = \frac{1}{s_{\text{obs}} \cdot \Delta t} = \frac{30\,\text{Hz}}{47{,}2\,\text{px/m}} = 0{,}64\,\text{m/s} \tag{3.7}$$

erforderlich. Tabelle 3.5 zeigt den Verlauf.

Tabelle 3.5: Bildverschiebung je Einzelbild in der Beobachtung des Agenten, für beide erprobten Auflösungen.

Geschwindigkeit	166 × 100	250 × 150
0,10 m/s	0,16 px	0,24 px
0,30 m/s	0,47 px	0,71 px
0,50 m/s	0,79 px	1,18 px
0,80 m/s	1,26 px	1,90 px
1,90 m/s	2,99 px	4,50 px
Schwelle (1 px/Frame)	0,64 m/s	0,42 m/s

Warum das den Agenten blockiert. Das Fahrzeug ist in der Beobachtung lediglich 4,6 × 1,8 Pixel groß. Es startet aus dem Stand und benötigt unter `measured_v2` rund 2,8 s – etwa 84 Einzelbilder –, um 0,6 m/s zu erreichen. In diesem gesamten Bereich sind die drei gestapelten Bilder *identisch*: Der Bildstapel enthält dieselbe Information dreimal. Der Agent müsste über viele Zeitschritte hinweg kontinuierlich Vollgas geben, bevor überhaupt ein optischer Unterschied entsteht, aus dem er auf seine eigene Beschleunigung schließen könnte. Er ist beim Anfahren also blind für seine eigene Bewegung – und wird zeitgleich von der Belohnungsfunktion für zu geringe Geschwindigkeit bestraft, denn $v_{\text{slow}} = 0{,}6\,\text{m/s}$ liegt fast exakt auf der Wahrnehmungsschwelle. Insgesamt verbringt der Agent rund ein Drittel seiner Zeit unterhalb dieser Schwelle.

Bemerkenswert ist die Einordnung: Das alte Fahrzeugmodell hat dieses Problem nicht verursacht, sondern *verdeckt*. Bei 5 m/s bis 12 m/s verschob sich das Fahrzeug um 8 bis 19 Pixel je Einzelbild, sodass der Bildstapel Bewegung unübersehbar zeigte. Die guten Zahlen aus der ersten Projektphase sind damit teilweise ein Artefakt dieses Modellierungsfehlers.

Der Kontrollversuch als Beweis. Um auszuschließen, dass die Ursache doch tiefer liegt – etwa in einer Instabilität von SAC, in der Belohnungsfunktion oder in der trägeren Physik –, wurde am 14.08.2026 ein Kontrollversuch gefahren (`12_Lidar_SAC_Kontrolle_20260814_1748`). Alle Randbedingungen blieben konstant: Physik `measured_v2`, Belohnungsfunktion `v3_rundenzeit`, Algorithmus SAC, dieselbe Pufferkonfiguration, dieselbe Zeitlimitbehandlung, derselbe Diskontfaktor. Getauscht wurde ausschließlich die Beobachtung: exakte Zahlenwerte nach Gleichung (3.3) statt 166 × 100 Pixel.

Das Ergebnis ist eindeutig (Tabelle 3.6): Der Agent löst die Aufgabe. Nach 50 000 Schritten erreicht er die vollen 2000 Frames, nach 70 000 Schritten über 5000 Belohnungspunkte, und der gesamte Lauf über 500 000 Schritte dauert 54 min. Der Kamera-Agent steht nach 5 000 000

Tabelle 3.6: Kontrollversuch, Auswertungsstand 14.08.2026. Alle Modelle wurden zur Vergleichbarkeit in derselben Umgebung und mit demselben Belohnungsstand neu gefahren (`python -m src.eval.vergleich`). Die Zahlen sind untereinander, aber nicht mit der späteren Nachauswertung in Tabelle 6.2 vergleichbar – siehe die Anmerkung zur Vergleichbarkeit in Abschnitt 6.2.

Modell	Beobachtung	Reward	Frames	Runden	∅ Tempo
Lidar-Experte (PPO)	exakt	6401,7	2000	11	1,22 m/s
Lidar SAC `1748`	**exakt**	6116,4	2000	11	1,19 m/s
Vision SAC `2210`	166 × 100 px	−30,3	256	0	0,81 m/s
Vision SAC `2147`	166 × 100 px	−266,5	174	0	0,62 m/s
Vision SAC `2331`	166 × 100 px	−349,8	33	0	0,12 m/s

Schritten und 27 h bei null Runden. Damit sind Physik, Belohnungsfunktion, Algorithmus, Replay-Buffer, Zeitlimitbehandlung und Diskontfaktor als Ursache ausgeschlossen. Übrig bleibt die Wahrnehmung.

Ablation: Welcher Teil der Wahrnehmung fehlt? Der Kontrollversuch allein beantwortet noch nicht, *welcher* Anteil der Beobachtung entscheidend ist – die Lidar-Beobachtung liefert zwei Dinge zugleich: die exakte Geschwindigkeit *und* die Streckengeometrie als fertige Abstände. Eine Ablationsstudie (`src/eval/ablation_lidar.py`) trennt beides, indem dem fertig trainierten Lidar-Agenten gezielt Teile seiner Beobachtung verdorben werden, während die Physik unverändert weiterläuft (Tabelle 3.7).

Zwei Befunde folgen daraus. Erstens trägt die **Geschwindigkeit** die Leistung: Jede Verfälschung – null, eingefroren, verrauscht oder auch nur auf vier Stufen quantisiert – kostet über 100 % des Ertrags und führt zu null Runden. Zweitens ist die **Geometrie robuster**: Verrauschte Abstände bei exakter Geschwindigkeit ergeben noch 1018,7 Punkte und 2,7 Runden. Der quantitative Anschluss ist auffällig – die Kamera-Agenten liegen mit −266.5, −349.8 und −30.3 genau im Bereich „Lidar-Agent mit beschädigtem Geschwindigkeitssignal“. Genau das sagt die Subpixel-Rechnung voraus.

Die Grenze dieser Aussage sei benannt: Geprüft wird eine Policy, die *mit* exakter Geschwindigkeit trainiert wurde und sie nachträglich entzogen bekommt. Ein von Anfang an ohne Geschwindigkeitsinformation trainierter Agent könnte eine andere Strategie entwickeln. Belegt ist „diese Policy hängt an der Geschwindigkeit“, nicht „keine Policy kommt ohne aus“.

Der Fehlerfall im Einzelnen. Auch dort, wo ein Kamera-Agent bereits Runden fährt, bleibt die Geschwindigkeitsauflösung die begrenzende Größe. Lauf `2230` (250 × 150) verunglückt

Tabelle 3.7: Ablation am trainierten Lidar-Agenten: gezielte Verfälschung einzelner Beobachtungsanteile.

Variante	Reward	Frames	Runden	∅ Tempo
unverändert	6116,4	2000	11,0	1,19 m/s
v auf null gesetzt	−347,5	146	0,0	0,53 m/s
v eingefroren auf 0,5 m/s	−191,4	187	0,0	0,71 m/s
v durch Rauschen ersetzt	−500,4	455	0,7	0,51 m/s
v **grob gerastert (4 Stufen)**	−197,9	192	0,0	0,70 m/s
Abstände auf null, v exakt	−3100,5	750	0,0	0,00 m/s
Abstände verrauscht, v exakt	1018,7	664	2,7	0,73 m/s

reproduzierbar an derselben Stelle – in vier von vier beobachteten Episoden außen in der linken Haarnadel (Abbildung 3.3). Der Vergleich mit dem Experten an genau dieser Stelle zeigt einen bemerkenswert kleinen Unterschied: 1,11 m/s gegenüber 1,04 m/s, also 7 %. Der Agent beschleunigt in die Kurve hinein, erreicht sein Höchsttempo unmittelbar davor und bremst dann zu spät voll; der Experte bremst rund 40 Frames vorher und sanft, der Agent 12 Frames vorher und mit voller Kraft.

Die Ursache ist weder mangelnde Sicht – er hat die Vogelperspektive der ganzen Strecke – noch ein zu kurzer Planungshorizont, denn 40 Frames liegen bequem innerhalb der 100 Frames, die $\gamma = 0{,}99$ abdeckt. Sie liegt in der Auflösung der Geschwindigkeit: Der Unterschied zwischen 1,04 m/s und 1,11 m/s beträgt in seinem Bild 0,17 Pixel. Der Agent sieht, *dass* er fährt, aber nicht hinreichend genau, *wie schnell*.

3.2.8 Technische Lösungsansätze gegen die Wahrnehmungsschwelle

Aus der Herleitung folgen unmittelbar zwei Hebel, die sich in ihren Kosten deutlich unterscheiden.

Temporales Spreizen des Framestapels. Statt der Bilder t, $t-1$, $t-2$ werden t, $t-4$, $t-8$ abgetastet. Die Verschiebung zwischen den gestapelten Bildern vervierfacht sich damit, und die Wahrnehmungsschwelle sinkt rechnerisch von 0,64 m/s auf 0,16 m/s – ohne zusätzlichen Rechen- oder Speicheraufwand, da der Replay-Buffer die Stapel ohnehin über Indexversätze zusammensetzt.

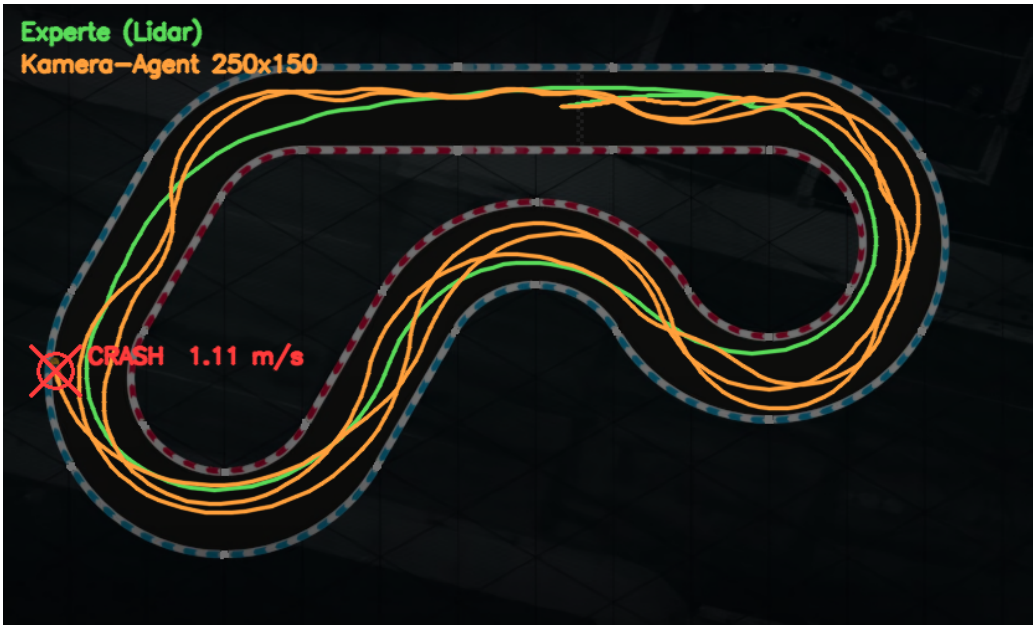


Abbildung 3.3: Fahrlinienvergleich. Grün die Linie des Lidar-Experten, orange mehrere Episoden des Kamera-Agenten `2230` bei 250 × 150. Der Kamera-Agent pendelt über die Streckenbreite und verunglückt reproduzierbar außen in der linken Haarnadel bei 1,11 m/s; der Experte durchfährt dieselbe Stelle mit 1,04 m/s. Erzeugt mit `src/eval/fahrlinien.py`.

Auflösungserhöhung. Eine Anhebung der Bildauflösung auf 250 × 150 Pixel senkt die Schwelle auf 0,42 m/s und vergrößert das Fahrzeug in der Beobachtung von 4,6 × 1,8 auf rund 7 × 3 Pixel. Der Preis ist erheblich: Der Durchsatz fällt von rund 135 000 auf 75 000 Schritten je Stunde, und der Replay-Buffer wächst von 15,5 GB auf 34,9 GB.

Ergebnis. Die Vorhersage lautete, dass das Spreizen der stärkere *und* der billigere Hebel sei. Die Auflösungserhöhung wirkte wie erwartet: Lauf `2230` ist der erste Kamera-Agent unter `measured_v2`, der überhaupt Runden fährt, und erreicht das bei einem Drittel der Schritte des Vorgängers. Der Ertrag je Frame verdoppelte sich nahezu, von 1,18 auf 2,01.

Das temporale Spreizen dagegen lieferte ein **negatives Ergebnis** und wurde zurückgenommen. Lauf `2307` mit Abstand 4 lag bei jeder Schrittzahl hinter dem Vorgänger, und der Abstand wuchs im Verlauf (Tabelle 3.8). Von 150 Auswertungen war keine einzige positiv.

Zwei Erklärungen bieten sich an, beide ungeprüft. Erstens die *Zuordnung*: Bei Abstand 1 überlappen sich die drei Fahrzeugabbildungen (4,6 px groß, 2,5 px Versatz) zu einem Schmierstreifen, dessen Form das Tempo codiert; bei Abstand 4 liegen sie rund 10 px auseinander und bilden drei

Tabelle 3.8: Gespreizter Framestapel gegen aufeinanderfolgende Bilder, beide bei 250 × 150 und 1,5 Mio. Schritten.

Schritte	Abstand 4 (2307)	Abstand 1 (2230)	Differenz
300 000	−361,9	−278,4	−83,5
700 000	−335,1	−153,6	−181,5
1 200 000	−281,4	67,9	−349,3
bester Einzelwert	−134,3	1181,2	
positive Auswertungen	0 von 150	23 von 150	

getrennte Flecken, die das Netz einander erst zuordnen muss. Zweitens die *Positionsunschärfe*: Das älteste Bild ist 0,27 s alt und beschreibt eine Lage, die für die aktuelle Entscheidung bereits überholt ist.

Einschränkend ist festzuhalten, dass dieser Vergleich zwei Variablen enthält: Neben dem Abstand wurde auch das Backbone neu erzeugt, weil der Datensatz zum Abstand passen musste; dabei wurde zusätzlich der Experte neu trainiert und der Aufzeichnungspfad umgestellt. Dass es am Abstand lag, ist plausibel, aber nicht bewiesen. Der Code bleibt erhalten (`src/utils/framestapel_gespreizt.py`, geprüft für die Abstände 1, 2, 4 und 8); die Voreinstellung ist Abstand 1.

3.2.9 Reward Engineering: Evolution zu `v3_rundenzeit`

Die Belohnungsfunktion erwies sich als die empfindlichste Komponente des gesamten Aufbaus. Vier Fehlerklassen traten auf, von denen zwei über Wochen unbemerkt blieben.

Die Sperrklinke (Ratchet-Bug). Jede Runde wurde gegen die bisherige Bestzeit der laufenden Episode gerechnet – und zwar jedes Mal aufs Neue. Eine einzelne schnelle Runde hob damit die Messlatte und kostete über den Rest der Episode mehr, als sie einbrachte:

sieben gleichmäßig langsame Runden	0
sieben Runden, davon eine schnelle	−250
sieben stetig schneller werdende Runden	300

Schnellfahren war unterm Strich also *bestraft*. Der mathematische Grund: Die Boni teleskopieren

– ihre Summe beträgt stets „erste minus beste Runde“ –, die Strafen dagegen nicht, sie fallen bei jeder Runde erneut an. Die relative Bestrafung wurde in `v3` ersatzlos entfernt.

Der Oszillations-Bug (w_{integral}). Eine Strafe gegen hochfrequentes Lenkzappeln bestrafte tatsächlich *jedes* Lenken. Die Schwelle war auf 2,0 gesetzt, während der Mittelwert der Lenksumme beim gut fahrenden Lidar-Experten bei 5,9 liegt. Dieser zahlte dadurch −625 Punkte über 2000 Frames – mehr als seine gesamten Rundenboni von +600. Der Parameter wurde auf 0,0 gesetzt; bei einer Wiederaktivierung wäre die Schwelle bei 6 bis 8 anzusetzen.

Die tote Zone. Eine harte Strafe unterhalb von 0,1 m/s in Verbindung mit einer quadratisch wachsenden Belohnung darüber ließ den Bereich dazwischen praktisch leer: Bei 0,15 m/s betrug die Strafe null und die Belohnung 0,045 je Frame. Genau dort landet das Fahrzeug beim Bremsen in Kurven. Ein lineares Gefälle anstelle der Klippe schließt die Lücke; dies ist der in Tabelle 3.2 sichtbare Posten aus v_{slow} und w_{slow}.

Absolute Strafen gegen skalierende Belohnungen. Der Tempo-Reward wächst mit v^2, die Lenkstrafen sind konstant. Der Gleichstand liegt bei $2v^2 = 0{,}58$, also $v = 0{,}54\,\text{m/s}$. Unterhalb davon kostet Lenken mehr, als Fahren einbringt – ein langsamer Agent hat damit den Anreiz, *weniger* zu lenken, und fährt geradeaus in die Wand.

Die feste Crash-Strafe macht die Kennzahl bimodal. Der wichtigste und am längsten übersehene Befund zeigt sich erst beim Zerlegen einer einzelnen Episode (Tabelle 3.9). Das *Fahren* ist längst positiv; was die Bilanz kippt, ist ein einziger Crash – eine feste Strafe, die nicht mit der Episodenlänge skaliert.

Daraus folgt eine Schwelle. Der Ertrag wächst linear mit der Episodenlänge n, die Strafe ist konstant:

$$R(n) \approx e \cdot n - 300, \qquad n_0 = \frac{300}{e}, \tag{3.8}$$

mit dem Ertrag e je Frame. Eine Regression über die Auswertungen ergibt für 250 × 150 einen Ertrag von 2,009 je Frame und damit eine Schwelle von 282 Frames, für 166 × 100 dagegen 1,182 und 465 Frames. Lauf `2210` lag mit typischen 200 bis 250 Frames weit unter seiner Schwelle und konnte sie praktisch nie erreichen – nur 16 von 500 Auswertungen waren positiv.

Die methodische Konsequenz ist gravierend: Die Kennzahl steigt nicht allmählich, sondern *kippt* an

Tabelle 3.9: Zerlegung einer Episode des Kamera-Agenten nach Belohnungsposten (Stand 850 000 Schritte, 250 × 150, 193 Frames).

Posten	Summe	je Frame
Tempo $2w_v v^2$	313,2	1,623
Langsam-Strafe	−145,7	−0,755
Lenk-Glättung	−26,7	−0,138
Rundenboni	45,3	0,235
Fahren zusammen	186,1	0,964
Crash	−300,0	−1,554
gemessen	−113,9	−0,590

dieser Schwelle. Ein Lauf knapp darunter produziert lauter negative Werte und gelegentlich, wenn eine Episode zufällig lang genug wird, einen weit positiven Ausreißer. Genau das ist der berühmte Spitzenwert von 875,7 bei Lauf `2210`: kein erworbenes Können, sondern ein Ziehungsergebnis. Zu berichten sind daher stets Spitzenwert, Endstand *und* die zugehörige Episodenlänge.

3.2.10 Der Diskontfaktor als teuerster Einzelfehler

Das Fahrzeug benötigt aus 1,3 m/s rund 1,9 s und 97 Frames bis zum Stillstand. Der Planungshorizont bei $\gamma = 0{,}99$ beträgt nach Gleichung (2.4) genau 100 Frames. Der Agent operierte damit exakt an der Grenze dessen, was er überhaupt vorausschauen konnte.

Die naheliegende Korrektur – γ auf 0,999 anzuheben – zerlegte das Training vollständig. Lauf `2331` brach nach 250 000 Schritten zusammen und lag danach 1,25 Mio. Schritte flach bei einer Belohnung von −355; der Agent fuhr 33 Frames weit. Die Ursache liegt in der Wertskala: Bei rund +2,9 Belohnung je Frame wächst die zu lernende Wertfunktion von etwa 290 auf etwa 2900, während die Lernrate unverändert bei 5×10^{-5} blieb. Der Critic muss also zehnfach größere Zahlen vorhersagen, ohne dass ihm dafür größere Schritte zugestanden werden.

Der Befund ist doppelt lehrreich. Erstens: Wer γ deutlich anhebt, muss Lernrate, Belohnungsskalierung oder beides mitdenken. Zweitens ist dies ein sauber dokumentierter Fehlschlag mit klar benennbarer Ursache – als nächster Versuch wäre $\gamma = 0{,}995$ mit einem Horizont von 6,7 s anzusetzen.

3.2.11 Systematischer Algorithmenvergleich: PPO gegen SAC

Der strukturelle Unterschied zwischen beiden Verfahren wirkt sich unmittelbar auf die Zahl der Gradientenschritte je gesammelter Erfahrung aus. Bei der hier verwendeten Konfiguration gilt:

SAC (off-policy)	**PPO (on-policy)**
`buffer_size` 500 000	`n_steps` 512 je Umgebung
`train_freq` 2, `gradient_steps` 1	`n_envs` 12 ⇒ 6144 Frames je Runde
`batch_size` 512 aus dem Puffer	`n_epochs` 10, `batch_size` 256
rund 3072 Gradientenschritte je 6144 Frames	240 Gradientenschritte je 6144 Frames, danach verworfen

Der Faktor beträgt somit rund 13, und SAC verwendet jede Erfahrung darüber hinaus mehrfach. Auf Bilddaten schlägt das voll durch: PPO erwies sich als extrem sample-ineffizient. Der einzige PPO-Lauf, der überhaupt messbar etwas gelernt hat, benötigte 90 Mio. Schritte – das 36-Fache des besten SAC-Laufs – und kam damit auf −3,9 Punkte gegen dessen 3828,2 (Tabelle 3.10).

Tabelle 3.10: PPO-Läufe vom 26.07.2026, alle unter dem alten Fahrzeugmodell `force_drag_v1`. `BSP` bezeichnet die Backbone-Transplantation.

Lauf	**Lernrate**	**BSP**	**Schritte**	**bester Wert**	**Frames**
`1601`	3×10^{-4}	ja	2,4 Mio.	−338,8	9
`2233`	3×10^{-4}	nein	2,5 Mio.	−235,7	—
`2328`	1×10^{-4}	nein	90 Mio.	−3,9	76

Umgekehrt ist SAC nicht ohne Schwächen. Auf Bilddaten neigt es zu Instabilitäten durch Critic-Divergenz. Im Lauf `1432` springt der Critic-Verlust unmittelbar nach dem Leistungshöhepunkt von 22,2 auf 26 670 – um den Faktor 1200 –, und die Leistung fällt von 3828,2 auf −1420,8. Bemerkenswert ist die Reihenfolge: erst das Minimum des Verlusts, dann die Explosion. Das passt zu einer Überanpassung des Critics an den Pufferinhalt, die zusammenbricht, sobald die Policy diesen Bereich verlässt. Der Lidar-Kontrollversuch zeigt dieses Verhalten *nicht* – dort blieb der Critic-Verlust bei höchstens 314 und endete bei 9,6, und ein Einbruch zwischen 410 000 und 450 000 Schritten erholte sich von selbst. Der Verdacht verschiebt sich damit von „SAC ist instabil“ zu „SAC wird instabil, wenn der Critic aus mehrdeutigen Bildern lernen muss“.

Zwei methodische Einschränkungen des Vergleichs sind zu nennen. Erstens liefen alle PPO-Läufe unter dem alten Fahrzeugmodell und ohne Versionskennung der Belohnungsfunktion; ein

Zahlenvergleich mit den SAC-Läufen unter `measured_v2` ist deshalb unzulässig. Zweitens enthält der bisherige Aufbau einen klaren methodischen Fehler: Stable-Baselines3 zählt `eval_freq` *je Umgebung*. Aus 50 000 bei zwölf Umgebungen wird damit eine Messung alle 600 000 Schritte – sechzigmal seltener als bei SAC mit einer Umgebung. Da das beste Modell nur bei einer Auswertung gesichert wird, ist der berichtete PPO-Bestwert systematisch eine untere Schranke. Eine saubere Wiederholung ist auf dem Branch `ppo-vergleich` vorbereitet; die noch offene Vergleichszelle ist die Kombination aus niedriger Lernrate und Backbone.

Für die Vergleichbarkeit bei gleichem *Zeitbudget* – dem für die Praxis sinnvolleren Maßstab, weil ein Schritt bei beiden Verfahren unterschiedlich teuer ist – gilt: Lauf `2139` (SAC, 1,5 Mio. Schritte) lief 11,17 h und erreichte 2427,5, Lauf `2328` (PPO, 90 Mio. Schritte) lief 12,15 h und erreichte −3,9. PPO hatte also sogar knapp eine Stunde mehr Rechenzeit. Erschwerend kommt hinzu, dass SAC rund 11 % seiner Schritte für Zwischenauswertungen verbrauchte, PPO praktisch keine.

3.2.12 Software- und Buffer-Optimierungen

Speicheroptimierter Replay-Buffer. Stable-Baselines3 legt die *fertig gestapelte* Beobachtung im Replay-Buffer ab, weil der Framestapel als Wrapper um die Umgebung realisiert ist. Benachbarte Übergänge teilen sich jedoch zwei ihrer drei Bilder – jedes Einzelbild liegt also etwa dreifach im Speicher. Bei 166 × 100 und 500 000 Übergängen ergibt das 23,2 GB.

Der eigens entwickelte Puffer `src/utils/framestapel_buffer.py` speichert stattdessen nur das jeweils neueste Einzelbild und setzt die Stapel erst beim Ziehen eines Minibatches aus Nachbarindizes zusammen. Der Speicherbedarf sinkt damit auf 15,5 GB.

Zwei Feinheiten sind dabei entscheidend und wurden testgetrieben abgesichert. Erstens die **Episodengrenzen**: Überschreitet ein Index eine Episodengrenze, wird das älteste Bild der neuen Episode dupliziert, damit kein Bild aus einer Vorepisode in den Stapel gerät. Zweitens die **Naht des Ringpuffers**: Genau $n_{\text{stack}} - 1$ Indizes ab der Schreibposition greifen beim Zusammensetzen auf Einträge zu, die bereits überschrieben wurden. Bemerkenswert ist weniger der Schaden – zwei von 500 000 Indizes, also 0,0004 % – als die *Testlücke*: Beide ursprünglichen Tests liefen mit halbleerem Puffer, der Ring lief nie um, der fehlerhafte Pfad wurde nie ausgeführt, und beide Tests meldeten bitgleiche Ergebnisse. Der erweiterte Test erzwingt nun einen 1,75-fachen Umlauf.

Bei der Bewertung der Ersparnis ist Sorgfalt geboten. Gegen einen Standardpuffer *ohne* Spei-

cheroptimierung beträgt der Faktor 3,0; gegen den tatsächlichen Vorzustand, der bereits mit `optimize_memory_usage=True` lief, jedoch nur 1,5. Wer eine Ersparnis angibt, muss dazusagen, wogegen. Die Umstellung kostet zudem 11,8 % Rechenzeit je Update (49,1 ms auf 54,9 ms), weil der Stapel beim Ziehen erst entstehen muss. Die gezogenen Minibatches sind bitgleich mit denen des Standardpuffers.

Ein naheliegender weiterer Schritt – *Bit-Packing*, also vier Pixel à zwei Bit in ein Byte zu packen – wurde gemessen und verworfen: Es spart zwar den Faktor 4 (5,8 GB statt 23,2 GB), kostet aber 28,8 ms Entpackzeit bei *jedem* Gradientenschritt, also 59 % Aufschlag. Aus zwölf Stunden Training würden neunzehn.

Korrekte Behandlung des Zeitlimits. Eine Episode kann aus zwei grundverschiedenen Gründen enden. Bei einem Crash ist der Folgezustand absorbierend, sein Wert ist tatsächlich null. Beim Erreichen des Zeitlimits von 2000 Frames ist er das *nicht* – das Fahrzeug wäre weitergefahren, wir haben lediglich aufgehört hinzusehen. Für das Lernziel des Critics macht das den Unterschied zwischen

$$y_t = r_t \qquad \text{und} \qquad y_t = r_t + \gamma\, Q(s_{t+1}, a_{t+1}). \tag{3.9}$$

In diesem Projekt stand `handle_timeout_termination` auf `False` – nicht absichtlich, sondern weil `optimize_memory_usage=True` es in Stable-Baselines3 erzwingt. Beide Optionen sind dort unvereinbar, und es gibt keine Warnung: Der Lernfehler kam als stillschweigende Nebenwirkung einer Speicherentscheidung herein.

Der Schaden bleibt dabei nicht lokal, und der Grund ist die Beobachtung selbst: *Der Agent sieht keine Uhr.* Seine Beobachtung ist ein Kamerabild ohne Framezähler und ohne Restzeitangabe. Für einen Agenten, der saubere Runden fährt, sieht Frame 1990 identisch aus wie Frame 300 – dieselbe Stelle der Strecke, dieselbe Lage. Der Critic erhält für denselben Eingang widersprüchliche Ziele und mittelt sie. Der falsche Nullwert klebt damit nicht am Episodenende, sondern verschmiert über alle Zustände, die *gut* aussehen. Verschärfend ist die Richtung der Korrelation: Ein schlechter Agent crasht früh und erreicht das Limit nie, ein guter fährt durch und erreicht es immer. Der Schaden schaltet sich genau in dem Moment ein, in dem der Agent aufhört zu crashen.

Lauf `1432` hatte auf seinem Höhepunkt eine mittlere Auswertungs-Episodenlänge von exakt 2000. Als alleinige Erklärung für die Einbrüche taugt der Mechanismus dennoch nicht: Im *Training* lag die Episodenlänge zum selben Zeitpunkt bei 398 Frames, weil die Exploration den Agenten von der Ideallinie schob. Es landeten also kaum Timeout-Übergänge im Puffer. Die Korrektur bleibt

richtig, ist aber nicht die Ursache.

Headless-Rendering via OpenCV. Ursprünglich erzeugte die Umgebung ihre Bilder über Pygame. Bei zwölf parallelen Trainingsumgebungen öffnete jeder Unterprozess ein eigenes Fenster, und Windows gingen die Fenster-Handles aus. Die Umstellung auf reines OpenCV- und NumPy-Rendering behob das.

Beiläufig deckte diese Umstellung ein weit gravierenderes Problem auf. Beide Renderpfade unterscheiden sich in lediglich 0,05 % der Pixel – reine Kantenglättung des Fahrzeug-Sprites, denn OpenCV glättet, Pygame nicht. Das genügt, um die Aktionen des Netzes vollständig zu ändern:

	Gas	**Bremse**	**Lenkung**
OpenCV-Bild	0,488	0,974	0,302
Pygame-Bild	0,875	0,908	−0,801

Derselbe Agent fuhr im Trainings-Renderpfad 256 Frames und 6,9 m, im anderen 0,00 m. Erklärbar ist das über die Größenordnung: Die acht abweichenden Pixel entsprechen ungefähr dem *gesamten Fahrzeug* in der Beobachtung – aus Sicht des Netzes war das Auto schlicht weg.

3.3 Erkenntnisse (Gut und Schlecht)

Oszillierendes Stellverhalten. Erste Agenten zeigten teils starkes Oszillieren: Lenkung und Gas sprangen praktisch binär zwischen -1 und $+1$. Für den Realtransfer ist ein solches Verhalten unbrauchbar, da reale Servos träger sind und vorausschauend gelenkt werden muss. Die Gegenmaßnahmen waren die Begrenzung der Lenkänderungsrate auf 0,5 je Frame und die Lenk-Glättungsstrafe w_{smooth} in der Belohnungsfunktion.

Vernachlässigte Reifenhaftung. Der Grip wurde anfangs nicht modelliert und musste nachgebessert werden. Ohne das Haftungslimit $\omega_{\max} = \mu g/|v|$ folgte das Fahrzeug der Lenkvorgabe exakt, statt bei zu hoher Kurvengeschwindigkeit zu untersteuern – ein Verhalten, das auf realer Hardware nicht existiert.

Konvergenzverhalten der Eingabevarianten. Einige Varianten konvergieren nur sehr schwer oder gar nicht. Das betrifft insbesondere die Single-Frame-Eingaben, was aus der POMDP-Perspektive erwartbar ist: Ohne Zeit- oder Telemetrieinformation existiert keine eindeutige Abbildung von Beobachtung auf richtige Aktion.

Die fahrzeugfeste Ansicht wurde durch die Physikkorrektur nachträglich unbrauchbar. Das Fahrzeug wäre darin mit 12,9 × 5,0 px achtmal so groß wie in der globalen Ansicht, und die Subpixelgrenze sänke auf 0,23 m/s. Der Ausschnitt zeigt jedoch nur 0,64 m Kantenlänge, also 0,32 m nach vorn – etwa 8 % einer Runde. Gegen den gemessenen Bremsweg gehalten ist das bei *jedem* Tempo zu kurz: aus 0,5 m/s bräuchte es 0,38 m, aus 1,2 m/s bereits 1,52 m. Der Agent sähe die Wand grundsätzlich erst, wenn Bremsen nicht mehr ausreicht. Mit dem alten Fahrzeug (Bremsweg 6 cm bei 1,5 m/s) war die Ansicht brauchbar – deshalb funktionierte sie im Juni. Das Grundproblem ist strukturell: Mehr Pixel für das Fahrzeug bedeuten bei mitgeführtem Ausschnitt zwangsläufig weniger Sichtweite; beides zugleich geht nur über die Auflösung.

Das Henne-Ei-Problem hochdimensionaler Eingaben. Bei sehr vielen Parametern und zufälliger Gewichtsinitialisierung scheitert das Lernen häufig an einer Rückkopplung: Da der Agent zu Beginn nicht fahren kann, sieht er fast ausschließlich Crashs und kann deshalb nicht lernen, welche visuellen Merkmale für das Fahren entscheidend sind – was wiederum verhindert, dass er fahren lernt. Die Vision-Policy bei 166 × 100 umfasst 16 261 098 Parameter, bei 250 × 150 rund 41 Mio.; der Lidar-Agent kommt mit rund 10 000 aus.

Ein vortrainiertes Netz hilft – aber anders als erwartet. Der praktische Befund ist positiv: Trotz des Risikos katastrophalen Vergessens konvergiert der Agent mit Vortraining aus Demonstrationen des stochastisch gestörten Lidar-Experten deutlich zuverlässiger als ohne.

Der genauere Blick relativiert jedoch, *was* dabei übertragen wird. Vom vortrainierten CNN werden nur die drei Faltungsschichten kopiert; die beiden Linear-Schichten – also genau der Teil, der aus Bildmerkmalen Gas, Bremse und Lenkung macht – werden verworfen. Übertragen wird damit Sehfähigkeit, nicht Fahrwissen. Von den elf Runden des Lidar-Experten kommt beim Vision-Agenten nichts an.

Bemerkenswerterweise kann das vortrainierte Netz auch als Ganzes nicht fahren. Lässt man es selbst steuern (`src/eval/bc_fahren.py`), ergibt sich für alle drei je erzeugten Backbones dasselbe Bild: 24, 68 beziehungsweise 230 Frames, jeweils null Runden, jeweils Crash. Das ist überraschend, weil das Netz die Expertenaktionen sehr genau vorhersagt (MSE 0,0034) und

die zeitliche Information nachweislich nutzt – friert man den Stapel auf drei identische Bilder ein, steigt der Fehler um den Faktor 125 auf 0,426. Die Erklärung ist der Covariate Shift aus Abschnitt 2.1.5, quantifiziert über die Abweichung zwischen Schüler und Experte im selben Zustand: 0,046 auf dem Datensatz, 0,264 beim Selbstfahren, 0,644 allein auf der Lenkung. Zwei alternative Erklärungen – unterschiedliche Stapel-Konvention beim Reset und unterschiedlicher Renderpfad – wurden geprüft und ausgeschlossen.

Und dennoch: Lauf `1432` erreichte 3828,2 Punkte mit genau dem Backbone, das eigenständig 24 Frames weit fährt. RL kann also auf einer unbrauchbaren Fahrpolitik aufsetzen und trotzdem fahren lernen – die Faltungsschichten liefern brauchbare Augen, unabhängig davon, wie schlecht der Kopf ist. Offen bleibt allerdings, ob das Vortraining seinen Aufwand überhaupt wert ist: Eine Sondierung der übertragenen Merkmale (`src/eval/backbone_probe.py`) misst sie nicht besser als zufällige Anfangsgewichte (R^2 0,735 gegen 0,748), und ein SAC-Lauf *ohne* Backbone wurde nie gefahren. Für PPO existiert der Vergleich, für SAC nicht. Solange das aussteht, ist der Backbone-Schritt eine Gewohnheit, keine belegte Verbesserung.

Implizites Lernen von Geschwindigkeit funktioniert schlechter als erwartet. Die einschlägige Literatur und die vorbereitende Recherche legten nahe, dass ein CNN Geschwindigkeit und Beschleunigung aus einem Bildstapel zuverlässig implizit erlernt. Unter realistischer Fahrzeugdynamik und der hier verwendeten Auflösung trifft das nicht zu – die Voraussetzung dafür, eine messbare Pixelverschiebung, ist im relevanten Geschwindigkeitsbereich schlicht nicht gegeben.

Zielsetzung bleibt der globale Ansatz. Ziel des Projekts ist ein Agent, der menschliche Rundenzeiten schlägt und dabei eine mit einem menschlichen Fahrer vergleichbare Übersicht besitzt, also die gesamte Bahn sieht. Aus dieser Zielsetzung heraus wurde der global funktionierende Ansatz weiter verfeinert, obwohl der Ego-Crop lokal höhere Auflösung böte.

3.4 Fallen und Fehler

Die Belohnungsfunktion ist hochgradig sensibel. Änderungen führen häufig zu Rückschritten, weil neue Strafen oder Boni unvorhergesehene, gegenläufige Effekte auslösen. Die vier in Abschnitt 3.2.9 beschriebenen Fehlerklassen entstanden sämtlich aus lokal plausiblen Einzelentscheidungen. Besonders tückisch sind Wechselwirkungen zwischen Posten unterschiedlicher

Skalierung – eine konstante Strafe gegen einen quadratisch wachsenden Bonus verhält sich in verschiedenen Geschwindigkeitsbereichen qualitativ verschieden.

Angst vor der Ziellinie. Visuelle Agenten mieden die Ziellinie. Die Erklärung liegt in der Lernreihenfolge: Bis ein Agent die Ziellinie überhaupt zum ersten Mal erreicht, hat er über viele Episoden gelernt, dass dunkler Asphalt sicher ist und farbliche Abweichungen Begrenzungen darstellen. Die karierte Markierung der Start-/Ziellinie wird von diesem gelernten Merkmalsdetektor als Hindernis eingeordnet. Der Lidar-Agent zeigt dieses Verhalten nicht, weil seine Beobachtung keine Farbinformation enthält.

Eine feste Startposition erzeugt auswendig gelernte Trajektorien. Dies ist der wichtigste Einzelbefund des Projekts und wurde erst spät entdeckt. Alle Kamera-Läufe bis zum 20.08.2026 starteten am identischen Punkt, 14 Pixel vor der Ziellinie. Zusammen mit deterministischer Physik und deterministischer Auswertung lief damit jede Episode exakt gleich ab – erkennbar an einer Standardabweichung von exakt ±0,00 in sämtlichen Logs. Fünf Auswertungsepisoden lieferten fünfmal dasselbe Ergebnis; es wurde fünffacher Aufwand für eine einzige Messung bezahlt, ohne unterscheiden zu können, ob ein Wert typisch oder ein Glücksfall war.

Der Kreuzvergleich beider Agenten unter beiden Bedingungen, je zwölf Episoden, macht die Tragweite sichtbar (Tabelle 3.11).

Tabelle 3.11: Kreuzvergleich von festem und streuendem Startpunkt. Lauf `2254` wurde mit festem, Lauf `2215` mit streuendem Startpunkt trainiert; beide wurden unter beiden Bedingungen ausgewertet.

	fester Start		**streuender Start**	
Agent	Reward	Runden	Reward	Runden
`2254` (fest trainiert)	5080,5	9,0	−1829,2	0,6
`2215` (streuend trainiert)	1027,6	2,0	1545,7	3,1

Lauf `2254` sah mit 5080,5 Punkten und neun Runden großartig aus – aber nur von seinem einen Startpunkt aus. Aus zufälligen Lagen fällt er auf 0,6 Runden. Er hatte keine Fahrpolitik gelernt, sondern eine Bahn abgespult. Lauf `2215` ist unter beiden Bedingungen mittelmäßig, aber konsistent – und der einzige der beiden, der überhaupt *fahren* kann.

Daraus folgt eine unangenehme Konsequenz für die gesamte Messreihe: Alle Kamera-Läufe vor dem 20.08.2026 messen zu einem unbekannten Anteil Auswendiglernen statt Fahren, und zwar

umso stärker, je besser die Zahl aussieht. Rückwirkend erklärt der Befund auch den Sprung von 1506 auf 5080 nach der Reparatur des Rundenzeit-Bonus: Dieser Bonus belohnt exakt das Perfektionieren einer wiederholten Runde.

Die Korrektur streut den Startpunkt seither über 0,9 m entlang der Geraden vor der Ziellinie und 0,08 m nach jeder Seite (Abbildung 3.4). Ein Variieren der Startpositionen von Anfang an – und ein Start mit Anfangsgeschwindigkeit statt aus dem Stand – hätte die Konvergenz vermutlich beschleunigt und wäre die billigste aller Verbesserungen gewesen.



Abbildung 3.4: Streuung der Startpunkte nach der Korrektur. Alle möglichen Startlagen liegen auf der Geraden *vor* der Ziellinie, damit die Rundenzählung ihren Sinn behält; grün markiert sind zwölf tatsächlich gezogene Startpunkte.

Der Rundenzeit-Bonus hat vier Wochen lang nie gefeuert. Der Startpunkt lag 14 Pixel vor der Ziellinie, die erste Überquerung erfolgte daher nach neun Frames und wurde als Rundenzeit gewertet. Danach stand die Bestzeit bei etwa zehn Frames, und keine echte Runde – rund 172 Frames – konnte das je unterbieten. Der Parameter $w_{\text{imp}} = 5{,}0$ war damit vom 28.07. bis zum 17.08.2026 wirkungslos. Repariert: Die erste Überquerung vergibt den Rundenbonus, startet aber lediglich die Zeitmessung, ohne selbst eine Zeit zu setzen.

Der Seed wurde bei jedem Reset neu gesetzt. In dieser Konstellation zieht der Zufallsgenerator bei jedem Zurücksetzen dieselbe Zahl – streuende Startpunkte wären trotz Streuung

in jeder Episode identisch geblieben. Einmal säen genügt; die Folge der Ziehungen bleibt dabei reproduzierbar.

Auswertung auf anderer Hardware als das Training. Dasselbe Modell, dieselbe Umgebung, lediglich CPU statt GPU:

	Frames	Runden	Reward
CPU	216	0	−273,7
CUDA	2000	9	5080,5

Gleitkomma-Unterschiede in der Größenordnung 10^{-7} schaukeln sich über eine Episode zu einem völlig anderen Verlauf auf. Dies ist keine Fehlfunktion, sondern eine Eigenschaft der Policy – und ein deutliches Warnsignal für den späteren Realtransfer.

Protokollierung und Wirklichkeit laufen auseinander. Das durchgehende Muster fehlgeschlagener Nachvollziehbarkeit lautet: Ein Wert steht im Code und wird getrennt davon von Hand ins Protokoll geschrieben. Aufgetreten sind unter anderem folgende Fälle: Die drei Koeffizienten der Längsdynamik waren hartcodiert und tauchten in keiner Konfiguration auf, während `mass_kg` protokolliert wurde und Vollständigkeit vortäuschte. `n_envs` stand auf 1, das Modellarchiv nannte 3. `total_timesteps` stand zweimal mit verschiedenen Werten. `gamma` war gar nicht gesetzt, und Stable-Baselines3 nahm stillschweigend 0,99. Die Parameter `best_lap_bonus` und `slow_lap_penalty` wurden in jeder Konfiguration protokolliert, aber nie gelesen – die tatsächlich wirksamen Faktoren waren hartcodiert.

Automatisches Zurücksetzen verdeckt den Fehlerfall. Endet eine Episode, ruft der Vektorisierungs-Wrapper von Stable-Baselines3 sofort `reset()` auf – noch bevor die eigene Beobachtungsschleife `render()` erreicht. Der Crash-Frame wird damit nie gezeichnet; sichtbar ist bereits der Startzustand. Das führte zu einer völlig falschen Diagnose: Das Fahrzeug schien einfach stehen zu bleiben und neu zu starten. Tatsächlich fuhr es mit 1,19 m/s in die Außenwand und bremste die letzten sieben Frames vergeblich.

Die Auswertung selbst kostet Rechenzeit. SAC verbrauchte 11 % seiner Schritte für Zwischenauswertungen (150 Auswertungen × 5 Episoden = 167 905 Frames), PPO dagegen 4090 Frames.

Bei einem Vergleich nach Zeitbudget ist das zu berücksichtigen.

4 Umsetzung in 3D (Isaac Sim / Isaac Lab)

Phase 1 hat gezeigt, dass ein Agent lernen kann, ein Fahrzeug anhand eines Kamerabildes der Strecke und des Autos zu steuern. Die Umgebung ist dort jedoch zu Gunsten von Rechenzeit und Aufwand stark vereinfacht gestaltet. Aufgrund dessen sind die trainierten Agenten nicht dazu geeignet, ein reales Fahrzeug zu steuern.

Ziel dieses Kapitels ist daher, das Training in eine physikalisch realistische, dreidimensionale und realitätsnah gerenderte Umgebung zu übertragen und zu beleuchten, welche Anpassungen für die erfolgreiche Erweiterung der Simulation nötig sind. Als Simulationsplattform dient Isaac Sim; die Lernumgebung wird mit Isaac Lab [25] aufgebaut, das auf Isaac Sim aufsetzt und viele parallele Umgebungen simuliert. Dieses Kapitel beschreibt den Aufbau der Umgebung; Kapitel 5 beschreibt den Trainingsablauf und die beiden fahrenden Kamera-Agenten und ordnet deren Ergebnisse und Schwächen ein.

4.1 Einstieg in Isaac Sim und Isaac Lab

Um die Software zu testen, wird zunächst ein stark vereinfachtes Fahrzeugmodell implementiert. Dieses besteht aus einem Quader und vier Zylindern als Räder und ist in der Lage, auf einer einfachen ersten Teststrecke zu fahren. Dafür werden Funktionsfähigkeit von Gas, Bremse, Lenkung und Kollision bzw. die Physik der Fahrbahn getestet. Weiterhin wird getestet, ob die Simulation eine Kamera bereitstellt, deren Bild sich für die Beobachtung des Agenten eignet. Nach erfolgreichen Tests werden Auto und Strecke durch realitätsnahe Modelle ersetzt. Alle Versuche und Trainings laufen auf demselben Rechner (Tabelle 4.1).

Tabelle 4.1: Verwendete Hardware und jeweilige Aufgaben während des Trainings.

Komponente	Ausführung	Rolle im Training
Grafikkarte	NVIDIA GeForce RTX 5080, 16 GB Grafikspeicher (Architektur „Blackwell“)	Physik, Rendern der 16 Kamerabilder, Training des Netzes
Prozessor	AMD Ryzen 7 9800X3D, 8 Kerne / 16 Threads	Verwaltung des Trainings, Ziehen der Stichproben aus dem Erfahrungsspeicher
Arbeitsspeicher	64 GB DDR5-6000	Erfahrungsspeicher (bis 26,5 GB) und Simulator; knappste Ressource
Datenträger	NVMe-SSD	Projekt, Modelle, Trainingsprotokolle

4.2 Modellierung einer realitätsnahen Umgebung

Der Agent soll durch das Training in der Lage sein, das reale Carrera-Hybrid-Auto ohne oder mit geringem Nachtraining zu fahren. Um dies zu erreichen, muss die Umgebung, in der der Agent lernt, der Realität so nah wie möglich nachempfunden sein. Für einen Kamera-Agenten sind dabei vor allem zwei Dinge entscheidend: wie das Kamerabild aussieht und wie das Fahrzeug auf seine Eingaben reagiert. Die folgenden Abschnitte beschreiben, wie Fahrzeug, Fahrverhalten, Strecke, Kamera und Belohnung in Isaac nachgebildet werden.

4.2.1 Fahrzeug

Das simulierte Fahrzeug besteht in Isaac aus zwei getrennten Bestandteilen, dem sichtbaren Modell, das nur gerendert wird, und einem Kollisionsmodell, welches das physikalische Verhalten des Autos beschreibt. Diese Trennung wird gewählt, da die Physik nur eine einfache Kollisionsform benötigt und eine detaillierte Karosserieform die Berechnung unnötig verlangsamen würde.

Als sichtbares Modell dient ein frei verfügbares Low-Poly-Modell eines Porsche 911 GT2 RS Weissach [26] (Abbildung 4.1, links). Die Fahrzeuge der Carrera-Hybrid-Bahn basieren auf dem Porsche 991 GT3 R (Abbildung 4.1, rechts). Dabei handelt es sich um unterschiedliche Auto-Modelle, die sich jedoch sehr ähnlich sehen und nur im Detail unterscheiden. Bei einer Kameraauflösung von 250 × 150 Pixeln für die gesamte Strecke ist das Fahrzeug im Bild nur etwa 9 × 4 Pixel groß, sodass der Unterschied nicht erkennbar ist. Ein möglichst ähnliches Modell ist dennoch sinnvoll: Ändern sich später Kameraposition oder Auflösung, wird das Fahrzeug im Bild größer, und seine Form gewinnt an Bedeutung.

Das Kollisionsmodell des Autos ist weiterhin ein Quader mit vier Zylindern. Die Maße des Quaders



Abbildung 4.1: Sichtbares Fahrzeugmodell in Isaac Sim (links, Porsche 911 GT2 RS Weissach) und reales Fahrzeug der Carrera-Hybrid-Bahn (rechts, Porsche 991 GT3 R).

und der Karosserie wurden jedoch am realen Carrera-Hybrid-Auto gemessen, damit Masse, Schwerpunktlage und Radaufstandspunkte mit der Realität übereinstimmen (Tabelle 4.2); die Trägheitsmomente berechnet die Physik-Engine PhysX aus Masse und Form. Masse, Schwerpunkt und Reifenreibung beeinflussen, wie stark das Fahrzeug in Kurven schiebt oder ausbricht. Daher wurden sie so genau wie möglich übernommen.

Tabelle 4.2: Gemessene Fahrzeugdaten im Kollisionsmodell.

Größe	**Wert**
Länge × Breite × Höhe der Karosserie	92,5 × 38,4 × 21,5 mm
Radstand	49,65 mm
Spurweite vorne / hinten	26,0 mm / 24,5 mm
Raddurchmesser vorne / hinten	13,5 mm / 14,0 mm
Bodenfreiheit	1,85 mm
Schwerpunkt	3,75 mm hinter der Fahrzeugmitte, 10,1 mm über dem Boden
Gesamtmasse	28,8 g
Haftreibung Reifen vorne / hinten	0,9 / 1,0

Die Lackierung des sichtbaren Modells (eine Textur der Karosserie) lag während aller hier beschriebenen Trainings nicht vor, der Agent hat das Fahrzeug nur als einfarbige Form gesehen (Abbildung 4.2). Darauf geht der Abschnitt Einschränkungen ein.

4.2.2 Antrieb und Fahrverhalten

Anfangs beschleunigt und bremst das simulierte Fahrzeug sehr ruckartig und die Lenkung ist sehr direkt. Das reale Fahrzeug reagiert dagegen träge, da seine Elektronik das Fahrverhalten eines echten Autos nachbildet. Für den Längsantrieb wird deshalb das gemessene Modell aus

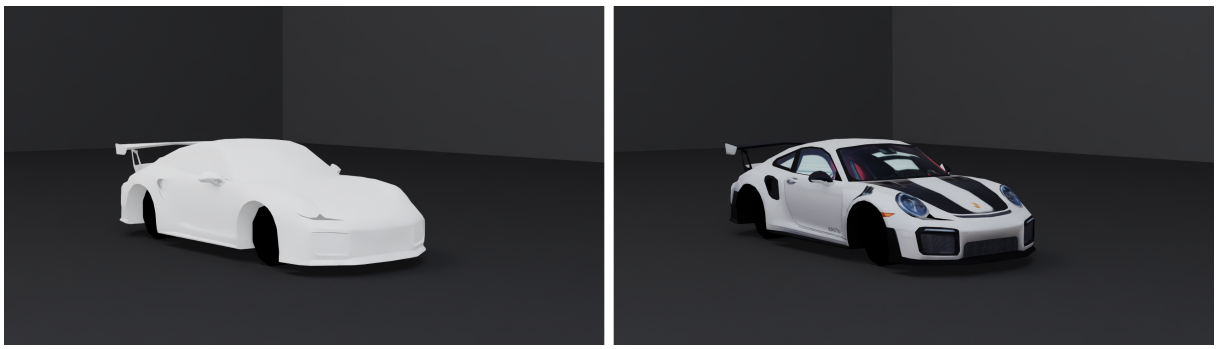


Abbildung 4.2: Fahrzeugmodell ohne Lackierung, wie es der Agent im Training gesehen hat (links), und mit Lackierung (rechts).

Phase 1 unverändert übernommen (Gleichung (3.4)). Die Lenkung ist auf einen Einschlag von 28,6° begrenzt, was annähernd den 30° der 2D-Umgebung entspricht.

Das Auto wird über drei Werte gesteuert, für Gas und Bremse im Bereich 0 bis 1 und für die Lenkung von -1 bis 1. Gas und Bremse sind dabei den analogen Pedalen eines Controllers nachempfunden, die Lenkung dem Analogstick. Werden beide Pedale gleichzeitig betätigt, wirkt nur das stärkere. Darin weicht die 3D-Umgebung von Phase 1 ab, in der sich Gas und Bremse überlagern (vgl. die offenen Punkte in Abschnitt **??**).

4.2.3 Strecke und Streckenbegrenzung

Als Strecke wird während des Trainings zunächst dieselbe Streckenführung wie in der 2D-Umgebung verwendet. Da sie keine Höhenunterschiede hat, wird das Streckenbild als flache, texturierte Ebene in die Szene eingefügt.

Für das Reinforcement Learning muss jederzeit feststehen, wo sich das Fahrzeug auf der Strecke befindet. Dazu werden per Bildverarbeitung die Begrenzungslinien aus dem Streckenbild extrahiert, geglättet und als Linienzüge gespeichert. Daraus werden vier Bereiche abgeleitet:

- die gültige Fahrbahn,
- die Randsteine links und rechts,
- der Bereich neben der Strecke,
- die Start- und Ziellinie.

Zur Kontrolle lassen sich die Linien in Isaac als dreidimensionale Wände anzeigen (Abbildung 4.3).

Im Training werden sie nicht geladen.

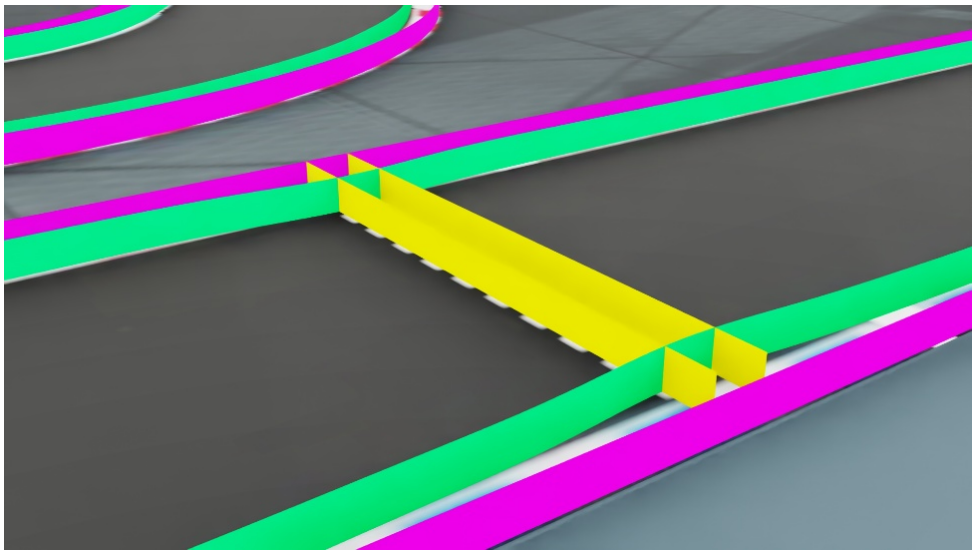


Abbildung 4.3: Begrenzungslinien als Kontrollanzeige in Isaac Sim (grün: Fahrbahnkante, magenta: Außenkante der Randsteine, gelb: Start- und Ziellinie).

Ob das Auto die Strecke verlassen oder eine Runde beendet hat und wie weit es auf der Strecke vorangekommen ist, wird rein geometrisch berechnet. Für die äußeren Aufstandspunkte der Reifen wird geprüft, ob sie noch auf der gültigen Fahrbahn liegen. Liegt einer davon außerhalb, gilt das als Crash, und die Episode endet. Die Randsteine zählen dabei bereits als außerhalb. Eine Mittellinie dient als Bezug für den Fortschritt entlang der Strecke. Weiterhin ist auf den letzten 0,9 m vor der Ziellinie eine Startzone definiert. Dort wird das Auto zu Beginn jeder Episode an einer zufälligen Stelle aufgestellt und bis zu 5 cm seitlich zur Mittellinie versetzt (Abbildung 4.4).

4.2.4 Kamera und Beobachtung

Der Agent bekommt die Informationen der Umgebung über eine simulierte Kamera, deren Blickwinkel dem eines real umsetzbaren Aufbaus entspricht. Die Kamera hängt fest montiert 3,05 m über der Mitte der Strecke und erfasst eine Fläche von 2,66 × 1,60 m, also die gesamte Strecke. Wie in Phase 1 wird das Bild auf 250 × 150 Pixel verkleinert und in Graustufen umgerechnet. Die Beobachtung besteht analog zu Phase 1 aus den letzten drei Bildern als Stapel (Abschnitt 2.1.7).

Ein Pixel deckt damit 1,06 cm der Strecke ab (93,9 px/m), das Fahrzeug ist im Bild etwa 9 × 4 Pixel

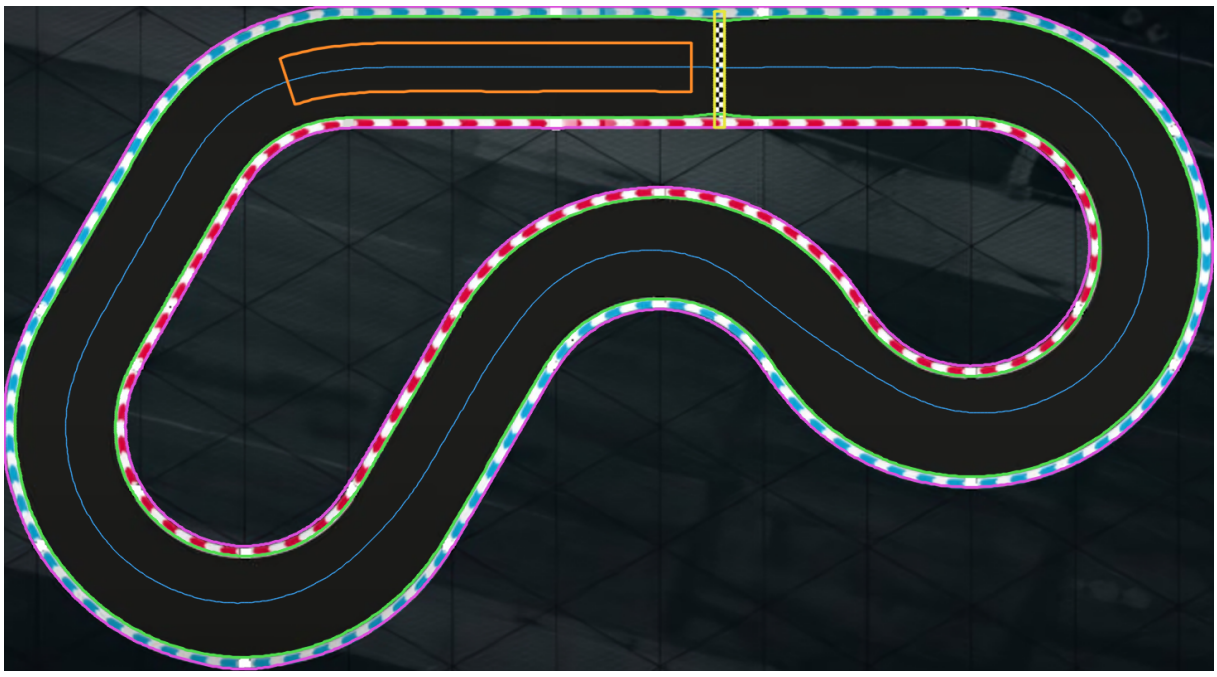


Abbildung 4.4: Trainingsstrecke mit den abgeleiteten Grenzen. Grün: Fahrbahnkante; Magenta: Außenkante der Randsteine; Gelb: Start- und Ziellinie; Orange: Startzone; Blau: Mittellinie.

groß, gegenüber 7×3 Pixeln in Phase 1 bei gleicher Auflösung. Nach der Wahrnehmungsschwelle aus Phase 1 (Gleichung (3.7)) muss sich das Fahrzeug zwischen zwei Bildern um mindestens einen Pixel bewegen, damit der Agent seine Bewegung im Bildstapel erkennen kann. Bei 30 Bildern je Sekunde liegt diese Schwelle hier bei 0,32 m/s, in Phase 1 bei 0,42 m/s. Die fahrenden Agenten bewegen sich mit 0,53 m/s bis 0,65 m/s, also 1,7 bis 2,0 Pixel je Bild, und damit deutlich oberhalb der Schwelle.

4.2.5 Belohnung und Episodenende

Die Belohnungsfunktion (`v3_rundenzeit`) wird unverändert aus Phase 1 übernommen (Gleichung (3.1), Tabelle 3.2) und wie dort aus den exakten Zuständen der Simulation berechnet, nicht aus dem Kamerabild. Neu ist nur ein Term für den Fortschritt entlang der Strecke. Tabelle 4.3 fasst die Terme zusammen.

Eine Episode endet nach 2000 Schritten (66,67 s), wenn das Auto die Fahrbahn verlässt (Crash) oder wenn die aufsummierte Belohnung unter -3000 fällt. Anschließend wird das Auto an einer neuen zufälligen Position in der Startzone aufgestellt.

Tabelle 4.3: Terme der Belohnung je Schritt.

Term	Wirkung	Gewicht
Geschwindigkeit	$v^2 \cdot 2\,w_v$, belohnt schnelles Fahren	$w_v = 1$
Langsam-Gefälle	lineare Strafe unter 0,6 m/s, voll bei Stillstand	$w_{\text{slow}} = 4$ je Schritt
Fortschritt (neu in 3D)	zurückgelegte Strecke entlang der Mittellinie	50 je Meter
Rundenbonus	einmalig je vollständiger Runde	+50
Rundenzeit	Verbesserung der eigenen Bestzeit, je eingesparten Schritt	5
Lenkglättung	Strafe je Lenkbewegung	0,5
Falsche Richtung	Grundstrafe je Schritt	−30
Crash	ersetzt alle anderen Terme, Episode endet	−300

4.3 Vergleich zur 2D-Python-Umgebung

Dieser Abschnitt fasst zusammen, welche Bestandteile der 2D-Umgebung unverändert in die 3D-Umgebung übernommen werden und welche angepasst werden.

4.3.1 Was übernommen und geändert wird

Tabelle 4.4 stellt die 2D-Umgebung aus Phase 1 und die 3D-Umgebung gegenüber. Die meisten Bestandteile werden unverändert übernommen; die Unterschiede ergeben sich vor allem aus der physikalischen Simulation, dem gerenderten Bild und den parallelen Umgebungen.

4.3.2 Parallele Umgebungen und Erfahrungsspeicher

Der größte technische Unterschied zwischen der Python-Umgebung und Isaac Lab ist die Parallelisierung der Umgebungen. In Isaac dauert ein Simulationsschritt mit gerendertem Kamerabild rund 96 ms. Jede weitere parallele Umgebung verlängert diesen Schritt jedoch kaum, da Isaac die Physik aller Umgebungen gemeinsam berechnet und alle Kamerabilder in einem Aufruf rendert: Vier Umgebungen benötigen nur 4 % mehr Zeit als eine, 16 Umgebungen rund 14 % mehr (109 ms).

Schritt und Übergang. Mit parallelen Umgebungen muss zwischen Schritt und Übergang unterschieden werden. Ein Schritt ist ein Simulationsschritt von $\frac{1}{30}$ s; ein Übergang ist ein einzelnes Erfahrungstupel aus Beobachtung, Aktion, Belohnung und Folgebeobachtung einer Umgebung

Tabelle 4.4: Bestandteile der 2D-Umgebung aus Phase 1 und der 3D-Umgebung im Vergleich.

Bestandteil	**Phase 1 (2D)**	**3D (Isaac)**
Simulation	kinematisches Modell in Python	Physik-Engine PhysX mit gemessenen Fahrzeugdaten
Bild	gezeichnetes 2D-Bild	gerenderte 3D-Szene mit Licht und Schatten
Beobachtung	3 Bilder à 250 × 150 Pixel, 4 Graustufen	übernommen; Agent 1 mit 256 Graustufen
Längsdynamik	gemessenes Antriebsmodell	übernommen
Pedale	Gas und Bremse überlagern sich	nur das stärkere Pedal wirkt
Streckengrenzen	Polygonüberschneidung	geometrische Prüfung der Reifenaufstandspunkte
Belohnung	`v3_rundenzeit`	übernommen, ergänzt um den Fortschrittsterm
Episoden	2000 Schritte, Abbruch unter −3000, zufällige Startpositionen	übernommen
Lernverfahren	SAC (Stable-Baselines3 [3]), NatureCNN, Lernrate 5×10^{-5}, $\gamma = 0{,}99$, 0,5 Updates je Übergang	übernommen
Erfahrungsspeicher	jedes Bild einmal abgelegt, Stapel beim Ziehen	übernommen; bei Agent 2 mit 2 bit je Pixel
Parallele Umgebungen	eine	4 (Agent 1) bzw. 16 (Agent 2)
Lehrer	Lidar-Strahlen	exakter Zustand aus der Simulation
Datenaufzeichnung	mit injiziertem Rauschen	deterministisch
Vortraining	Nachahmen der Lehreraktionen	Agent 1 wie Phase 1; Agent 2 auf die Lage des Fahrzeugs
Encoder zu Beginn des RL-Trainings	lernt sofort mit	eingefroren
Darstellung der Strecke	eine	Agent 2: zwölf Darstellungen und Bildstörungen

(Abschnitt 2.1.8). Bei 16 Umgebungen entstehen je Schritt 16 Übergänge. Trainingslängen werden im Folgenden in Übergängen angegeben, Episodenlängen in Schritten.

Lernen und Erfahrungsspeicher. Mehr Umgebungen beschleunigen jedoch nur das Sammeln von Erfahrung, nicht das Lernen. Damit das Verhältnis von 0,5 Updates je Übergang aus Phase 1 erhalten bleibt, muss die Zahl der Updates mit der Zahl der Umgebungen wachsen, und ein Update dauert mit rund 168 ms länger als ein Simulationsschritt. Außerdem teilt Stable-Baselines3 den Replay-Buffer gleichmäßig auf die Umgebungen auf.

5 Trainingsablauf in Isaac

Wie in Phase 1 gezeigt, ist es von Vorteil, das RL-Training nicht ohne Vorbereitung zu starten. Ein Kamera-Agent, der von Null an lernt, muss zwei Dinge gleichzeitig herausfinden: was die Pixel bedeuten und wie man fährt. Das einzige Signal dafür ist die Belohnung. Um das zu erleichtern, wird der Bildteil des Netzes vorab überwacht trainiert. Anschließend folgen die beiden fahrenden Kamera-Agenten, die im Verlauf der Arbeit entstanden sind, im Folgenden Agent 1 und Agent 2. Beide durchlaufen dieselben vier Stufen, unterscheiden sich aber im Ziel des Vortrainings und in den Darstellungen der Strecke: Agent 1 lernt auf einer einzigen Darstellung, Agent 2 auf zwölf.

5.1 Die Trainingspipeline in 3D

Entsprechend der Python-Umgebung wird auch hier eine mehrstufige Trainingspipeline verwendet, um den Ablauf des RL-Trainings effizienter zu gestalten. Diese Pipeline besteht aus vier separaten Stufen, wie in Abbildung 5.1 zu sehen.

Stufe 1 – Lehrer. Wie in Phase 1 wird zuerst ein Agent trainiert, der als Lehrer für den späteren bildbasierten Agenten dient (Abschnitt 2.1.6). Während der Lehrer in der 2D-Umgebung anhand von Lidar-Strahlen lernte, ist er hier zustandsbasiert. Er erhält den exakten Zustand aus der Simulation, also Geschwindigkeit, Drehrate, Richtungsfehler, Querversatz und die Abstände zu beiden Fahrbahnkanten, sowie seine letzte Aktion. Da Isaac alle diese Größen ohnehin kennt, ist es nicht nötig, Abstandsstrahlen nachzubilden.

Dieser Lehrer wird mit SAC auf 64 parallelen Umgebungen trainiert und fährt nach 768 000 Übergängen 8 Runden je Episode ohne Crash, mit einer Rundenzeit von ca. 8,3 s. Damit ist gezeigt, dass Physik und Belohnungsfunktion in 3D lösbar sind.



Abbildung 5.1: Trainingspipeline in vier Stufen. Oben der gemeinsame Ablauf, darunter die Unterschiede zwischen Agent 1 (blau) und Agent 2 (orange).

Stufe 2 – Datenaufzeichnung. Anschließend fährt der Lehrer in der Kamera-Umgebung. Zu jedem Schritt werden der Bildstapel und die Aktion des Lehrers gespeichert, insgesamt 48 000 Paare. Abweichend von Phase 1 fährt der Lehrer dabei deterministisch, ohne injiziertes Rauschen (Gleichung (2.9)), Vielfalt im Datensatz entsteht nur durch die zufälligen Startpositionen. Für Agent 2 stammen die Bilder stattdessen aus einer Fahrt von Agent 1, die in allen zwölf Darstellungen der Strecke gerendert wird (Abschnitt 5.6).

Stufe 3 – Vortraining. Vortrainiert wird der Teil des Netzes, der aus dem Bildstapel Merkmale berechnet, im Folgenden Merkmalsextraktor (Encoder). Er besteht wie in Phase 1 aus den drei Faltungsschichten des NatureCNN [12] und einer voll verbundenen Schicht mit 512 Neuronen. Für Agent 1 lernt das Netz analog zu Phase 1, die Aktionen des Lehrers nachzuahmen (BC, Abschnitt 2.1.5). Bestätigt wurde dabei die Warnung aus Abschnitt 6.1, dass ein niedriger Vorhersagefehler kein Fahrkönnen belegt: Von zwei Netzen fuhr das mit dem schlechteren Fehler ($R^2 = 0{,}25$ gegen $0{,}47$) 60 % weiter. Nach dem Vorhersagefehler wäre das schlechtere Netz ausgewählt worden. Vortrainierte Netze wurden deshalb im Fahrversuch bewertet, nicht am Verlust.

Für Agent 2 lernt der Encoder dagegen nicht die Aktionen, sondern nur die Lage des Fahrzeugs, Querversatz zur Mittellinie und Richtungsfehler, beide aus der Simulation exakt bekannt. Vortrainiert wird damit nur das Sehen, nicht das Fahren, wie auf eine Situation zu reagieren ist, lernt der Agent erst im RL-Training.

Stufe 4 – Online-Training mit SAC. Die vortrainierten Gewichte werden in das Netz des RL-Agenten übertragen und als Grundlage für das SAC-Training verwendet. Für Agent 1 werden nur die Faltungsschichten übernommen, für Agent 2 der gesamte Encoder. Der Kopf hinter dem Encoder, der aus den Merkmalen die Aktionen und ihre Bewertung berechnet, ist zu Beginn zufällig initialisiert.

Gerade am Anfang des Trainings sind die Gradienten des Reinforcement Learning stark verrauscht, der Agent verlässt nach wenigen Schritten die Fahrbahn, und der Critic kann den Wert einer Aktion kaum einschätzen. Lernt der Encoder in dieser Phase mit, überschreiben diese Gradienten das vortrainierte Sehen, bevor der Kopf gelernt hat, es zu nutzen.

Der Encoder wird deshalb zu Beginn eingefroren: Seine Gewichte bleiben für eine festgelegte Zahl von Übergängen unverändert, und nur der Kopf lernt. Danach wird der Encoder aufgetaut und lernt mit. Die Gradienten verfeinern das Sehen nun, statt es zu zerstören. Bei Agent 1 waren die Faltungsschichten bis 0,3 Mio. Übergänge eingefroren, bei Agent 2 der gesamte Encoder bis 2 Mio. Wie entscheidend dieser Schritt ist, zeigt ein Gegenversuch (Abschnitt 5.4).

5.2 Training des Kamera-Agenten

Der Kamera-Agent wird, wie in Phase 1 erarbeitet, mit SAC trainiert (Abschnitte 2.1.4 und 3.2.11); auch die Netzarchitektur, das NatureCNN, stammt aus Phase 1. Actor und Critic besitzen jeweils einen eigenen Encoder, dahinter je einen Kopf, der die drei Aktionen bzw. deren Bewertung berechnet.

Während des Trainings fährt der Agent parallel in mehreren Umgebungen. Jeder Übergang landet im Erfahrungsspeicher, aus dem nach jedem Simulationsschritt Stichproben für die Updates gezogen werden. Die Aktionen sind dabei zufällig gestreut, damit der Agent erkundet.

Als Eingabe erhält der Agent ausschließlich den Bildstapel. Belohnung, frühere Aktionen oder die Geschwindigkeit werden nicht zurückgeführt, die Belohnung dient nur als Lernsignal für den Critic.

Bewertung. Die Agenten werden nach dem Training deterministisch bewertet, also ohne Zufallsanteil der Aktionen und ohne Bildstörungen, mit zufälligen Startpositionen und getrennt je Darstellung der Strecke. Dabei werden Runden je Episode, mittleres Tempo, beste Rundenzeit

und der Anteil der Episoden, die mit einem Crash enden, gemessen. Die Werte aus dem Training eignen sich dafür nicht, weil die Aktionen dort zufällig gestreut und über alle Darstellungen gemittelt sind. Da gute Agenten in 66,67 s das Zeitlimit von etwa sechs Runden erreichen, wurden die fertigen Modelle zusätzlich mit 200 s langen Episoden gemessen.

5.3 Probleme auf dem Weg zum ersten Agenten

Bis zum ersten fahrenden Kamera-Agenten vergingen etwa 125 h Rechenzeit in Läufen, die nichts lernen konnten. Die wesentlichen Fehler traten ohne Absturz oder Warnung auf. Mehrere Fehler betrafen die Vortrainingskette: invertierte Lehrerbilder, Lehreraktionen außerhalb des gültigen Wertebereichs und das Speichern der letzten statt der besten Trainingsepoche.

Auch mit korrektem Bild lernte der Agent zunächst nicht zu fahren. Zum einen bestand eine Belohnungsfalle: Ohne den Fortschrittsterm brachte Geradeausfahren erst ab 0,52 m/s Pluspunkte, sodass ein schnelles Verlassen der Fahrbahn für den Agenten günstiger war als langsames Weiterfahren. Erst der Fortschrittsterm (Abschnitt 4.2.5) beseitigte diese Falle. Zum anderen verlor der Agent seine vortrainierte Sehfähigkeit. Das vortrainierte Netz fuhr allein 120 Schritte weit, der Agent nach 762 000 Übergängen nur noch 44 und damit kürzer als ein Fahrzeug, das ohne Lenken 86 Schritte weit fährt. Das RL hatte die vortrainierten Merkmale überschrieben.

5.4 Einfrieren des Encoders

Wie wichtig das Einfrieren des Encoders ist, zeigt ein Gegenversuch: zwei Läufe mit identischem Aufbau, einmal mit eingefrorenem Encoder bis 2 Mio. Übergänge und einmal ohne Einfrieren. Der Aufbau entspricht dem von Agent 2 (Abschnitt 5.6).

Neben der Episodenlänge werden dafür zwei Kennzahlen aufgezeichnet, beide auf einer Stichprobe von Bildern aus dem Erfahrungsspeicher. Die erste ist der Anteil inaktiver Neuronen der 512er-Schicht, also der Neuronen, die für alle Bilder der Stichprobe null ausgeben. Ein solches Neuron trägt keine Information über das Bild mehr, ist fast die gesamte Schicht inaktiv, erreicht den Kopf des Netzes kaum noch Bildinformation. Die zweite ist die Streuung des Lenkbefehls über die Bilder der Stichprobe. Ist sie null, gibt der Agent für jedes Bild denselben Lenkbefehl aus, er ist blind. In den üblichen Trainingskurven ist beides nicht zu erkennen, sie verlaufen lediglich flach. Tabelle 5.1 und Abbildung 5.2 zeigen den Verlauf.

Tabelle 5.1: Gegenversuch Einfrieren, Werte im Trainingsverlauf (Episodenlänge in Schritten / Anteil inaktiver Neuronen der 512er-Schicht / Streuung des Lenkbefehls).

Übergänge	ohne Einfrieren	eingefroren bis 2 Mio.
0,5 Mio.	33 / 89 % / 0,000	42 / 81 % / 0,99
2,0 Mio.	31 / 92 % / 0,000	58 / 63 % / 0,91
2,5 Mio.	31 / 96 % / 0,000	270 / 22 % / 1,49
2,95 Mio.	31 / 99,6 % / 0,000	444 / 18 % / 1,42

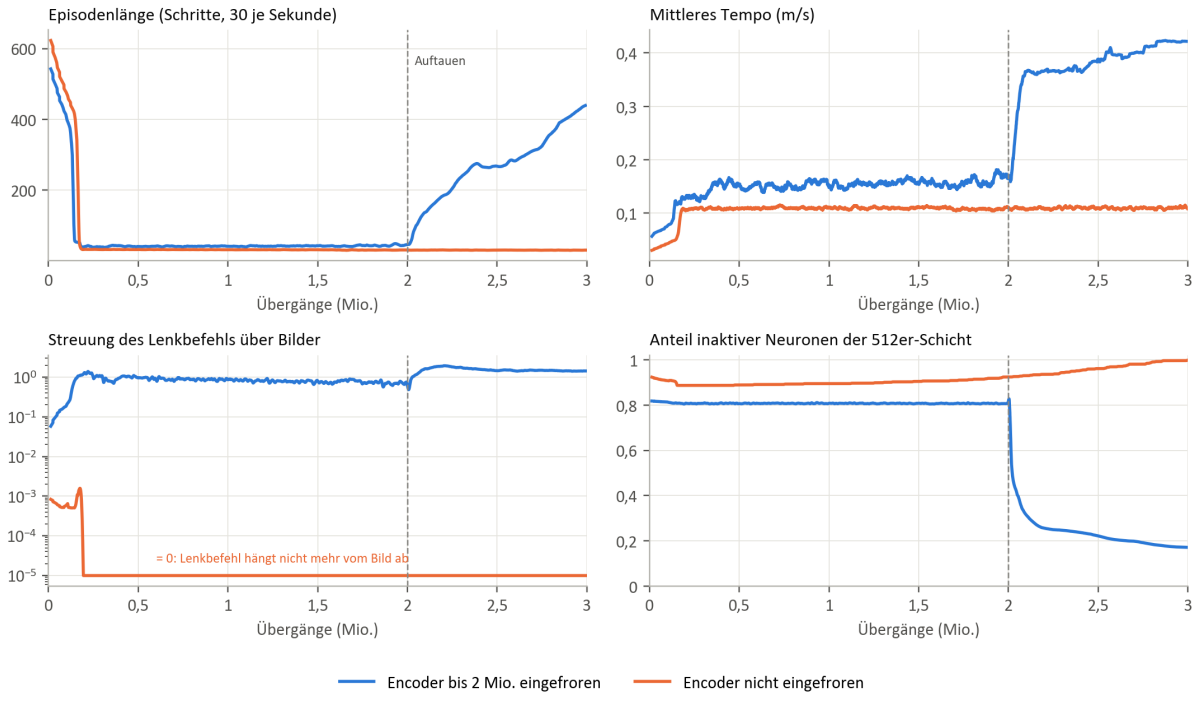


Abbildung 5.2: Gegenversuch Einfrieren: identischer Aufbau mit eingefrorenem (blau) und nicht eingefrorenem Encoder (orange) auf `kurs_A`.

Ohne Einfrieren wird der Agent blind. Schon nach einer halben Million Übergängen hängt der Lenkbefehl nicht mehr vom Bild ab, und die 512er-Schicht stirbt bis zum Ende fast vollständig ab. Der Agent fährt keine Runde, die Episoden enden nach etwa einer Sekunde. Mit Einfrieren bleibt das Sehen erhalten; nach dem Auftauen steigen Episodenlänge und Tempo, und der Agent fährt nach 2,9 Mio. Übergängen seine erste Runde.

Damit ist das Einfrieren als notwendig belegt. Ohne Schutz zerstört das RL den vortrainierten Encoder in den ersten Hunderttausend Übergängen, bevor der Kopf gelernt hat, ihn zu nutzen.

5.5 Der erste fahrende Kamera-Agent

Nach Behebung dieser Probleme gelingt mit Agent 1 das erste Training eines Kamera-Agenten, der zuverlässig Runden fährt.

5.5.1 Genutzte Parameter

Mit den Einstellungen aus Tabelle 5.2 gelang das erste Training eines Kamera-Agenten, der zuverlässig Runden fährt.

Gegenüber den vorherigen Läufen wurden zwei Dinge geändert: Statt 64 laufen nur noch 4 parallele Umgebungen, wodurch der Erfahrungsspeicher je Umgebung 69 statt 4 Minuten Fahrt enthält, und die Faltungsschichten bleiben bis 0,3 Mio. Übergänge eingefroren.

Tabelle 5.2: Einstellungen von Agent 1.

Einstellung	**Agent 1**
Parallele Umgebungen	4
Beobachtung	3 Bilder à 250 × 150 Pixel, 256 Graustufen
Darstellungen der Strecke	1
Vortraining	BC auf 48 000 Lehrerpaaren; übernommen: Faltungsschichten
Eingefroren	Faltungsschichten bis 0,3 Mio. Übergänge
Batchgröße / Updates	512 / 2 je Simulationsschritt (0,5 je Übergang)
Erfahrungsspeicher gesamt / je Umgebung	500 000 / 125 000 Übergänge (≈ 69 min Fahrt)
Lernrate / Diskontfaktor γ	5×10^{-5} / 0,99
Episodenlänge	66,67 s; in der Fortsetzung 133,33 s
Übergänge im fertigen Modell	2,2 Mio. (1,2 + 1,0)
Durchsatz	≈ 11 Übergänge/s (rund 38 h je 1,5 Mio.)

5.5.2 Das Ergebnis

Der erste erfolgreiche Agent fährt seine erste vollständige Runde nach rund 0,92 Mio. Übergängen. Da das Training auf 1,5 Mio. Übergänge ausgelegt ist und die Ergebnisse erstmals vielversprechend sind, wird es fortgesetzt: ausgehend vom Stand bei 1,2 Mio. Übergängen um weitere 1,0 Mio. Übergänge, mit doppelt so langen Episoden (4000 statt 2000 Schritte). Längere Episoden setzen den Schwerpunkt auf stabiles Fahren, da der Agent mehr Runden am Stück fahren muss.

Dieser Agent lernt auf einer Strecke mit stets gleicher Darstellung. Abbildung 5.3 zeigt den Verlauf beider Trainingsabschnitte.

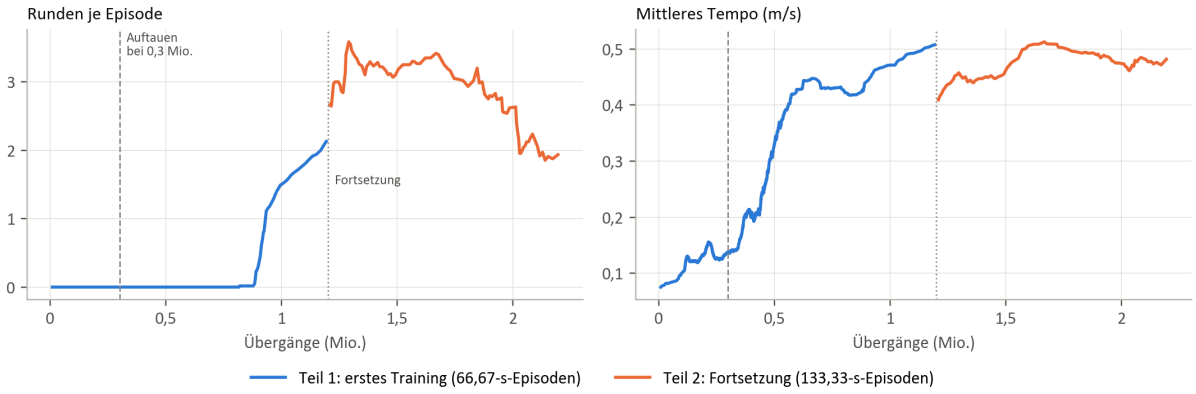


Abbildung 5.3: Trainingsverlauf von Agent 1 (Werte mit Explorationsrauschen). Blau: erstes Training, orange: Fortsetzung mit doppelt so langen Episoden; gestrichelt: Auftauen der Faltungsschichten.

Deterministisch gemessen, also ohne Explorationsrauschen und über zehn Episoden, fährt Agent 1 auf seiner Darstellung 6,4 Runden je Episode von 66,67 s, mit einer mittleren Geschwindigkeit von 0,65 m/s und einer besten Runde von 8,65 s. Ca. 10 % der Episoden enden mit einem Crash. Zum Vergleich fährt der zustandsbasierte Lehrer, der alle Größen exakt kennt, auf der gleichen Strecke etwa 8,3 s je Runde. Agent 1 ist also rund 4 % langsamer, obwohl dieser die Lage und Bewegungen des Fahrzeugs aus 9 × 4 Pixeln ablesen muss. In Episoden von 200 s fährt er im Mittel 10,4 Runden mit einer besten Runde von 8,59 s. In 3 von 10 Episoden fährt er rund 20 Runden ohne Crash, 70 % der langen Episoden enden jedoch vorzeitig. Der beste Bild-Agent der 2D-Umgebung schaffte unter vergleichbaren Bedingungen mit gestreuten Startpunkten 3,1 Runden.

Der kurze Einbruch zu Beginn der Fortsetzung in Abbildung 5.3 ist kein Verlernen. Stable-Baselines3 wählt in den ersten Schritten eines Laufs Zufallsaktionen, um den Speicher zu füllen;

bei der Fortsetzung waren das 50 000 Übergänge, in denen der Agent zufällig fuhr.

Ursache des Durchbruchs. Der Durchbruch dieses Agenten kann nicht eindeutig auf eine einzelne Änderung zurückgeführt werden. Gegenüber den vorherigen Läufen wird zum einen die Zahl der Umgebungen von 64 auf 4 reduziert, wodurch jede Umgebung einen größeren Anteil des Erfahrungsspeichers erhält, zum anderen werden erstmals die Faltungsschichten eingefroren. Damit wurde Regel 2 aus Phase 1, nur eine Änderung je Lauf (Abschnitt **??**), verletzt.

Einen Hinweis liefert der vorherige Gegenversuch zum Einfrieren des Encoders (Abschnitt 5.4). Dort erblindet der Agent ohne Einfrieren, obwohl jede Umgebung mit 104 min Fahrt über einen noch größeren Speicheranteil verfügt. Das spricht dafür, dass das Einfrieren den größeren Anteil am Durchbruch hatte. Für Agent 1 selbst ist es nicht belegt, da beide Änderungen nie getrennt untersucht wurden.

5.5.3 Eine Einschränkung

Um zu prüfen, wie weit Agent 1 verallgemeinert, wurde er ohne weiteres Training auf derselben Streckenführung in anderen Darstellungen eingesetzt. Schon eine andere Darstellung genügt, damit er versagt. Auf jeder geprüften Darstellung fährt er null Runden, und jede Episode endet mit einem Crash, meist nach etwa einer Sekunde, ohne dass der Agent in Fahrt kommt. Das ist kein Fehler des Agenten, sondern eine Folge des Trainingsaufbaus. Über mehr als zwei Millionen Übergänge hinweg war das Kamerabild bis auf die Lage des Fahrzeugs identisch. Unter diesen Bedingungen ist Auswendiglernen die einfachste Lösung, und genau diese hat das Training gefunden.

Das deckt sich mit dem Befund aus Phase 1, dass bereits 0,05 % abweichende Pixel eine Policy umwerfen können (Abschnitt 3.2.12).

5.6 Der zweite Agent

Dieser Abschnitt beschreibt den zweiten fahrenden Kamera-Agenten, im Folgenden Agent 2. Anders als Agent 1 soll er auch bei verändertem Aussehen der Strecke fahren. Dazu wechseln im Training die Darstellungen der Strecke, und der Encoder wird nicht auf die Aktionen eines Lehrers, sondern auf die Lage des Fahrzeugs vortrainiert.

5.6.1 Verallgemeinern der Erscheinung

Das Problem von Agent 1 ist, dass er das Erscheinungsbild der Strecke und des Hintergrunds auswendig gelernt hat. Da der Agent auf einer realen Bahn fahren soll, muss dies behoben werden, da das reale Bild niemals exakt der Erscheinung der Simulation entsprechen wird. Untergrund, Licht, Unebenheiten und Farbunterschiede in der echten Strecke unterscheiden sich von der Simulation. Der Abstand zwischen der Darstellung im Training und der beim Einsatz heißt Domänenlücke (engl. *domain gap*). Die übliche Abhilfe ist, im Training viele zufällig veränderte Darstellungen zu zeigen (*Domain Randomization* [27]).

Für die Randomisierung wurde ein Streckengenerator entwickelt, der Strecken aus Bildern der realen Carrera-Streckenteile zusammensetzt. Damit werden für dieselbe Streckenführung zwölf Darstellungen erzeugt, drei Untergründe kombiniert mit vier Helligkeitsstufen der Fahrbahn (Abbildung 5.4). Derselbe Generator erzeugt auch neue Streckenführungen, die in Abschnitt 5.8 verwendet werden.

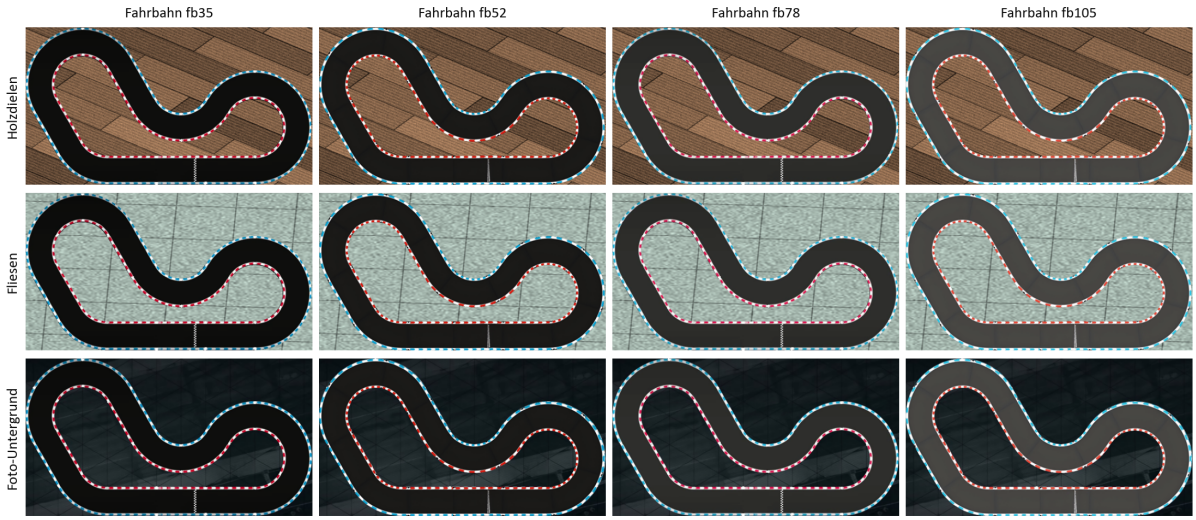


Abbildung 5.4: Die zwölf Darstellungen der Trainingsstrecke: drei Untergründe (Zeilen) und vier Helligkeiten der Fahrbahn (Spalten).

Weiterhin wird das Bild für Agent 2 auf vier Graustufen reduziert. Feine Unterschiede der Oberflächen verschwinden dadurch weitgehend, während Fahrbahn, Randsteine und Fahrzeug unterscheidbar bleiben. Zugleich lassen sich die Bilder mit 2 bit je Pixel im Erfahrungsspeicher ablegen.

5.6.2 Das Training

Agent 2 wird mit 16 parallelen Umgebungen trainiert. Der vortrainierte Encoder wird vollständig übernommen und bleibt bis 2 Mio. Übergänge eingefroren. Seine erste vollständige Runde fährt der Agent nach 3,9 Mio. Übergängen. Das fertige Modell wurde über 14,5 Mio. Übergänge in rund 92 h trainiert.

Abbildung 5.5 zeigt den Trainingsverlauf. Die Kurven enthalten das Explorationsrauschen und sind über alle Darstellungen gemittelt. Sie liegen daher deutlich unter den deterministisch gemessenen Werten. Zwischen 4,4 und 5 Mio. Übergängen stieg der Verlust des Critics um das Hundertfache, und das Fahren brach vorübergehend ein. Zeitlich fällt das mit den ersten Runden zusammen, deren Bonus die Zielwerte des Critics sprunghaft erhöht. Der Encoder blieb dabei intakt, und der Agent erholte sich ohne Eingriff. Das ist ein Hinweis zu der in Phase 1 offenen Frage, warum SAC-Läufe nach ihrem Höhepunkt einbrechen (Abschnitt **??**), aber keine Erklärung.

5.6.3 Das Ergebnis

Abbildung 5.6 zeigt das Ergebnis von Agent 2 nach 14,5 Mio. Übergängen auf allen zwölf Darstellungen. Im Mittel fährt er 4,5 Runden je Episode bei einem Tempo von 0,53 m/s. Die beste Runde dauert 9,65 s, und 31 % der Episoden enden mit einem Crash. Auf allen zwölf Darstellungen fährt er Runden, auf sieben davon im Mittel mindestens fünf.

Drei Darstellungen erreichen bei 66,67 s das Zeitlimit von sechs Runden in jeder Episode, die Messung zeigt dort also nicht das Können, sondern die Episodenlänge. Mit 200 s langen Episoden fährt Agent 2 auf diesen drei Darstellungen 19 bis 19,7 Runden ohne einen einzigen Crash, im Mittel über alle zwölf 12,4 Runden.

Gegenprobe mit Agent 1. Als Gegenprobe wurde Agent 1 auf denselben zwölf Darstellungen gemessen: Er fährt auf keiner einzigen eine Runde und verlässt nach etwa einer Sekunde die Fahrbahn. Die Verallgemeinerung von Agent 2 ist damit nicht nur behauptet, sondern gegen ein Modell ohne Randomisierung geprüft.

Wo die schwachen Darstellungen scheitern. Auf den Darstellungen mit vielen Crashes verlässt der Agent fast immer in derselben Kurve die Fahrbahn. Der Hergang ist jedes Mal gleich: Innerhalb

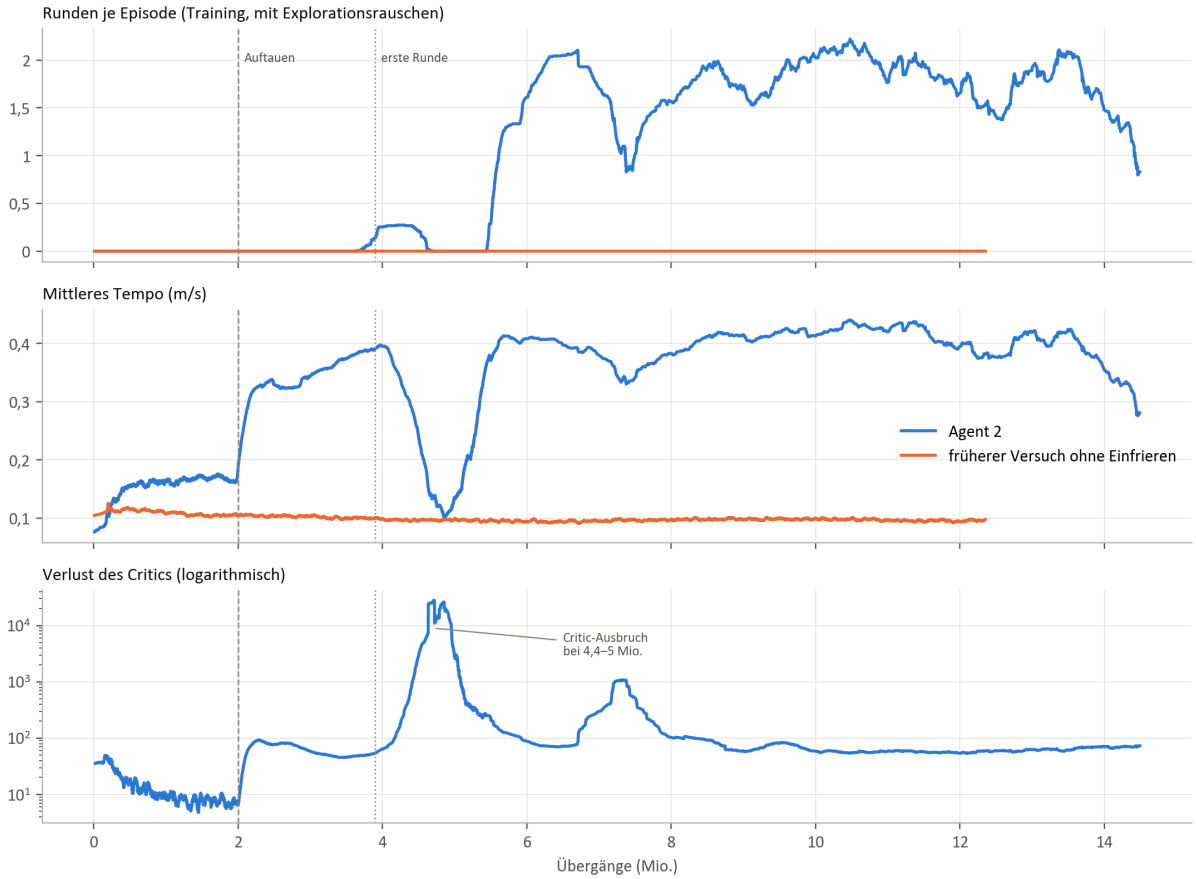


Abbildung 5.5: Trainingsverlauf von Agent 2 im Vergleich zu einem früheren Versuch ohne Einfrieren (Werte mit Explorationsrauschen). Gestrichelt: Auftauen; gepunktet: erste vollständige Runde.

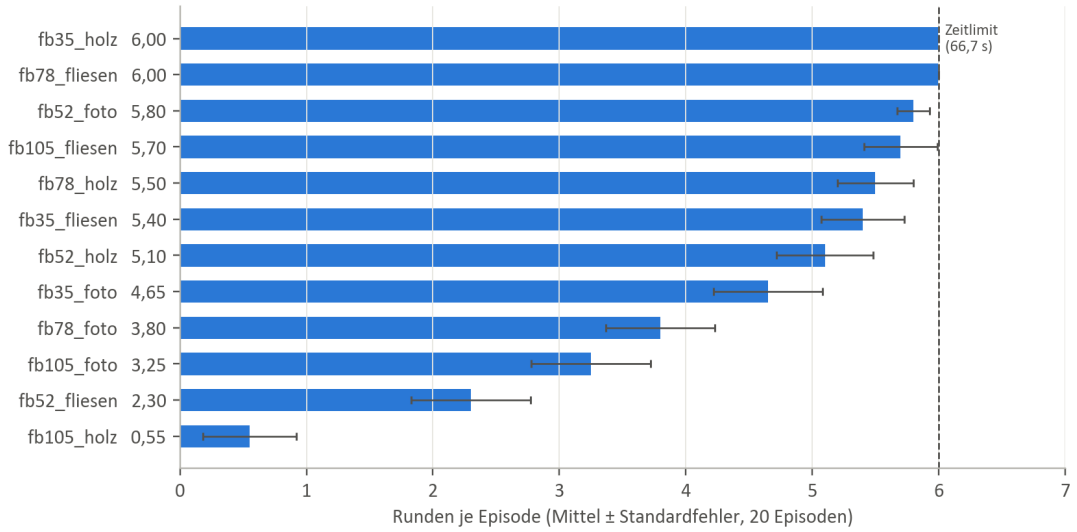


Abbildung 5.6: Agent 2 auf den zwölf Darstellungen (je 20 Episoden à 66,67 s, deterministisch). Bei 6 Runden liegt das Zeitlimit. Die Werte der unteren Darstellungen sind Grenzfälle und schwanken zwischen Messungen; für `fb105_holz` lagen sie in verschiedenen Messungen zwischen 0 und 4 Runden.

von 0,3 s wächst der Richtungsfehler von nahezu null auf über 30°, und der Gegenlenkbefehl kommt zu spät. Das Tempo ist dabei nicht die Ursache; auf einer sicher gefahrenen Darstellung durchfährt er dieselbe Kurve sogar schneller. Auch die Sichtbarkeit ist es nicht: Der Encoder liest die Lage auf der schwächsten Darstellung genauer ab als auf mancher sicher gefahrenen. Der Agent hat an dieser Stelle schlicht eine sehr kleine Reserve, sodass jede kleine Änderung der Eingabe das Ergebnis umkippen kann.

Feinere Graustufen. Dazu passt ein überraschender Befund. Wird das Kamerabild bei der Messung auf acht statt auf die trainierten vier Graustufen reduziert, sinkt die Crashrate auf der Darstellung `fb52_fliesen` von 80 % auf 0 %, ohne jedes Nachtraining, auch mit 16 und 256 Stufen gab es keinen Crash. Der Agent verträgt also feinere Eingaben als die, auf denen er trainiert wurde. Für die Übertragung auf eine echte Kamera mit 256 Graustufen ist das die günstige Richtung.

Lohnt sich das lange Training? Agent 2 wurde nach 6,5 und nach 14,5 Mio. Übergängen gemessen. Die Differenz von 0,54 Runden je Episode liegt innerhalb der Messunsicherheit (Standardfehler 0,61), und die spätere Fassung ist nur auf sieben von zwölf Darstellungen besser. Sicher nachweisbar ist nur, dass sie schneller fährt, auf allen zwölf Darstellungen, mit kürzeren

Rundenzeiten. Die zweite Hälfte des Trainings, rund 50 h, hat also im Wesentlichen das Tempo verbessert, nicht die Zuverlässigkeit.

5.7 Vergleich der beiden Modelle

Tabelle 5.3 stellt Agent 1 und Agent 2 gegenüber. Die beiden Modelle lösen verschiedene Aufgaben und sind deshalb nicht direkt vergleichbar. Agent 1 fährt auf einer Darstellung und ist auf diese spezialisiert. Agent 2 dagegen fährt auf 12 unterschiedlichen Darstellungen.

Tabelle 5.3: Die beiden fahrenden Kamera-Agenten.

	Agent 1	**Agent 2**
Aufgabe	1 Darstellung	12 Darstellungen mit Bildstörungen
Vortraining	BC eines Lehrers (Fahrwissen)	Encoder für Versatz und Richtung (nur Sehwissen)
Eingefroren	Faltungen bis 0,3 Mio. Übergänge	gesamter Encoder bis 2 Mio. Übergänge
Parallele Umgebungen	4	16
Graustufen	256	4
Speicher je Umgebung	69 min Fahrt	104 min Fahrt
Übergänge bis zur ersten Runde	0,92 Mio.	3,9 Mio.
Übergänge im fertigen Modell	2,2 Mio.	14,5 Mio.
Runden je Episode (66,67 s)	6,4 auf seiner Darstellung, 0 auf den 12 anderen	4,5 im Mittel über 12
Beste Runde	8,65 s	9,65 s
Episoden mit Crash	10 %	31 %

Neben der Aufgabe unterscheiden sich die beiden Agenten vor allem im Vortraining. Agent 1 hat Fahrwissen vom Lehrer geerbt, Agent 2 hat nur gelernt, wo das Auto auf der Bahn liegt, und das Fahren vollständig selbst gelernt. Dass Agent 2 viermal so viele Übergänge bis zur ersten Runde braucht, ist vor diesem Hintergrund und bei zwölffacher Aufgabe nicht überraschend.

5.8 Test des zweiten Agenten auf anderen Streckenführungen

Agent 2 fährt auf allen zwölf Darstellungen der Trainingsstrecke. Offen ist, ob er damit das Fahren an sich gelernt hat oder nur diese eine Streckenführung. Dazu wird er zunächst ohne weiteres Training auf neuen Streckenführungen getestet und anschließend auf zwei davon weitertrainiert.

5.8.1 Test ohne Nachtraining

Agent 2 wird ohne weiteres Training auf vier neuen Streckenführungen (`kurs_A` bis `kurs_D`) getestet, die mit dem Streckengenerator aus Abschnitt 5.6.1 erzeugt wurden. Damit sich nur die Führung verändert und nicht gleichzeitig das Aussehen, wurden für `kurs_A` dieselben zwölf Darstellungen erzeugt, auf denen Agent 2 trainiert wurde (Tabelle 5.4).

Tabelle 5.4: Agent 2 ohne Nachtraining (deterministisch).

Strecke	Darstellung	Runden je Episode
Original	trainiert (12)	4,5 (66,67 s) bzw. 12,4 (200 s)
Original	unbekannt	2,3
`kurs_A`	trainiert (12)	0,00 auf allen 12
`kurs_A` bis `kurs_D`	unbekannt	0,00

Eine unbekannte Darstellung der bekannten Strecke fährt der Agent noch mit 2,3 Runden. Eine unbekannte Streckenführung fährt er dagegen nicht, auch wenn sie in den bekannten Darstellungen gezeigt wird. Sein Verhalten auf `kurs_A` ist dabei kein schlechteres Fahren, sondern eine Sättigung, voller Lenkeinschlag, Vollgas und Vollbremse gleichzeitig, ohne jede Korrektur. Es ist dasselbe Muster wie bei den blinden Agenten, obwohl der Encoder hier intakt ist.

Besonders aufschlussreich ist, dass `kurs_A` die um 180° gedrehte Originalstrecke ist. Der Agent hat also nicht die Form der Strecke gelernt, sondern die Form in dieser Lage im Kamerabild. Für eine fest montierte Kamera über einer festen Strecke ist das kein Nachteil. Ein Agent, der jede beliebige Strecke fährt, entsteht so aber nicht.

5.8.2 Weitertrainieren von Agent 2 gegen Neustart

Da Agent 2 neue Streckenführungen ohne weiteres Training nicht fährt, wird untersucht, ob sein Vorwissen das Erlernen einer neuen Strecke zumindest beschleunigt. Dazu wird auf zwei Streckenführungen jeweils ein Weitertraining von Agent 2 mit einem Neustart verglichen:

Weitertraining: ausgehend vom fertigen Agent 2, also mit seinem Encoder und seiner Fahrweise;
Neustart: ein neuer Agent mit demselben vortrainierten Encoder wie zu Beginn von Agent 2, eingefroren bis 2 Mio. Übergänge.

Als Strecken dienen `kurs_A`, die gedrehte Originalstrecke, und `kurs_D`, deren Fahrbahnfläche sich im Kamerabild nur zu 21 % mit der Originalstrecke deckt. Gemessen wurde, nach wie vielen

Übergängen die erste vollständige Runde fällt (Tabelle 5.5, Abbildung 5.7).

Tabelle 5.5: Übergänge bis zur ersten vollständigen Runde.

Strecke	Weitertraining (von Agent 2)	Neustart (nur Encoder)	Ersparnis
`kurs_A` (Original, um 180° gedreht)	2,81 Mio.	2,90 Mio.	3 %
`kurs_D` (unähnlichste Führung)	2,92 Mio.	3,29 Mio.	11 %
zum Vergleich: Agent 2 auf der Originalstrecke	–	3,90 Mio.	–

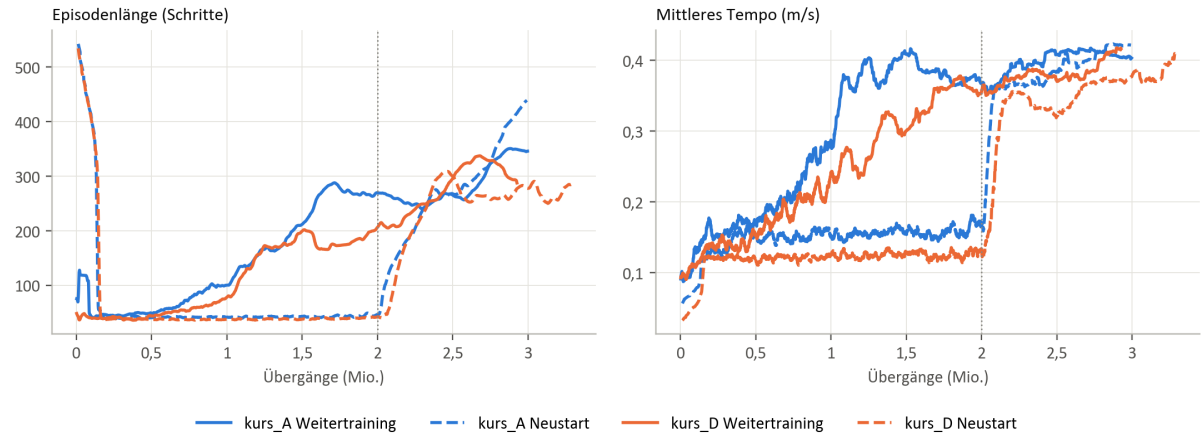


Abbildung 5.7: Weitertrainieren (durchgezogen) gegen Neustart (gestrichelt) auf `kurs_A` (blau) und `kurs_D` (orange). Bei 2 Mio. Übergängen wird der Encoder der Neustarts aufgetaut.

Das Vorwissen aus dem Training von Agent 2 bringt wenig: 3 % auf `kurs_A` und 11 % auf `kurs_D`. Da je Bedingung nur ein Lauf gefahren wird, kann ein Unterschied von 3 % auch zufällig entstehen. Abbildung 5.7 zeigt, woher der Unterschied kommt. Der weitertrainierte Agent lernt von Beginn an, weil sein Encoder nicht eingefroren werden muss, und ist nach 600 000 bis 800 000 Übergängen deutlich weiter. Sobald der Encoder des neu gestarteten Agenten bei 2 Mio. Übergängen aufgetaut wird, holt dieser den Rückstand innerhalb von etwa 600 000 Übergängen auf.

Übertragbar ist damit das *Sehen*, also die Fähigkeit, die Lage des Fahrzeugs im Bild abzulesen. Die *Fahrweise* ist nicht übertragbar. Der Agent lernt nicht das allgemeine Fahren, sondern diese eine Strecke in dieser einen Kameraansicht.

Zwei Einschränkungen gelten für diesen Vergleich. Zum einen wird der Encoder des neu gestarteten Agenten auf Bildern der Originalstrecke trainiert. Beide Bedingungen bringen also Vorwissen von der Originalstrecke mit, und gemessen wurde genau der Beitrag der Fahrweise. Wie viel der Encoder auf einer fremden Strecke beiträgt, wurde nicht gemessen. Ein Neustart ganz ohne Encoder wurde dort nicht gefahren. Zum anderen musste der Neustart auf `kurs_D` ab 3 Mio.

Übergängen mit leerem Erfahrungsspeicher fortgesetzt werden, weil sich der gesicherte Speicher nicht laden ließ. Das hat ihn vermutlich gebremst; die 11 % sind daher eher eine Obergrenze.

6 Ergebnisse

Das wesentliche Ergebnis der 2D-Untersuchungen ist keine einzelne Zahl, sondern eine *Pipeline*: eine Abfolge von Arbeitsschritten, mit der sich komplexere Agenten trainieren lassen, und die zugehörigen Prüfkriterien, an denen jeder Schritt gemessen wird. Dieses Kapitel stellt zunächst diese Pipeline in ihrem aktuellen Stand dar, belegt sie anschließend mit den gemessenen Werten und bewertet abschließend ihre Übertragbarkeit in die 3D-Simulation.

6.1 Die aktuelle Trainingspipeline

Die Pipeline gliedert sich in vier Stufen. Entscheidend ist nicht nur ihre Reihenfolge, sondern dass jede Stufe eine eigene, unabhängig prüfbare Aussage liefert – so lässt sich ein Fehlschlag eindeutig zuordnen, statt die gesamte Kette in Frage zu stellen.

Stufe 1 – Privilegierter Lehrer-Agent. Ein zweischichtiges MLP wird mit PPO auf der vierdimensionalen Raycast-Beobachtung nach Gleichung (3.3) trainiert. Der Agent konvergiert in rund zwölf Minuten und fährt anschließend elf fehlerfreie Runden bei einem Durchschnittstempo von etwa 1,2 m/s.

Prüfkriterium: Erreicht der Lehrer die volle Episodenlänge ohne Crash, sind Physik, Belohnungsfunktion, Streckengeometrie und Trainingspipeline lösbar. Erst dann ist eine Kameravariante sinnvoll.

Stufe 2 – Datenaufzeichnung mit Rausch-Injektion. Der Lehrer fährt die Strecke unter additivem gaußschem Rauschen auf seine Aktionen (Gleichung (2.9)). Das Fahrzeug driftet dadurch von der Ideallinie ab, und der Lehrer muss aktiv gegensteuern. Aufgezeichnet werden Paare aus visuellem Dreierstapel und der *unverrauschten* Korrekturaktion. Der Datensatz enthält dadurch Rettungsmanöver, die eine perfekte Vorführung nie erzeugen würde.

Prüfkriterium: Der Datensatz muss Zustände abseits der Ideallinie enthalten; ein Datensatz ohne Abweichungen erzeugt einen Schüler, der beim ersten eigenen Fehler ausfällt.

Stufe 3 – Visuelles Vortraining. Ein CNN lernt überwacht, die Expertenaktionen aus dem Bildstapel vorherzusagen (Gleichung (2.8)). Erreicht wird ein MSE von 0,0034. Übertragen werden anschließend ausschließlich die drei Faltungsschichten; die Linear-Schichten werden verworfen.

Prüfkriterium: Hier ist besondere Vorsicht geboten. Ein niedriger MSE belegt *nicht*, dass das Netz fahren kann – in diesem Projekt fuhr kein einziges vortrainiertes Netz eigenständig mehr als 230 Frames. Was der Schritt liefert, ist Sehfähigkeit, nicht Fahrwissen. Ob er seinen Aufwand rechtfertigt, ist für SAC bis heute nicht belegt (Abschnitt 3.3).

Stufe 4 – Online-Feintuning mit SAC. Der so initialisierte Agent wird mit SAC auf der Bildbeobachtung weitertrainiert. Maßgeblich sind dabei die in Tabelle 6.1 zusammengefassten Einstellungen; sie stammen aus dem aktuellsten großen Trainingslauf und sind die Konfiguration, die für Phase 2 übernommen werden soll.

Tabelle 6.1: Konfiguration der aktuellen Pipeline-Stufe 4, entnommen aus `train_config.json` des Laufs `09_Vision_SAC_Finetuned_20260820_2215`.

Bereich	Einstellung	Begründung
Beobachtung	3 Graustufenbilder, 250 × 150, aufeinanderfolgend	Auflösung senkt die Wahrnehmungsschwelle auf 0,42 m/s; Spreizung wurde geprüft und verworfen
Perspektive	globale Vogelperspektive, 4 Graustufen	entspricht der Informationslage eines menschlichen Fahrers
Aktion	[Gas, Bremse, Lenken], $\Delta\delta \leq 0{,}5$ je Frame	entspricht dem realen Fahrzeug, verhindert binäres Stellverhalten
Algorithmus	SAC, `CnnPolicy`	off-policy, dadurch rund 13-fach mehr Gradientenschritte je Frame als PPO
Lernrate	5×10^{-5}	höhere Werte zerstören das vortrainierte Backbone
γ	0,99 ($\hat{=}$ 3,3 s)	0,999 führte zum Kollaps (Abschnitt 3.2.10)
Puffer	500 000, `FramestapelReplayBuffer`	Einzelbildspeicherung statt fertiger Stapel
Takt	`train_freq` 2, `gradient_steps` 1, `batch_size` 512	
Zeitlimit	2000 Frames, `handle_timeout_termination=True`	Zeitlimit ist kein Endzustand
Startlage	Streuung 0,9 m längs, 0,08 m quer	verhindert auswendig gelernte Trajektorien
Auswertung	10 Episoden alle 10 000 Schritte	

6.2 Quantitative Ergebnisse

Anmerkung zur Vergleichbarkeit

Die zentrale methodische Einsicht dieses Projekts betrifft die Frage, welche Zahlen überhaupt nebeneinander gestellt werden dürfen. Drei Einschränkungen sind zu beachten:

1. **Über Physikstände hinweg ist kein Zahlenvergleich zulässig.** Läufe unter `force_drag_v1` fuhren ein rund zwanzigfach schnelleres Fahrzeug. Ihre Werte – einschließlich des Projektbestwerts von 3828,2 aus Lauf `1432` – sind mit Läufen unter `measured_v2` nur qualitativ vergleichbar.
2. **Über Belohnungsversionen hinweg ebenso wenig.** Der Rundenzeit-Bonus feuerte vom 28.07. bis zum 17.08.2026 nie (Abschnitt 3.4). Nach seiner Reparatur liegen die Belohnungswerte derselben Modelle deutlich höher.
3. **Auch dieselbe Auswertung liefert nicht zwangsläufig dieselben Zahlen.** Die Nachauswertung in Tabelle 6.2 weist für mehrere Modelle andere Episodenlängen und Rundenzahlen aus als die frühere Messung in Tabelle 3.6 – für Lauf `2210` etwa 759 statt 256 Frames. Als Ursachen kommen die reparierte Belohnungsfunktion, zwischenzeitliche Änderungen an der Umgebung und das Auswertungsgerät in Frage; Letzteres ist nachweislich hinreichend, um eine Episode vollständig zu verändern (Abschnitt 3.4). Welcher Anteil überwiegt, ist **nicht abschließend geklärt**. Für diesen Bericht gilt daher die Regel, ausschließlich Werte aus *demselben* Auswertungslauf miteinander zu vergleichen.

Aus diesem Grund werden im Folgenden zwei Größen bevorzugt, die über Belohnungsänderungen hinweg tragen: die Zahl vollständiger Runden und das Durchschnittstempo.

Nachauswertung aller Modelle

Tabelle 6.2 zeigt die aktuellste Nachauswertung. Alle Modelle unter `measured_v2` wurden dafür in derselben Umgebung, mit derselben Belohnungsversion und auf demselben Gerät neu gefahren.

Aus dieser Zusammenstellung lassen sich vier Aussagen ableiten.

Die Aufgabe ist lösbar, und zwar sicher. Vier unabhängig trainierte Lehrer-Agenten erreichen reproduzierbar elf Runden bei voller Episodenlänge – unabhängig vom Algorithmus, denn sowohl

Tabelle 6.2: Nachauswertung aller Modelle unter `measured_v2` im aktuellen Codestand, je drei deterministische Episoden vom festen Startpunkt (`models/vergleich.json`). Die Rundenzahl zählt die erste Zielüberquerung als Start und nicht als Runde.

Modell	Beobachtung	Reward	Frames	Runden	∅ Tempo
Privilegierte Beobachtung					
Lidar-Experte `2236` (PPO)	exakt	6711,7	2000	11	1,23 m/s
Lidar-Experte `2223` (PPO)	exakt	6643,6	2000	11	1,22 m/s
Lidar-Experte `1800` (PPO)	exakt	6625,0	2000	11	1,22 m/s
Lidar SAC `1748` (Kontrolle)	exakt	6561,0	2000	11	1,19 m/s
Lidar-Experte `1757` (PPO)	exakt	5093,9	2000	10	1,04 m/s
Bildbeobachtung					
Vision SAC `2254`	250×150	5080,5	2000	9	1,05 m/s
Vision SAC `2210`	166×100	1516,6	759	3	1,00 m/s
Vision SAC `2230`	250×150	1506,2	769	3	0,97 m/s
Vision SAC `2147`	166×100	−153,7	209	0	0,73 m/s
Vision SAC `2331` ($\gamma = 0{,}999$)	166×100	−349,8	33	0	0,12 m/s
Vision SAC `2307` (gespreizt)	250×150	−439,5	76	0	0,22 m/s

PPO als auch SAC lösen sie. Die Streuung zwischen den Läufen betrifft nur das Tempo (1,04 m/s bis 1,23 m/s), nicht das grundsätzliche Gelingen.

Bildbeobachtung kostet mindestens den Faktor 70 an Aufwand. Der Lehrer erreicht die volle Episodenlänge nach 50 000 Schritten und über 5000 Belohnungspunkte nach 70 000 Schritten. Der beste Kamera-Agent benötigt dafür Millionen von Schritten und Stunden bis Tage an Rechenzeit. Der Unterschied zwischen zwölf Minuten und über zwanzig Stunden ist der Preis dafür, aus Pixeln zu lernen.

Der scheinbar beste Kamera-Agent ist der unehrlichste Messwert. Lauf `2254` steht mit 5080,5 Punkten und neun Runden an der Spitze der Bildbeobachtungen. Er wurde jedoch mit festem Startpunkt trainiert und wird hier vom selben festen Startpunkt ausgewertet. Aus zufälligen Startlagen fällt derselbe Agent auf −1829,2 Punkte und 0,6 Runden (Tabelle 3.11). Die neun Runden sind eine abgespulte Trajektorie, keine Fahrfähigkeit. Der mit streuendem Startpunkt trainierte Lauf `2215` erreicht 1545,7 Punkte und 3,1 Runden aus zufälligen Lagen – deutlich weniger auf dem Papier, aber das erste ehrliche Ergebnis der Reihe.

Beide untersuchten Hebel gegen die Wahrnehmungsschwelle sind vermessen. Die Auflösungserhöhung wirkt: Der Ertrag je Frame stieg von 1,18 auf 2,01, und die zum Erreichen einer

positiven Bilanz nötige Episodenlänge sank von 465 auf 282 Frames. Das temporale Spreizen wirkt nicht: Lauf 2307 liegt bei jeder Schrittzahl hinter dem Vergleichslauf und erreicht in 150 Auswertungen keinen einzigen positiven Wert.

Was die 2D-Untersuchungen belegt haben

Zusammengefasst liefert Phase 1 folgende belegte Aussagen:

1. **Der Engpass ist die Wahrnehmung, nicht der Algorithmus.** Der Kontrollversuch schließt Physik, Belohnungsfunktion, Algorithmus, Replay-Buffer, Zeitlimitbehandlung und Diskontfaktor aus; die Ablationsstudie grenzt die Ursache weiter auf das Geschwindigkeitssignal ein.
2. **Die Wahrnehmungsschwelle ist vorab berechenbar.** Gleichung (3.7) liefert sie aus Auflösung, Maßstab und Bildrate, ohne dass ein einziger Trainingslauf nötig wäre.
3. **SAC ist auf Bilddaten PPO deutlich überlegen** – bei gleicher Schrittzahl ebenso wie bei gleichem Zeitbudget.
4. **Der Startpunkt muss streuen**, sonst misst die Auswertung Wiederholbarkeit statt Fahrfähigkeit.
5. **Eine feste Terminalstrafe macht die Belohnungskennzahl bimodal** und damit über weite Strecken des Trainings uninformativ.
6. **Vortraining überträgt Sehfähigkeit, nicht Fahrwissen.** Der praktische Nutzen ist beobachtet, der Beleg für SAC steht aus.

6.3 Übertragbarkeit in die 3D-Simulation

Die abschließende Frage dieses Kapitels lautet, ob die entwickelte Pipeline in der 3D-Simulation anwendbar ist oder dort versagt. Eine gemessene Antwort setzt Läufe in Isaac Sim voraus und kann daher an dieser Stelle noch nicht gegeben werden. Was sich bereits sagen lässt, ist, *woran* die Antwort entschieden wird – und das ist selbst ein Ergebnis von Phase 1.

Was unverändert überträgt. Algorithmus, Hyperparameter, Aktionsdefinition, die Struktur der Belohnungsfunktion und der speicheroptimierte Replay-Buffer sind von der Simulationsplattform unabhängig. Ebenso überträgt die Reihenfolge der Inbetriebnahme aus Abschnitt **??**, deren wichtigster Schritt lautet: zuerst die Lidar-Variante trainieren.

Was neu bestimmt werden muss. Sämtliche zeitbasierten Größen hängen an der Fahrzeugdynamik und müssen nachgerechnet werden, sobald PhysX andere Geschwindigkeiten liefert: Diskontfaktor, Episodenlänge, Framestapel, v_{slow} und das Verhältnis von Crash-Strafe zu Fahrertrag.

Das entscheidende Kriterium. Die Wahrnehmungsschwelle nach Gleichung (3.7) ist eine Funktion von Kameraauflösung, Maßstab, Bildrate und Fahrzeuggeschwindigkeit. Sie ist damit *plattformunabhängig* und tritt in Isaac Sim in genau derselben Form erneut auf. Daraus folgt eine vorab prüfbare Aussage: Ergibt die Neuberechnung mit der neuen Kamera und den neuen Geschwindigkeiten wieder Bruchteile eines Pixels je Einzelbild, wird der Kamera-Agent auch dort nicht konvergieren – und zwar unabhängig davon, wie fotorealistisch die Szene ist. Ergibt sie deutlich mehr als einen Pixel, ist die wesentliche Hürde aus Phase 1 in Phase 2 nicht vorhanden.

Das ist das eigentliche Ergebnis dieser Arbeit: nicht ein Agent, der schnell fährt, sondern ein Kriterium, das *vor* dem Training entscheidet, ob er es je können wird.

Tabelle 6.3: Abnahmekriterien für die 3D-Umsetzung. Sie sind aus den Ergebnissen von Phase 1 abgeleitet und in dieser Reihenfolge zu prüfen; die Messwerte trägt Phase 2 nach.

Nr.	Kriterium	Ergebnis
1	Vollgas- und Bremsversuch in Isaac decken sich mit der Videomessung	*offen*
2	Wahrnehmungsschwelle nach Gl. (3.7) liegt deutlich unter dem Arbeitsbereich	*offen*
3	Rundenzählung feuert nur bei vollständigen Runden	*offen*
4	Startlagen streuen, Auswertungsstreuung ist größer als null	*offen*
5	Lidar-Agent löst die Aufgabe in vergleichbarer Größenordnung	*offen*
6	Kamera-Agent erreicht eine Episodenlänge oberhalb der Amortisationsschwelle	*offen*

7 Fazit und Ausblick

7.1 Fazit zu Phase 1

Die zu Beginn gestellte Frage lautete, ob sich die konzeptionellen Entscheidungen für einen bildbasierten Rennagenten in einer billigen 2D-Umgebung vorab klären lassen, statt sie in der teuren 3D-Simulation zu erproben. Diese Frage ist zu bejahen – allerdings anders, als zu Projektbeginn erwartet.

Erwartet worden war eine Rangfolge von Beobachtungsformen und Algorithmen. Tatsächlich geliefert hat die 2D-Phase etwas Grundsätzlicheres: ein *Ausschlusskriterium*. Die Wahrnehmungsschwelle nach Gleichung (3.7) entscheidet vor dem ersten Trainingsschritt, ob eine gewählte Kombination aus Auflösung, Maßstab, Bildrate und Fahrzeugdynamik überhaupt Bewegungsinformation transportiert. Sie ist eine Zeile Rechnung, sie ist plattformunabhängig, und ihre Verletzung war in diesem Projekt die Ursache für mehrere Wochen erfolgloser Trainingsläufe. Dieses Kriterium vorab anzuwenden, ist der unmittelbarste praktische Nutzen dieser Arbeit.

Die drei eingangs formulierten Ziele sind damit wie folgt erreicht:

Schnelles 2D-Prototyping. Erreicht. Die Umgebung erlaubt einen vollständigen Lehrer-Lauf in zwölf Minuten und einen bildbasierten Lauf in Stunden statt Tagen. Wesentlich wichtiger als die absolute Geschwindigkeit erwies sich jedoch die Möglichkeit, *Kontrollversuche* zu fahren: Der Austausch einer einzigen Komponente bei sonst konstantem Aufbau hat in 54 min eine Frage beantwortet, an der zuvor wochenlang gearbeitet worden war.

Reward Engineering für stabiles Fahren. Teilweise erreicht. Die Belohnungsfunktion in der Fassung `v3_rundenzeit` ist nachweislich lösbar – der Lehrer-Agent erreicht damit elf fehlerfreie Runden. Vier Fehlerklassen wurden reproduziert, verstanden und behoben. Zugleich zeigte sich die grundsätzliche Schwäche des Entwurfs: Eine feste Terminalstrafe macht die Kennzahl bimodal und damit über weite Strecken des Trainings uninformativ. Eine mit der Restzeit skalierende Strafe oder ein reiner Fortschritts-Reward wären die naheliegenden

Alternativen und sind nicht erprobt.

Physikalisch präzises Fundament. Erreicht. Das Fahrzeugmodell `measured_v2` stammt aus einer Videomessung mit ausgewiesener Belastbarkeit – die Bremsverzögerung ist gut belegt, die Höchstgeschwindigkeit ist ausdrücklich eine Annahme. Diese Umstellung war zugleich der teuerste und der wertvollste Einzelschritt des Projekts: Sie entwertete sämtliche vorherigen Ergebnisse und deckte dabei das eigentliche Problem auf.

Die ehrliche Bilanz enthält auch das, was *nicht* erreicht wurde. Es existiert bis heute kein Kamera-Agent, der unter realistischer Physik und aus zufälligen Startlagen konstant Runden fährt; der beste ehrliche Wert liegt bei 3,1 Runden. Ein Agent, der menschliche Rundenzeiten schlägt, ist damit deutlich entfernt. Ob das Vortraining seinen Aufwand rechtfertigt, ist für SAC nicht belegt. Warum SAC-Läufe nach ihrem Höhepunkt einbrechen, ist beschrieben, aber nicht erklärt. Die vollständige Liste der noch zu erbringenden Belege führt Tabelle **??**.

Bemerkenswert ist rückblickend, wie viele der gewonnenen Erkenntnisse *negative* Ergebnisse sind: das gescheiterte γ-Experiment, der wirkungslose gespreizte Framestapel, das vortrainierte Netz, das nicht fahren kann, und die als Auswendiglernen entlarvten Spitzenwerte. Für die Fortsetzung des Vorhabens sind sie wertvoller als die guten Zahlen aus der Anfangsphase, die sich teilweise als Artefakt eines fehlerhaften Fahrzeugmodells erwiesen haben.

7.2 Fazit zu Phase 2

Phase 2 ist zum Redaktionsschluss in Bearbeitung. Ein belastbares Fazit über die 3D-Umsetzung setzt Messungen in dieser Umgebung voraus und wird von dem für sie verantwortlichen Projektmitglied nachgetragen. Die Grundlage dafür ist vollständig vorbereitet: Die zu übernehmenden Bestandteile, die neu zu bauenden Bestandteile, die Reihenfolge der Inbetriebnahme und die Abnahmekriterien sind in Abschnitt **??** und Tabelle 6.3 festgehalten.

Die aus Phase 1 abgeleitete Prognose lautet: Die Aufgabe ist mit privilegierter Beobachtung lösbar, die Wahrnehmungsschwelle tritt in identischer Form erneut auf und ist vor dem ersten Training zu prüfen, und die Sprödigkeit der Policy gegenüber minimalen Bildunterschieden wird durch Beleuchtung, Schattenwurf und Renderrauschen eher zunehmen. An diesen drei Punkten wird sich messen lassen, ob die entwickelte Pipeline überträgt.

7.3 Ausblick: Phase 3 – Sim-to-Real-Transfer

Das mögliche Anschlussprojekt hat ein klar benennbares Ziel: einen in Simulation trainierten Agenten auf die reale Carrera-Hybrid-Bahn zu bringen. Die dabei zu überbrückende Lücke zwischen Simulation und Wirklichkeit – der *Sim-to-Real-Gap* – ist in diesem Projekt bereits messbar geworden, wenn auch an einer unerwarteten Stelle.

Der Befund, der den Ausblick bestimmt. Zwischen den beiden verwendeten Renderpfaden unterscheiden sich 0,05 % der Pixel – nichts weiter als die Kantenglättung des Fahrzeug-Sprites. Das genügte, um die Ausgabe der Policy vollständig umzuwerfen: Die Lenkvorgabe kippte von $+0{,}30$ auf $-0{,}80$, und derselbe Agent, der im einen Pfad 6,9 m fuhr, bewegte sich im anderen keinen Zentimeter. Erklärbar ist das über die Größenordnung – die acht abweichenden Pixel entsprechen ungefähr dem gesamten Fahrzeug in der Beobachtung.

Für den Realtransfer ist dieser Befund vernichtend, und genau deshalb ist er wertvoll. Wenn bereits die Antialiasing-Kante des eigenen Renderers ausreicht, um die Policy lahmzulegen, dann hat eine reale Kamera mit Beleuchtungsschwankungen, Schattenwurf, Sensorrauschen, Bewegungsunschärfe und Übertragungslatenz keine Chance. Die Sprödigkeit ist damit kein Randproblem des Transfers, sondern seine zentrale Herausforderung.

Die Gegenmaßnahme: einen fahrenden Agenten mit Störungen bombardieren. Der vorgezeichnete Weg besteht darin, einen bereits fahrfähigen Agenten *nachträglich* robust zu machen, statt Robustheit von Anfang an zu erzwingen. Konkret bedeutet das, die Beobachtung während des Weitertrainierens systematisch zu stören:

- **Beobachtungs-Augmentierung:** additives Rauschen, Verschiebungen um wenige Pixel, zufällige Zuschnitte, Helligkeits- und Kontraständerungen. Verfahren wie *DrQ* und *RAD* sind dafür etabliert und in diesem Projekt bislang *vollständig ungenutzt*.
- **Domain Randomization:** Randomisierung von Reibwerten, Beleuchtung, Kameraposition und -kalibrierung, Fahrzeugmasse und Schwerpunktlage über die Trainingsläufe hinweg.
- **Latenz und Aktorträgheit:** Modellierung der Funkübertragungsverzögerung und der realen Servostellzeit, die im 2D-Modell nur als Begrenzung der Lenkänderungsrate näherungsweise enthalten ist.

Die Steigerungsstufe dieses Gedankens ist der Übergang von der Störung zur *Rettung*: Ein Agent soll nicht nur kleine Abweichungen verkraften, sondern aus grob gestörten Zuständen

zurückfinden. Genau das ist die Aussage von VISTA [10], und genau dafür existiert in diesem Projekt bereits die Grundlage – der Lidar-Experte ist in *jedem* Zustand abfragbar und kann daher zu jeder gestörten Lage die richtige Rettungsaktion liefern. Damit ist auch der Weg zu *DAgger* offen, dem theoretisch sauberen Gegenmittel gegen den Covariate Shift, das in Phase 1 aus Zeitgründen nicht mehr erprobt wurde.

Weitere Anknüpfungspunkte. Über den Transfer hinaus bleiben aus Phase 1 mehrere Fäden offen, die sich mit vergleichsweise geringem Aufwand aufnehmen lassen:

- Ein SAC-Lauf *ohne* Backbone-Transplantation, um endlich zu belegen oder zu widerlegen, dass das Vortraining seinen Aufwand wert ist.
- Die Vorbefüllung des Replay-Buffers mit vollständigen Experten-Übergängen. Da SAC off-policy lernt, kann es direkt aus fremden Übergängen lernen – ein Weg, der im Gegensatz zur reinen Backbone-Übertragung auch Fahrwissen transportieren würde.
- Die Aufnahme der Geschwindigkeit als Skalarwert in die Beobachtung. Die Ablationsstudie legt nahe, dass dies die wirksamste Einzelmaßnahme wäre; sie verändert allerdings die Aufgabenstellung und wäre gegen das Ziel abzuwägen, dem Agenten dieselbe Informationslage zu geben wie einem menschlichen Fahrer.
- Der offene Vergleichslauf PPO mit niedriger Lernrate und Backbone, vorbereitet auf dem Branch `ppo-vergleich`.
- Die Klärung, wie ein realer Fahrtregler auf gleichzeitiges Gas und Bremsen reagiert. Der Agent nutzt beides in über der Hälfte der Frames; das Modell nimmt Superposition an, und das ist eine Annahme, keine Messung. Zehn Sekunden Videoaufnahme mit beiden Triggern würden genügen.

Abschließende Einordnung. Das übergeordnete Ziel – ein Agent, der ein reales Carrera-Hybrid-Fahrzeug allein anhand eines Deckenkamerabildes schneller über die Bahn steuert als ein Mensch – bleibt bestehen und ist nicht erreicht. Was in Phase 1 entstanden ist, ist die Vorarbeit dafür: eine kalibrierte Umgebung, eine geprüfte Trainingspipeline, ein Satz reproduzierter Fehlerquellen und vor allem eine Reihe von Kriterien, die vor dem Training entscheiden, ob ein Aufbau überhaupt tragfähig ist. Dass ein erheblicher Teil dieser Erkenntnisse aus Fehlschlägen stammt, mindert ihren Wert nicht – es ist die übliche Form, in der sie anfallen.

Quellenverzeichnis

[1] yoshtm, *YouTube-Kanal yoshtm – bildbasierte Reinforcement-Learning-Agenten für Trackmania*, `https://www.youtube.com/@yoshtm`, abgerufen am 11.09.2026.

[2] S. Lange, M. Riedmiller und A. Voigtländer, „Autonomous reinforcement learning on raw visual input data in a real world application," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2012.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus und N. Dormann, „Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, Jg. 22, Nr. 268, S. 1–8, 2021.

[4] *The FASTEST introduction to Reinforcement Learning on the internet*, YouTube-Video, veröffentlicht am 23.12.2024. `https://www.youtube.com/watch?v=VnpRp7ZglfA`, abgerufen am 11.09.2026, 2024.

[5] R. S. Sutton und A. G. Barto, *Reinforcement Learning: An Introduction*, 2. Aufl. Cambridge, MA, USA: MIT Press, 2018.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford und O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.

[7] T. Haarnoja, A. Zhou, P. Abbeel und S. Levine, „Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Ser. Proceedings of Machine Learning Research, Bd. 80, 2018, S. 1861–1870.

[8] S. Fujimoto, H. van Hoof und D. Meger, „Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Ser. Proceedings of Machine Learning Research, Bd. 80, 2018, S. 1587–1596.

[9] S. Ross, G. J. Gordon und J. A. Bagnell, „A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Ser. Proceedings of Machine Learning Research, Bd. 15, 2011, S. 627–635.

[10] A. Amini, I. Gilitschenski, J. Phillips, J. Moseyko, R. Banerjee, S. Karaman und D. Rus, „Learning Robust Control Policies for End-to-End Autonomous Driving From Data-Driven Simulation," *IEEE Robotics and Automation Letters*, Jg. 5, Nr. 2, S. 1143–1150, 2020.

[11] D. Chen, B. Zhou, V. Koltun und P. Krähenbühl, „Learning by Cheating," in *Proceedings of the Conference on Robot Learning (CoRL)*, Ser. Proceedings of Machine Learning Research, Bd. 100, 2019, S. 66–75.

[12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg und D. Hassabis, „Human-level control through deep reinforcement learning," *Nature*, Jg. 518, S. 529–533, 2015.

[13] X. Fang, Q. Zhang, Y. Gao und D. Zhao, „Offline Reinforcement Learning for Autonomous Driving with Real World Driving Data," in *IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, 2022.

[14] P. Cai, H. Wang, H. Huang, Y. Liu und M. Liu, „Vision-Based Autonomous Car Racing Using Deep Imitative Reinforcement Learning," *IEEE Robotics and Automation Letters*, Jg. 6, Nr. 4, S. 7262–7269, 2021.

[15] P. R. Wurman, S. Barrett, K. Kawamoto, J. MacGlashan, K. Subramanian, T. J. Walsh, R. Capobianco, A. Devlic, F. Eckert, F. Fuchs, L. Gilpin, P. Khandelwal, V. Kompella, H. Lin, P. MacAlpine, D. Oller, T. Seno, C. Sherstan, M. D. Thomure, H. Aghabozorgi, L. Barrett, R. Douglas, D. Whitehead, P. Dürr, P. Stone, M. Spranger und H. Kitano, „Outracing champion Gran Turismo drivers with deep reinforcement learning," *Nature*, Jg. 602, S. 223–228, 2022.

[16] P. S. Chib und P. Singh, *Recent Advancements in End-to-End Autonomous Driving using Deep Learning: A Survey*, arXiv:2307.04370, 2023.

[17] E. Delavari, F. K. Khanzada und J. Kwon, *A Comprehensive Review of Reinforcement Learning for Autonomous Driving in the CARLA Simulator*, arXiv:2509.08221, 2025.

[18] A. O. Ly und M. Akhloufi, „Learning to Drive by Imitation: An Overview of Deep Behavior Cloning Methods," *IEEE Transactions on Intelligent Vehicles*, Jg. 6, Nr. 2, S. 195–209, 2021.

[19] B. Peng, Q. Sun, S. E. Li, D. Kum, Y. Yin, J. Wei und T. Gu, „End-to-End Autonomous Driving Through Dueling Double Deep Q-Network," *Automotive Innovation*, Jg. 4, S. 328–337, 2021.

[20] A. Khlifi, M. Othmani und M. Kherallah, „A Novel Approach to Autonomous Driving Using Double Deep Q-Network-Based Deep Reinforcement Learning," *World Electric Vehicle Journal*, Jg. 16, Nr. 3, S. 138, 2025.

[21] J. Crewe, A. Humnabadkar, Y. Liu, A. Ahmed und A. Behera, „SLAV-Sim: A Framework for Self-Learning Autonomous Vehicle Simulation," *Sensors*, Jg. 23, Nr. 20, S. 8649, 2023.

[22] Z. Huang, J. Wu und C. Lv, *Efficient Deep Reinforcement Learning with Imitative Expert Priors for Autonomous Driving*, arXiv:2103.10690, 2021.

[23] R. Chekroun, M. Toromanoff, S. Hornauer und F. Moutarde, „GRI: General Reinforced Imitation and Its Application to Vision-Based Autonomous Driving," *Robotics*, Jg. 12, Nr. 5, S. 127, 2023.

[24] S. Mohammed, A. Argun, N. Bonnotte und G. Ascheid, *An Examination of Offline-Trained Encoders in Vision-Based Deep Reinforcement Learning for Autonomous Driving*, arXiv:2409.10554, 2024.

[25] NVIDIA, *Isaac Lab – Dokumentation*, `https://isaac-sim.github.io/IsaacLab/`, abgerufen am 27.10.2026.

[26] Hikan (@cerd1232), *Porsche 911 GT2 RS Weissach 2018*, 3D-Modell auf Sketchfab, veröffentlicht am 01.09.2021, Lizenz CC BY 4.0. `https://sketchfab.com/3d-models/porsche-911-gt2-rs-weissach-2018-35d042abed2741899acbb6f538529734`, abgerufen am 27.10.2026, 2021.

[27] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba und P. Abbeel, „Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, S. 23–30.